CLASSIFICATION OF ANCIENT LANGUAGES SIGNS USING MACHINE LEARNING

A Thesis
Submitted to the College of Information Engineering at Al-Nahrain University in Partial Fulfillment of the Requirements for the Degree of

MASTER OF SCIENCE
IN
SYSTEMS ENGINEERING

by
Elaf Ahmed Saeed Abdul-Jabbar
(B.Sc. in Systems Engineering 2018)

DHU AL-HIJJAH **1445**
JUNE **2024**

Certification

I certify that this thesis entitled "**Classification Of Ancient Languages Signs Using Machine Learning**" was prepared by **Elaf Ahmed Saeed** under my supervision at Al-Nahrain University/ College of Information Engineering in partial fulfillment of the requirements for the degree of **Master of Science in Systems Engineering.**

Signature:

Name: **Asst. Prof. Dr.Ammar D. Jasim**

(Supervisor)

Date: 26/ 6 / 2024

Signature:

Name: **Prof. Dr. Munther A. Abdul Malik**

(Supervisor)

Date: 26 / 6 / 2024

Signature:

Name: **Asst. Prof. Dr.Ammar I. Majeed**

(Head of Department)

Date: 26 / 6 / 2024

Certification

We certify, as an examining committee, that we have read this thesis entitled "**Classification Of Ancient Languages Signs Using Machine Learning**", examined the student **Elaf Ahmed Saeed** in its contents and found that it is adequate as a thesis for the degree of **Master of Science in Systems Engineering.**

Signature:
Name: **Prof. Dr. Ban N. Dhannoon**
(Chairman)
Date: 26/ 6 / 2024

Signature:
Name: **Asst. Prof. Dr.Ammar I. Majeed**
(Member)
Date: 26 / 6 / 2024

Signature:
Name: **Asst. Prof. Dr. Hamsa A. Abdullah**
(Member)
Date: 26 / 6/ 2024

Signature:
Name: **Asst. Prof. Dr.Ammar D. Jasim**
(Member& Supervisor)
Date: 26/ 6 / 2024

Signature:
Name: **Prof. Dr. Munther A. Abdul Malik**
(Member& Supervisor)
Date: 26/ 6 / 2024

Approved by the College of Information Engineering:

Signature: Hikmat
Name: **Prof. Dr. Hikmat N. Abdullah**
(Dean)
Date: 27 / 6 / 2024

ABSTRACT

Cuneiform writing offers valuable insights into the remote epochs of human history. Nevertheless, interpreting ancient clay tablets is time-consuming and requires extensive knowledge acquired over several years. The identification of languages represented by cuneiform symbols is a significant challenge due to limited resources and the issue of tokenization.

This thesis used ten machine learning techniques, including supervised, ensemble, and instance-based learning, as well as one natural language processing algorithm(NLP) called n-gram, to identify the cuneiform dialect. So, to detect the signs in the Assyrian cuneiform tablets, the You Only Look Once version 8(Yolov8) object detection pretraining model is used.

Ensembles produce the best results of the Random Forest classifier, working on features at the character level, with a macro averaged F1 score of 96%, and the best outcome for the n-grams algorithm is 82% of di-gram. The best mAP50 reached 82.7%, a precision of 71.319%, and a recall of 85.647%—computer-assisted Assyrian stroke detection in images of cuneiform tablets collected from the Iraqi Museum. Thus, 22 letters were identified and reached the best mean Average Percentage of 50 (mAP50 reached 92%, a precision of 93.2%, and a recall of 89.8%). For the classification, the Cuneiform tablets are collected from the Iraqi Museum and Cuneiform Digital Library Initiative (CDLI) website and reach approximately more than 1000 images in each language, the accuracy of top1 97% and top5 100%.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

AdaBoost	Adaptive Boosting
ANN	Artificial Neural Network
API	Application Programming Interface
AP	Average Precision
BCE	Before Christian Era
C2E	Cuneiform to English
C2F	Cross-stage Partial Bottleneck with Two Convolutions Layers
CDLI	Cuneiform Digital Library Initiative
CLIR	Cross-Language Information Retrieval
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CSP	Cross-Stage Partial
CSV	Comma-Separated Values
CLI	Cuneiform Language Identification
CR	Character Recognition
DL	Deep Learning
DNN	Deep Neural Network
DBN	Deep Belief Networks
DBM	Deep Boltzmann Machines
DT	Decision Tree
FN	False Negative
FP	False Positive
FPN	Feature Pyramid Networks
GB	Gradient Boost
gTTS	Google Text-To-Speech
IOU	Intersection Over Union
KNN	K-Nearest Neighbors
LM	Language Model
LR	Logistic Regression
LTB	Late Babylonian
mAP	Mean Average Percentage
ML	Machine Learning
MAE	Mean Absolute Error
mP	Mean Precision
MSE	Mean Square Error
NB	Naïve Bayes
NLP	Natural Language Processing
NEA	Neo-Assyrian

NMS	Non-Maximum Suppression
OLB	Neo-Babylonian
Orrac	Open Richly Annotated Cuneiform Corpus
RBM	Restricted Boltzmann Machines
RMSE	Root Mean Square Error
RPN	Region Proposal Network
RF	Random Forest
ROI	Regions of Interest
RCNN	Region-Based Convolutional Neural Networks
SSD	Single Shot Detector
SVM	Support Vector Machine
SUX	Sumerian
T2E	Transliteration to English
TP	True Positive
TN	True Negative
TTX	Text to Speech
VOC	Visual Object Classes
XGBoost	Extreme Gradient Boosting
YOLO	You Only Look Once

LIST OF SYMBOLS

P	Probability
P	Precision
R	Recall
F1	F1-Score
$\hat{y}_i$	Predicted Value
y_i	Actual Value
w	Word
N^o	Number of Output Neurons
N^h	Number of Hidden Neurons
N^i	Number of Output Neurons
n	Number of Samples
η	Learning Rate
L_{train}	Bounding Box Regression Loss
$L_{obj}.$	Objectness Loss
L_{cls}	Classification Loss.
y_i	Ground Truth Objectness Score
p_i	Predicted Objectness Score.
p_{ic}	Predicted Probability Of Input I Being Class C
B_i	Predicted Bounding Box
B_i^{gt}	Ground Truth Bounding Box
y_{ic}	ground truth label for class c
p_{ic}	predicted probability for class c
L_{val}	Validation Loss
L_{box}^{val}	Validation Bounding Box Regression Loss
L_{obj}^{val}	Validation Objectness Loss
L_{cls}^{val}	Validation Classification Loss
P_n	precision at the n-th threshold.
R_n	recall at the n-th threshold.
AP_c	Average Precision for class *c*
N	Number Of Points In The Precision-Recall Curve

LIST OF TABLES

LIST OF FIGURES

Chapter 1
INTRODUCTION

1.1 BACKGROUND

From the late fourth millennium before the Christian era (BCE) to the first or second century (CE), Mesopotamia and the surrounding areas extensively employed a logo-syllabic writing system known as the cuneiform script. It was initially created by the Sumerians circa 3200 BCE, and in the 25th and 24th centuries BCE, it was modified to write Sumerian and Akkadian. The use of cuneiform extended throughout the ancient Near East during the ensuing decades and was adapted from Akkadian to write several languages, including Elamite, approximately 2300 BCE, Hurrian, around 2000 BCE, and Hittite, around 1600 BCE [1]. Cuneiform writing in Mesopotamia had several stages of development between the first millennium BC and 600 BC, one of which is represented by the Assyrian cuneiform language. It relies on drilling cuneiform symbols in groupings corresponding to the fundamental language meanings on clay or stone tablets. There are letters in this language. Each letter in cuneiform comprises one or more wedge-shaped symbols arranged in one of four directions: horizontal, vertical, oblique, or diagonal. As a result, the number of symbols, their placement, and their direction vary from one character to the following [2].

Digital humanists, archivists, and archaeologists are interested in the automated annotation of historical documents. In contrast to painstakingly scrolling through and examining a set of photographs, an entirely digitized corpus of documents would allow for speedy search and analysis of historical archives. This would make it possible to gather primary sources for historical research much more swiftly. Beyond its effects on document retrieval, digital historical data can be used

for novel statistical analysis tasks like network analysis [3], the reconstruction of disjointed documents [4], and contemporary NLP tools for translation or decipherment [5] [6] [7].

Richly annotated digital archives could be quickly created using tools that partially automate or assist human annotation. Machine learning algorithms are effective for jobs involving picture analysis inside and outside document recognition. It is possible to use these techniques for the examination of historical documents [8] [9] [10], and the emergence of open-source software tools for creating machine learning products [11] presents this option. However, using machine learning in historical document archives is challenging. The first challenge is the generation of training data.

Most machine learning methods require large amounts of annotated documents to provide training data, which naturally assumes that the majority of the papers that a researcher may be interested in annotating have already been annotated. This has given rise to systems similar to ours that use machine learning methods designed for low-data environments [12] and current datasets to improve the annotation of future papers [13]. Various issues arise with recognition tasks. The first problem is the material used to write on, such as stone or clay, and the result is writing in three-dimensional form. The second concerns the three-dimensionality and geometry of the cuneiform symbol. The third issue is the cuneiform writing style, which can be written on all tablet surfaces rather than just one face [2]. The final issue when reading cuneiform involves unwanted outcomes like spots that appear after being subjected to the segmentation procedure and whose features vary from one image to another and negatively impact the recognition task [14]. Therefore, recent work has been done to confront these problems and create models to facilitate reading and translating cuneiform tablets.

This thesis examines the effects in Hebrew, a non-European language that employs a non-alphabetic writing system. Hebrew, like Arabic, is a Semitic language that

employs an abjad writing system, in which only consonants are shown, and the text is read from right to left. The Hebrew alphabet consists of 22 consonants. Vowels are denoted in two manners: (i) with four letters that can serve as both consonants and vowels and (ii) by a collection of 13 optional vowel marks resembling diacritics. These vowel marks are primarily used in Hebrew works intended for beginners, children's literature, poetry, and religious materials [15], [16]. The objective is to streamline the procedure of deciphering Hebrew characters to assist Scientists in their research endeavors. Our primary objective is to develop a Hebrew letter detector that precisely identifies the location of a sign inside a particular area and accurately determines its pronunciation.

1.2 LITERATURE REVIEW

A group of 2019–2023 studies will be presented that translated and read ancient manuscripts using diverse models and algorithms.

Tommi Jauhiainen et al. 2019 [17]**:** This article contained a corpus of cuneiform texts utilized for the Cuneiform Language Identification (CLI) 2019 shared task. It also described the early language identification tests undertaken using this corpus. The CLI dataset was created from the corpus, and the tests used the n-gram language identification technique. The paper examined the process of creating the dataset, presented the experimental results, and explored the importance of determining the language of individual out-of-domain lines in cuneiform texts.

Ali Adel Saeid et al. (2019) [2]**:** The authors demonstrated that the polygon approximation method is suitable for feature extraction in recognition tasks compared to the elliptic Fourier descriptor. The conclusion is based on the high-accuracy recognition results of applying multiple classes of support vector machine classifiers and their corresponding discriminant functions.

Ehsan Doostmohammadi et al. (2020) [18]**:** They explored the impact of hyperparameters on language identification tasks, comparing the performance of

various methods such as Naive Bayes, Support Vector Machine (SVM), Convolutional Neural Network (CNN), and Recurrent Neural Network (RNN). The study reveals that hyperparameter tuning significantly influences the results, with the ensemble of SVM and Naive Bayes outperforming SVM alone. Additionally, they discussed the use of different input features and their effects on the models' performance, providing insights into optimizing language identification systems.

A. Sahala et al. (2020) [6]**:** The research investigates the automated transcription of Akkadian cuneiform texts using machine learning models. The best-performing models achieved a 90% recall for syllabic tokens and a 69% recall for logograms. The study highlights the complexity of transcribing logograms due to their dependence on contextual information. It demonstrates that context-aware neural models significantly improve transcription accuracy. The successful implementation of these models is a breakthrough in creating computational resources for Akkadian, enhancing the potential for further linguistic analysis and educational applications.

Tobias Dencker et al. 2020 [12]**:** This study demonstrates that deep learning can be effectively applied to cuneiform sign detection, significantly easing the process of reading ancient texts. Using weak supervision through transliterations, the authors developed an iterative training process that progressively improves the sign detector. The approach achieved substantial accuracy improvements, showing a mean average precision (mAP) of 65.6 with semi-supervised learning. The research highlights the potential for integrating this tool into digital humanities projects, aiding Assyriologists in their analysis of cuneiform tablets.

Adéla Hamplová et al. (2022) [19]**:** Found the strokes of cuneiform glyphs in photographs of old tablets. They studied modern computer vision technologies for efficient optical character recognition (OCR). Two-dimensional photos instead of three-dimensional models distinguish this method from others. The availability of publicly accessible web archives with many more 2D photos influenced the choice. The software uses neural image filtering to highlight stroke characters. These edge

filters often decrease the backdrop of appealing objects and accentuate their edges. Use of Hough transformation attributes, specifically lines, and orientation. The neural network architectures YOLOv5 and Detecto accurately identify and locate horizontal strokes in 416x416 pixel cuneiform tablet photos. The Detecto algorithm classifier has 90.5% accuracy but 25% false positives. When applied to cuneiform data, the YOLOv5 classifier is less accurate.

Tabita L. et al. (2022) [20]**:** This study investigates the difficulties encountered by the character recognition (CR) system when analyzing the images of the Great Isaiah Scroll. They generated a novel dataset comprising photos of individual letters extracted from the scroll. This study also effectively evaluated the performance of four CNN models when applied to the dataset. The findings suggested that AlexNet and LeNet-5 are the most appropriate CNN models for accurately identifying handwritten ancient Hebrew letters. These models exhibited consistently low variability in accuracy and loss rate, with a test accuracy of 94% for AlexNet and 96% for LetNet-5.

Shai Gordin et al. (2022) [7]**:** This work aims to provide a novel approach that utilizes NLP techniques to transliterate automatically and segment Unicode cuneiform glyphs. To automatically transliterate and segment conventional Unicode cuneiform glyphs into words, the utilized models were based on machine learning algorithms such as RNN, which achieved an accuracy of up to 97%. In light of this, our approach and the results we obtained constitute a significant step toward developing a human-machine interface for digitized editions. Our code, which is known as Akkademia, is made accessible to the general public through the use of a web application, a Python package, and a repository on GitHub.

Ernst St¨otzner et al. (2023) [21]**:** A new R-CNN architecture approach is proposed to classify and localize pegs. Three-dimensional models of cuneiform tablets of 1977 from the collection of Frau Professor Hilprecht, available as open data, were used. The approach consists of a pipeline of two components: a signal detector and a wedge detector. The signal detector uses a RepPoints model with a ResNet18 backbone to

locate individual cuneiform characters in a tablet section image. The wedge detector is based on the idea of the Point RCNN approach. The results show that the signal detector performs better when trained on 3D views than photographs.

Ernst St¨otzner et al. (2023) [22]**:** The digital tools were developed for processing cuneiform script, a 3D script imprinted into clay tablets used for more than three millennia and at least eight major languages. The HeiCuBeDa and MaiCuBeDa datasets, which consist of around 500 annotated tablets, are created and used. A novel OCR-like approach to mixed image data was provided as an additional mapping tool for transferring annotations between 3D renderings and photographs. The results showed that using rendered 3D images for sign detection performs better than other work on photographs. In addition, the approach gave reasonable results for photographs only, while it is best used for mixed datasets.

Gai Gutherz et al. (2023) [23]**:** The research focuses on translating Akkadian texts into English using neural machine translation techniques. Two main tasks were identified: Cuneiform to English (C2E) and Transliteration to English (T2E). The study showed that high-quality translations can be achieved in both tasks, with a marginal difference between them. The Translation Memory Baseline, a database storing previously translated text segments, was used as a benchmark for evaluating the neural machine translation model's performance. The model outperformed the Translation Memory Baseline, indicating its effectiveness in translating Akkadian texts. The methods included data collection, tokenization, model design, training, and human evaluation. The research contributes to making ancient Akkadian texts more accessible and preserving cultural heritage.

Maha Mahmood et al. (2023) [24]**:** The authors addressed the challenge of recognizing cuneiform symbols using machine learning algorithms on a balanced dataset. They focused on the Cuneiform Language Identification (CLI) dataset, aiming to classify symbols from various cuneiform languages and dialects. By employing SVM, DT, RF, KNN, and DNN algorithms, the researchers demonstrated improvement in

accuracy after implementing oversampling to address class imbalance. The findings highlighted the significance of balanced datasets in enhancing classifier performance for cuneiform symbol classification.

Table 1.1 shows the literature review algorithms and accuracy results.

Table 1.1: The Literature Review

No.	Author/Year	Algorithms	Metrics
1	Tommi Jauhiainen et. al. 2019 [17]	Product of Relative Frequencies Method, Voting Ensemble, HeLI Method, and Sum of Relative Frequencies	F1-Score: 72.06%, 71.63%, 70.61%, and 60.16%
2	Ali Adel Saeid et. al. 2019 [2]	SVM classifiers were utilized with different kernel functions (RBF, polynomial, linear) for the recognition task. The SVM classifier with the RBF kernel function showed the highest accuracy results	The accuracy with three kernels: RBF = 94% Polygonal = 92.5% Linear = 61.3%
3	Ehsan Doostmohammadi et. al. 2020 [18]	RF, LR, NB, SVM, CNN, RNN	The accuracy results: 0.5615, 0.6982, 0.7301, 0.7309, 0.6249, 0.6364
4	A. Sahala et. al. 2020 [6]	Employs a statistical model and three neural encoder-decoder models, with and without context modeling (LSTM).	Open Richly Annotated Cuneiform Corpus (Oracc) containing both transliterations and transcriptions. The best model achieved a 90% recall for syllabic tokens and 69% recall for logograms.
5	Tobias Dencker et. al. 2020 [12]	Utilizes CNNs to detect and classify cuneiform signs. transliteration alignment to generate sign annotations	Achieved 65.6 mAP in detecting over 100 different sign code classes, significantly aiding the study of cuneiform texts.
6	ADELA et. al. 2022 [19]	Yolov5 Detecto	F1-Measure: 42,81 % 81,73 %

7	Tabita L. Tobing et. al. 2022 [20]	CNN (AlexNet, VGG16, ResNet 50, LetNet-5)	The LetNet-5 has the highest accuracy for validation is 97,48%, and for Testing is 97.48%
8	Shai Gordin et. al. 2022 [7]	NLP, (HMM) Hidden Markov Model (MEMM) Maximum-entropy Markov Model (BiLSTM)	The accuracy for three models: 89.5%, 94%, 96.7%
9	E. Stotzner et. al. 2023 [21]	R-CNN architecture to classify and localize wedges, two types of datasets were used HeiCuBeDa and MaiCuBeDa datasets	PaleoCodage and Gottstein measured the mean precision(mP) accuracy of two types of images. Photographs➔0.12 ➔ 0.22 3D renderings➔ 0.34 ➔ 0.36
10	Ernst St¨otzner et. al. 2023 [22]	CNN architecture was used to design OCR and train the model on three datasets.	mAP50 for three datasets: 1- mixed renderings ➔ 0.64 2- VirtualLight renderings ➔ 0.63 3- Photographs➔ 0.57
11	Gai Gutherz et. al. 2023 [23]	Neural machine translation (NMT) and Translation memory baseline.	Best BLEU4 score for C2E and T2E: NMT model : C2E 36.52% & T2E 37.47%. Translation memory baseline: C2E 27.09 & T2E 23.51
12	Maha Mahmood et. al. 2023 [24]	SVM, DT, RF, KNN, and DNN	The accuracy results: SVM (88%), KNN (88%), DT (94%), RF (95%), DNN (93%).
13	Thesis	ML Algorithms + N-gram for feature extraction Yolov8	The RF has the highest accuracy 96%, precision 96%, and recall 96%. Yolov8 Map50 82.7%.

Researchers have not yet achieved the necessary level of precision and dependable translations. Precise identification of cuneiform signs is a necessary skill in the field of cuneiform inscriptions. This skill enables the separation and interpretation of the signs, facilitating their reading.

1.3 RESEARCH GAP

Numerous scholarly investigations have been conducted to decipher the cuneiform script and uncover and interpret its constituent components. However, no comprehensive database is explicitly tailored for scientific studies on artificial intelligence. This contrasts with the databases on platforms like Kaggle, which offer datasets focused on specific linguistic dialects or languages. Numerous scholarly investigations have utilized digital reproductions of Assyrian tablets accessible on the official website of the British Museum.

Consequently, photographs were captured of the Assyrian tablets containing Neo-Assyrian script housed in the Iraqi Museum, and a dedicated database was established just for the Neo-Assyrian script inscribed on these stone tablets.

This also applies to the Hebrew script. We require data collection as no existing image database is readily accessible. A limited selection of photos, including a small proportion, is available online.

1.4 PROBLEM DEFINITION

The difficulties researchers face in establishing a corpus of texts associated with cuneiform and Hebrew scripts are a significant aspect of this challenge. The initial step entails the establishment of a comprehensive visual database encompassing a diverse array of clay and stone tablets. Subsequently, a range of algorithms were employed to accurately identify and locate individual markers, hence enhancing the efficiency of researchers in deciphering the content inscribed on the tablets.

1.5 THE WORK OBJECTIVES AND AIMS

The objectives of the work :

I. Initially utilized four databases. The first component consists of textual data encompassing texts from different dialects. The second and third components comprise pictorial representations of the Assyrian tablets found at the Iraqi Museum and CDLI website. The fourth component consists of pictorial representations of the Hebrew scripts.
II. A natural language processing algorithm and a machine learning algorithm are employed to categorize the many dialects of cuneiform writing.
III. Ultralytics has recently launched yolov8, an advanced and innovative computer vision model. Its purpose was to categorize the cuneiform tablets from Sumerian, Assyrian, and Babylonian civilizations and recognize the symbols in the Neo-Assyrian tablets and Hebrew letters.
IV. Analyzing the visual cues identified in the images and translating them into auditory signals to articulate all the signs and characters found.

This thesis aims to design four models to identify dialects of cuneiform languages, classify cuneiform tablets, and detect signs of Neo-Assyrian signs and Hebrew letters.

1.6 THESIS LAYOUT

This thesis is represented in five chapters. The first chapter contains a background on Coniform writing. The other chapters are as follows:

Chapter 2 presents the theoretical foundations for all system jobs.

Chapter 3 presented the architecture and methodology for the four parts. It also discussed the dataset, labeling tool, machine learning algorithms, and model in depth.

Chapter 4 presents the evaluation and performance results.

Chapter 5 presents the conclusion and the future work.

Chapter 2
THEORETICAL BACKGROUND

2.1 INTRODUCTION

This chapter explains the methods used to define the cuneiform languages and identify each text to which language or dialect it belongs. It also explains the method used to detect cuneiform signs. There are many language identification and signal detection methods. This thesis presents the methods based on ML algorithms for language identification and the Yolov8 object detection pre-training model.

2.2 MACHINE LEARNING (ML)

ML employs algorithms and statistical models of computer systems to perform specific tasks without explicit instructions or direct education [25]. ML is unprogrammed learning. ML teaches machines how to handle data better. Despite witnessing the data, understanding the resultant information is sometimes difficult. That scenario uses ML. The abundance of datasets has increased machine learning demand. Many industries use machine learning to extract essential data. Data-driven machine learning seeks information and insights. Many studies have examined how machines can learn without being programmed [26]. Mathematicians and programmers use several methods to manage large amounts of data for this challenging undertaking. ML uses many methods to solve data problems. Data scientists emphasize that no algorithm performs optimally in all cases. The method's efficacy depends on the problem domain, number of variables, model adequacy, and other considerations [27].

2.2.1 Machine Learning Techniques

As illustrated in Figure 2.1, ML algorithms can be broadly categorized into eight groups: supervised learning, unsupervised learning, semi-supervised learning, reinforcement learning, multi-task learning, ensemble learning, neural networks, and instance-based learning [28].

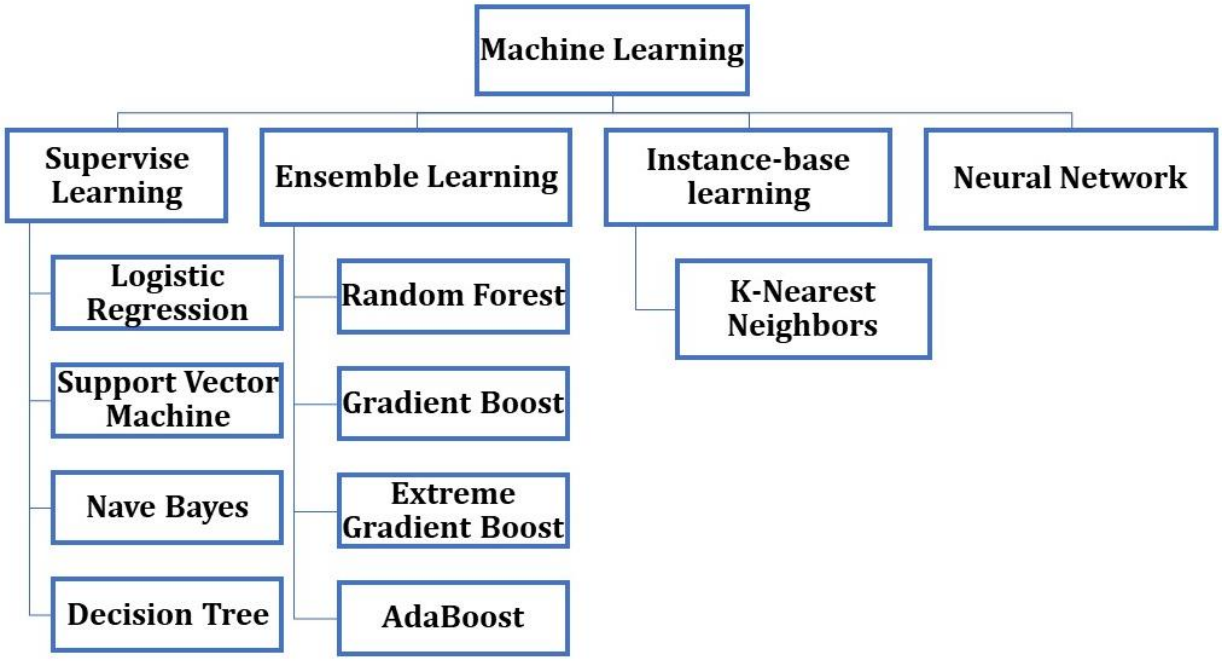


Figure 2.1: Machine Learning Techniques [28]

This thesis uses supervision, ensemble, neural network, and instance-based learning. The ML algorithms are shown in Table 2.1.

Table 2.1: ML Algorithms

Algorithm	Main Characteristics	Advantages	Disadvantages
Logistic Regression (Supervise)	Estimates probabilities using a logistic function. Outputs binary outcomes. Provides interpretable coefficients.Used for binary classification problems. [29]	Probabilistic interpretation of results. It can handle both numerical and categorical features. Less prone to overfitting compared to complex models.	Assumes linear decision boundary may not capture non-linear relationships. Requires large amounts of data for stable estimates. It can be sensitive to irrelevant features [30] [31].

Decision Trees (Supervise)	Constructs a tree structure by partitioning the feature space. Easy to interpret.Handles numerical and categorical data. Prone to overfitting. Can handle missing values [32].	Easy to understand and visualize.Can handle both numerical and categorical data. Automatically handles feature interactions and feature selection [30].	Prone to overfitting, especially with deep trees.High variance, sensitive to small changes in data. Lack of robustness can cause instability with small changes in data.
Support Vector Machines (SVM) (Supervise)	Finds the hyperplane that best separates classes in the feature space. Maximizes the margin between classes.Effective in high-dimensional spaces.Sensitive to the choice of kernel. [33].	It is effective in high-dimensional spaces. Using the kernel trick, it can handle non-linear decision boundaries. It is robust to overfitting, especially in high-dimensional spaces [34].	The choice of kernel and hyperparameters can be challenging. Due to long training times, it is memory-intensive for large datasets and unsuitable for them [35].
Naive Bayes (Supervise)	Based on Bayes' theorem and strong feature independence assumptions. Easy and effective. Works well with tiny datasets and high-dimensional features. Assumes feature independence, which may not be true in real data [31].	Simple and fast algorithm. Performs well with high-dimensional data. Efficient with small training datasets. Handles both numerical and categorical features [32].	Assumes independence between features, which may not hold in real-world data. Limited expressive power compared to more complex models. Sensitive to features with many categories or continuous features.
Neural Networks	Mimics the structure and function of the human brain.Comprises interconnected layers of nodes (neurons). Learns complex patterns and relationships in data. Requires large amounts of data and computational resources. Black box	Capable of learning complex patterns and relationships in data. Suitable for high-dimensional data. Can handle non-linear relationships between features and targets [37].	Requires large amounts of data for training. Prone to overfitting, especially with deep architectures. Black box nature makes interpretation challenging.

	nature interprets challenging [36].		
K-Nearest Neighbors (KNN) (Instance-based learning)	Non-parametric algorithm that classifies instances based on majority voting of k-nearest neighbors. Simple and easy to implement. Sensitive to the choice of distance metric and value of k. Computationally expensive for large datasets [38].	Simple and intuitive concept. Does not make strong assumptions about the underlying data distribution. Effective for nonlinear data and local patterns [39].	Computationally expensive for large datasets, as it requires storing all training instances. Sensitive to the choice of distance metric and value of k. Requires careful data preprocessing [40].
Random Forest (Ensemble Learning)	Ensembles of decision trees reduce overfitting compared to individual decision trees. They provide feature importance, handle high-dimensional data, and are robust to outliers and noise [41].	- Reduced overfitting compared to individual trees. Provides estimates of feature importance. Robust to noise and outliers.Can handle large datasets with high dimensionality [42].	Less interpretable compared to individual decision trees.Slower training and prediction compared to individual trees. Memory-intensive for large datasets [43].
AdaBoost (Ensemble Learning)	Trains weak learners sequentially, emphasizing their faults. A weighted majority vote combines poor learner guesses. Strong learners are less prone to overfitting. Detects noise and outliers.	Effectively mixes poor learners to boost performance. Less likely to overfit than weak learners. It handles numerical and category features. It can reveal feature relevance.	Noisy data and outliers might influence model performance. Large datasets and complex weak learners make it computationally expensive. The learning rate and number of weak learners may need to be adjusted.
Gradient Boosting (GBoost) (Ensemble Learning)	To fix earlier model flaws, sequentially create a weak learner ensemble and Combine weak learner predictions using gradient descent optimization. [44].	Builds powerful predictive models by iteratively correcting errors made by previous models. Provides flexibility in handling different types of data and tasks (regression, classification). Can	It requires careful tuning of hyperparameters, such as learning rate and tree depth. It can be computationally expensive, especially with large datasets and deep trees. If not properly regularized and

		handle missing values and outliers effectively.	validated, it is prone to overfitting.
Extreme Gradient Boosting (XGBoost) (Ensemble Learning)	Improved gradient boosting for efficiency and performance. Regularizes model formalization to reduce overfitting. Uses second-order loss function approximation for faster convergence. [34].	Predicts better than gradient boosting. Scalability comes from built-in parallel and distributed computing.	Needs thorough hyperparameter tuning for best performance. Deep trees and large datasets require memory. Harder to read than decision trees and logistic regression.

Multiple machine learning algorithms were employed to assess the accuracy of each method using the provided data and identify the ones that perform effectively.

2.3 NATURAL LANGUAGE PROCESSING (NLP)

NLP studies and uses computers to understand and manipulate natural language text and speech for various applications. The goal of NLP researchers is to understand how people use language. This knowledge is used to build tools and methods for computer systems to understand and modify natural languages to complete tasks [45]. Computer and information sciences, linguistics, mathematics, electrical and electronic engineering, artificial intelligence and robots, psychology, and others underpin NLP. NLP research and applications span many fields, such as machine translation, natural language processing and summarization, user interface design, speech recognition, AI, and expert systems [46]. N-grams, a part of NLP research, identify cuneiform texts by language.

2.3.1 N-Grams

An N-gram is a sequence of characters or words from a text. N-grams fall into two categories: Character-based and word-based. N-grams are employed in natural

language processing. A character n-gram is n consecutive characters from a word [47]. Character n-grams are manually designed features used in text categorization, authorship attribution [48], authorship verification [49], plagiarism detection [50], spam filtering [51], author native language identification, language variety distinction [52], and many other applications. They also improve classification accuracy in informal text tasks like sentiment analysis that include many unfamiliar terms by constructing efficient word embeddings [53]. Graphs based on character n-grams were created [54]. Topic categorization of news, blog, and Twitter data and automatic document summary evaluation are among the many uses of these graphs.

Example: An illustration of n-grams derived from the given phrase [54].

This is a sentence.

Word unigrams: this, is, a, sentence

Word bi-grams: this is, is a, a sentence

Character bi-grams: th, hi, is, s, a, ...

Character 4-grams: this, his, is, ...

The equations that represents the Bi-garam, Tri-gram, and log probability.

$$\textbf{Bigram model} = \boldsymbol{P(w_j|w_i)} \tag{2.1}$$

$$\textbf{Trigrm model} = \boldsymbol{P(w_j|w_h w_i)} \tag{2.2}$$

$$\boldsymbol{logp(w_1,\ldots,w_T)} = \boldsymbol{logp(w_1)} + \sum_{t=2}^{T} \boldsymbol{logp(w_t|w_{t-1})} \tag{2.3}$$

Where w indicates the word, T indicates the number of words.

2.4 DEEP LEARNING (DL)

Deep learning, a subfield of machine learning, draws inspiration from the information-processing mechanisms observed in the human brain, as illustrated in Figure 2.2. Deep learning matches input to predetermined labels without the aid of any rules created by humans. Instead, it makes use of a vast amount of data. Deep learning is created by layering numerous techniques, each producing a distinct

analysis of the data given to it [55] [56]. The classification problem can be solved using conventional machine learning methods through a series of sequential steps such as preprocessing and intelligent feature selection. Additionally, feature selection significantly impacts how well machine learning algorithms work. Inaccurate class separation may be the result of biased feature selection. On the other hand, deep learning allows for the automation of feature learning. Comparatively speaking, classical machine learning algorithms [57]. Using Deep Learning DL, learning and classification can be completed simultaneously. Deep learning has recently gained significant popularity as a machine learning algorithm due to the enormous growth and advancement in big data [58] [59].

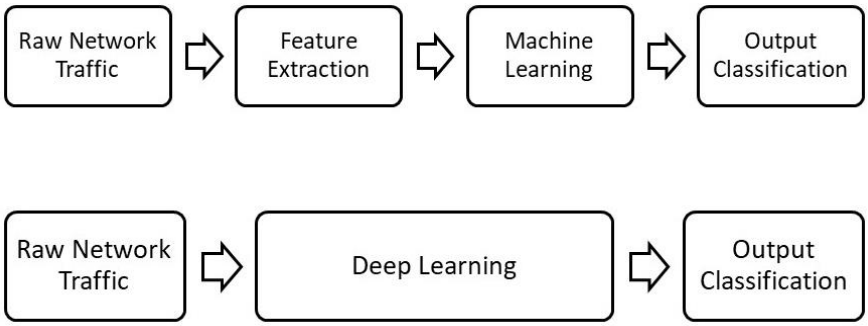


Figure 2.2: Comparison of Machine learning and deep learning [60]

2.4.1 Deep Neural Network (DNN)

DNNs, developed in recent years and inspired by neural networks, have revolutionized various fields, including NLP, video game control, navigation systems, computer vision [61], and computer vision and natural language processing. DNNs are mathematical models of multiple tiny processing units called cells or neurons. These cells operate concurrently and are organized in layers. Deep neural networks are characterized by their hierarchical structure, consisting of numerous layers stacked on top of one another. In contrast, simple neural networks comprise input and output layers [62]. The trained DNN is used to carry out the same task on new inputs. Each layer contains several nodes referred to as neurons. The edges that connect the

neurons in each layer have a weight that affects the neurons in the layer below. Each neuron in the first layer computes a weighted sum of the outputs from the neurons in the layer before it after receiving the training data. The weighted total is then subjected to the activation function. Prediction uses the output of the output layer, the final layer. The network then decides what weight is ideal for reducing the cost function. Neural network in depth Due to its complexity and subpar accuracy in comparison to other deep learning techniques, as illustrated in Figure 2.3 below, DNN is infrequently employed to categorize network traffic [63].

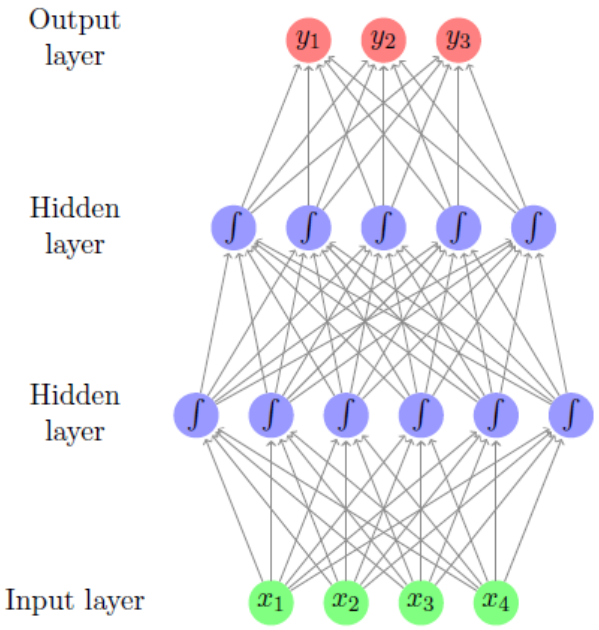


Figure 2.3: Deep neural network [63]

2.4.2 Convolutional Neural Network (CNN)

A CNN is a type of neural network that utilizes convolutions. The CNN is commonly acknowledged as the frequently utilized deep learning model. CNNs are designed with inspiration from the cells found in human and other animal brains, much like conventional neural networks. CNNs consist of an input layer, multiple hidden layers, and an output layer, in addition to the convolutional and pooling layers. CNNs can autonomously identify distinctive features from input traffic data and then create the

weights and other model parameters through training and learning operations. By removing some of the data, pooling can also reduce the impact of noise and interference while avoiding over-fitting. After obtaining the mid- and low-level data, the FC layers—the final layer—produce the high-level representation. Utilizing Softmax [64], the final layer determines the categorization likelihood.

2.5 COMPUTER VISION

Computer vision has emerged as a highly prevalent and multifaceted discipline in the contemporary era, attracting many scholars actively engaged in research within this domain. Computer vision enables machines to acquire the ability to grasp, analyze, and evaluate intricate visual content. Some subfields within computer vision include scene or object recognition, object detection, object segmentation, position and motion estimation, scene modeling, and image restoration [65]. Object detection, along with its related subfields of object localization and segmentation, is widely recognized as one of the most essential and popular jobs in the science of computer vision. CNNs, Deep Belief Networks (DBNs), Deep Boltzmann Machines (DBMs), Restricted Boltzmann Machines (RBMs), and Stacked Autoencoders represent a selection of established DL models that can address various computer vision challenges [66].

2.6 OBJECT CLASSIFICATION AND LOCALIZATION

Image categorization refers to assigning a given picture or object within an image to a predetermined set of categories. To address this challenge, supervised machine learning or DL methods are employed, wherein the model is trained using a substantial dataset that has been labeled. ANN, SVM, Decision trees, and KNN are among the frequently employed ML models for this particular purpose [67]. CNNs,

along with their architectural successors and variations, have emerged as the predominant deep models for the purpose of photo identification and related tasks.

Object localization, also known as bounding box detection, involves determining the precise location of one or several objects within an image or frame by enclosing them with a rectangular box. In contrast, picture segmentation is a process that partitions an image into many segments, where each segment might represent either a complete entity or a fragment of an object. Image segmentation is a commonly employed technique to discern and delineate the boundaries of an object or segment inside a given image. Furthermore, the integration of classification, localization, and segmentation techniques can be leveraged to constitute the object detection task. The task entails the precise classification and efficient localization of individual or many objects inside an image, typically utilizing supervised algorithms and a sufficiently large annotated training dataset. Figure 2.4 provides a comprehensive illustration of the processes involved in object detection, specifically the classification, localization, and segmentation of individual objects and many objects within an image [68].

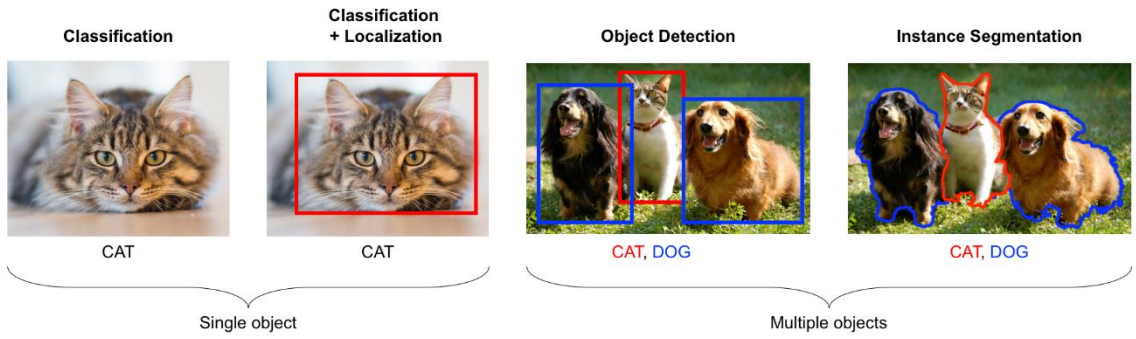


Figure 2.4: Classification, Localization, and Segmentation in Single and Multiple Objects Image [68]

2.6.1 Challenges in Object Detection

Object detection is used in autonomous driving, aerial object detection, text detection, surveillance, search and rescue operations, robotics, face and pedestrian detection, visual search engines, object of interest computation, brand detection, and

many other fields [69]. Object detection systems using ML and DL face the following challenges:

1- Recognizing objects across several scales to effectively recognize objects across several scales [70].

2- A foreground-background class imbalance can substantially impact a model's performance [70].

3- The performance of object detection techniques is generally optimized for larger objects when the model is trained with such items. These models exhibit suboptimal performance when applied to objects of relatively smaller sizes [71].

4- The need for large datasets and computing: Deep learning algorithms for object detection require large datasets, laborious annotation methods, and powerful computer resources. Due to the growing growth of data from multiple sources, annotating visual content has become more complex and time-consuming [71].

5- Smaller-sized datasets pose a challenge for deep learning models as they perform worse than standard machine learning techniques [70].

6- One issue with prediction accuracy is the presence of inaccurate localization, which can be mitigated by utilizing bounding boxes as the most proximate representations of real-world objects [70], [71].

2.6.2 Stages in object detection

Regression and classification are supervised ML issues. However, picture classification has the same categorization challenge. Localizing an object in an image follows classifying it, if necessary. Deep neural networks determine an item's rectangle bounding box. Object detection often requires feature extraction, followed by categorization and localization. A two-stage object detector performs this operation in two steps. The second stage of the two-stage object detector predicts items and bounding boxes for defined regions. However, a region proposal network (RPN) generates regions of interest in the first step. The first phase involves selecting

feasible region ideas using many methods, including negative proposal sampling. Region-based convolutional neural networks (RCNN), Fast RCNN, and Faster RCNN are popular computer vision models. Single-stage object detectors benefit from the simpler architecture designed for one-step object identification by examining all region options. By evaluating their spatial dimensions, these detectors create bounding boxes and class-specific probability for image elements. Compared to other approaches, two-stage object detectors perform better. The detectors analyze all spatial sizes in one operation to create bounding boxes and class-specific probability for image elements. Two-stage object detectors perform better than single-stage detectors because they focus on higher-probability item detection areas.

The You Only Look Once (YOLO) algorithm and its iterations have been lauded for achieving this goal in a single, efficient step. YOLO and other object detection methods use the Single Shot Detector (SSD). YOLO's accuracy and inference time are comparable to those of single-stage and two-stage detectors. This option is popular in production due to its simple architectural design, simplicity, and ease of implementation. Figure 2.5 shows the single-stage object detector design, which analyzes all spatial regions simultaneously to determine bounding boxes and class probabilities [72].

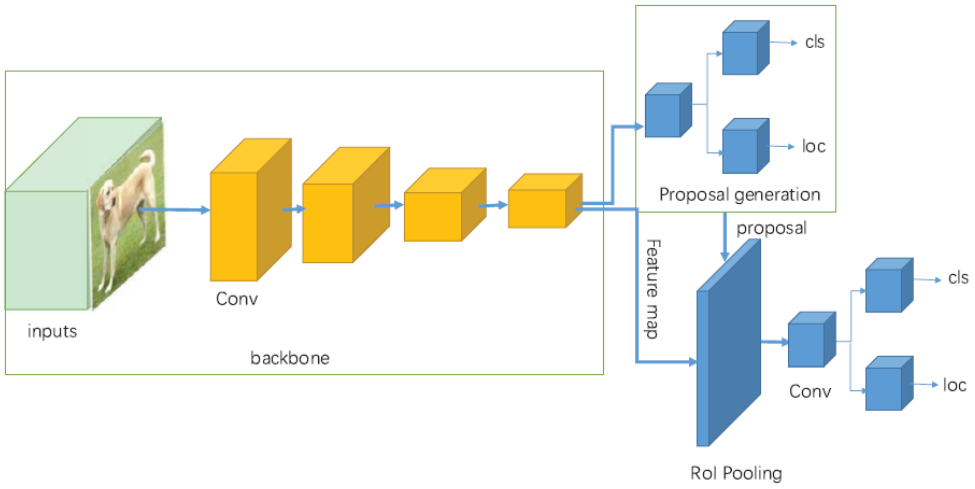


Figure 2.5: Object detectors with a single stage and a generic architecture[72]

2.6.3 Object Detection Metrics of Techniques

The most often used metric for assessing the effectiveness of object identification algorithms is Average Precision (AP), sometimes known as Mean Average Precision (mAP).

The purpose of AP and how it is calculated are covered in the following sections.

i. How does AP work?

Precision-recall measures are used to construct the AP metric, which evaluates several object categories and uses intersection over union to predict optimistically. Precision and recall typically determine a model's performance. Recall measures the model's positive example identification rate, whereas precision measures its optimistic forecast accuracy. AP's accuracy-recall curve shows precision and recall values at different confidence thresholds. This curve illustrates the precision-recall trade-off. Analysis of the area under the precision-recall curve metric measures precision and recall [73]. The equation of AP:

$$\boldsymbol{AP} = \sum_{\boldsymbol{n=1}}^{\boldsymbol{N}} (\boldsymbol{R_n} - \boldsymbol{R_{n-1}}) \boldsymbol{P_n} \quad (2.4)$$

N is the number of points in the precision-recall curve.

P_n and R_n are the precision and recall at the n-th threshold.

Localizing and classifying elements in a picture using object detection models is necessary for managing several categories. The mean average precision (mAP) measure solves the problem by calculating the average precision for each category and averaging these values across all categories. The equation of mAP If there are C classes:

$$\boldsymbol{mAP} = \frac{\boldsymbol{1}}{\boldsymbol{C}} \sum_{\boldsymbol{c=1}}^{\boldsymbol{C}} \boldsymbol{AP_c} \quad \mathbf{2.5})$$

Where AP_c is the Average Precision for class c.

A more complete model evaluation is possible by breaking down the model's performance into component pieces [74]. Estimating bounding boxes using

Intersection over Union (IoU) helps picture object identification identify objects. The IoU metric evaluates projected bounding box accuracy.

In Figure 2.6, the IoU is the ratio of the intersection area to the union of the anticipated and ground truth bounding boxes. The algorithm determines how much the predicted and observed boxes overlap [75].

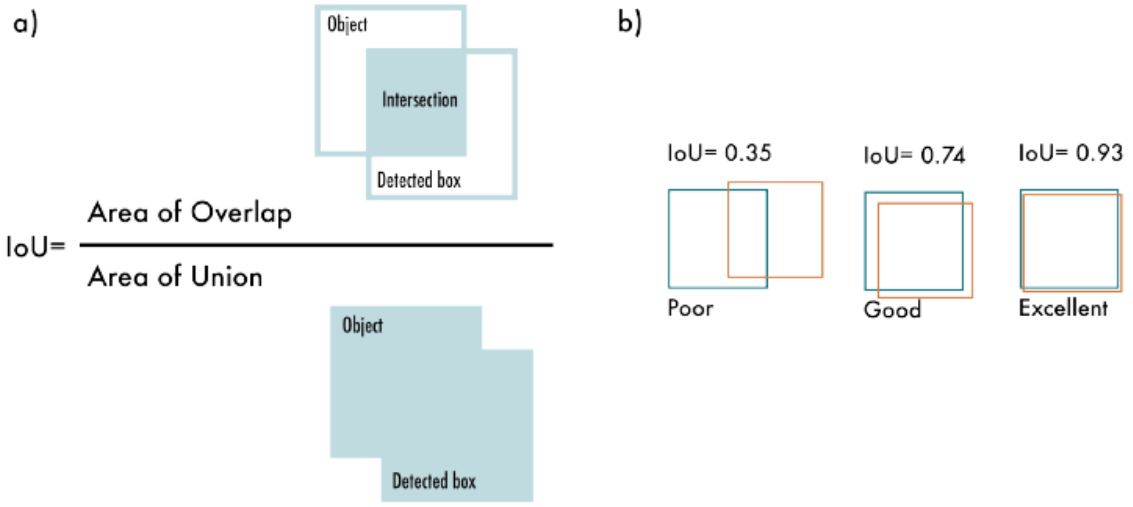


Figure 2.6: IOU measurements concept. a) The intersection area divided by the union's total area is the formula for the IoU measure. b) Examples displaying three different IoU values based on where the bounding box is placed [74].

ii. Non-Maximum Suppression (NMS)

Object detection techniques use a post-processing method called NMS to address the problem of overlapping bounding boxes and boost detection accuracy. Object detection algorithms often generate numerous bounding boxes around the same object, each with a unique confidence score. Only the most precise bounding boxes are retained after NMS filters out unnecessary and unimportant ones. Figure 2.7 displays the customary outcome of an object detection model that exhibits many bounding boxes that overlap. Additionally, the subsequent outcome after applying NMS is depicted [76].

Figure 2.7: NMS is short for "non-maximal suppression." Displays the typical result of an object identification model, which is many boxes overlapping [74]

2.6.4 Backbone, Neck, and Head

The backbone, neck, and head were the first three components utilized to characterize object detection architecture. Figure 2.8 depicts a high-level head, neck, and backbone schematic. The backbone extracts valuable features from the supplied image. A CNN trained on ImageNet or another large-scale photo classification assignment is usually employed. More advanced layers collect object fragments and semantic context, while closer to the network's core, they collect edges and textures. This hierarchical feature collecting occurs at many scales [77]. The neck architecture integrates and enhances the framework's properties to improve spatial and semantic information in several dimensions. The neck can have more convolutional layers, feature pyramid networks (FPN)[78], and other ways to display qualities better.

The head of an object detector makes predictions using the backbone and neck's data. Neural networks usually have subnetworks for classification, localization, and, more recently, instance segmentation and posture estimation. The head area processes neck data regarding unique features and provides predictions for each potential creature. Finally, NMS removes redundant predictions and keeps only the most confident detections. In the remaining YOLO models, the backbone, neck, and head will define the architecture [76].

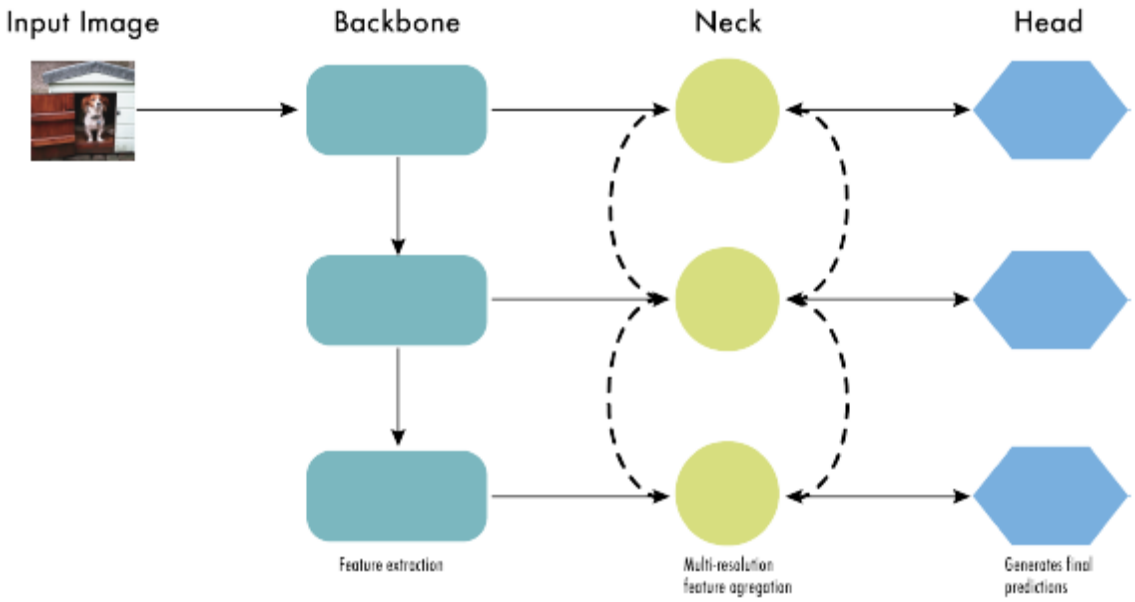


Figure 2.8: The architecture of contemporary object detectors can be likened to a framework with three components: the backbone, neck, and head [74].

2.7 YOLOv8

In January 2023, the YOLOv8 model was introduced by Ultralytics, the organization responsible for developing YOLOv5 [79]. YOLOv8 introduced five scaled versions, namely YOLOv8n (nano), YOLOv8s (small), YOLOv8m (medium), YOLOv8l (large), and YOLOv8x (extra enormous). The YOLOv8 [79] model is utilized for a wide range of vision-related tasks, including but not limited to object recognition, image segmentation, pose estimation, object tracking, and image classification.

2.7.1 The Architecture of YOLOv8

The cross-stage partial (CSP) Layer, the C2f module, has undergone substantial revisions in YOLOv8. These improvements have resulted in a structure that closely resembles that of YOLOv5. The C2f module, the cross-stage partial bottleneck with two convolutions, has been developed to improve detection accuracy by integrating high-level characteristics and contextual information. Using an anchor-free model incorporating a decoupled head, the YOLOv8 model effectively performs distinct

object recognition, classification, and regression tasks. Implementing this architecture has improved the overall correctness of the model and allows each branch to focus on its respective responsibilities. The activation algorithm employed for the abjectness score in the output layer of YOLOv8 was the sigmoid function. This function determines the likelihood of an object being included within the bounding box. The softmax function evaluates the probability of items being assigned to several classes. The YOLOv8 model employs the CIoU [80] and DFL [81] loss functions to address the bounding box loss, whereas binary cross-entropy handles the classification loss. Object detection performance has witnessed notable improvement due to the utilization of these loss functions, particularly in scenarios involving tiny items. The YOLOv8-Seg model, which is a semantic segmentation model, is also provided by YOLOv8. Instead of utilizing the traditional YOLO neck architecture, the system's backbone incorporates a CSPDarknet53[82] feature extractor, followed by a C2f module.

Two segmentation heads are then trained to predict the semantic segmentation masks for the input image in accordance with the C2f module. Model components include a prediction layer structurally similar to YOLOv8's detection heads and five independent detection modules. The YOLOv8-Seg model has achieved state-of-the-art performance on many object detection and semantic segmentation benchmarks, demonstrating efficiency and speed. can be run YOLOv8 from the command line or get it through the Python Package Index (PIP). The evaluation of YOLOv8x on the MS COCO dataset test-dev 2017, utilizing an NVIDIA A100 and TensorRT, resulted in an average precision (AP) of 53.9% when the image size was set to 640 pixels. In comparison, YOLOv5 achieved an AP of 50.7% on the same input size [83]. Figure 2.9 shows the architecture of YOLOv8.

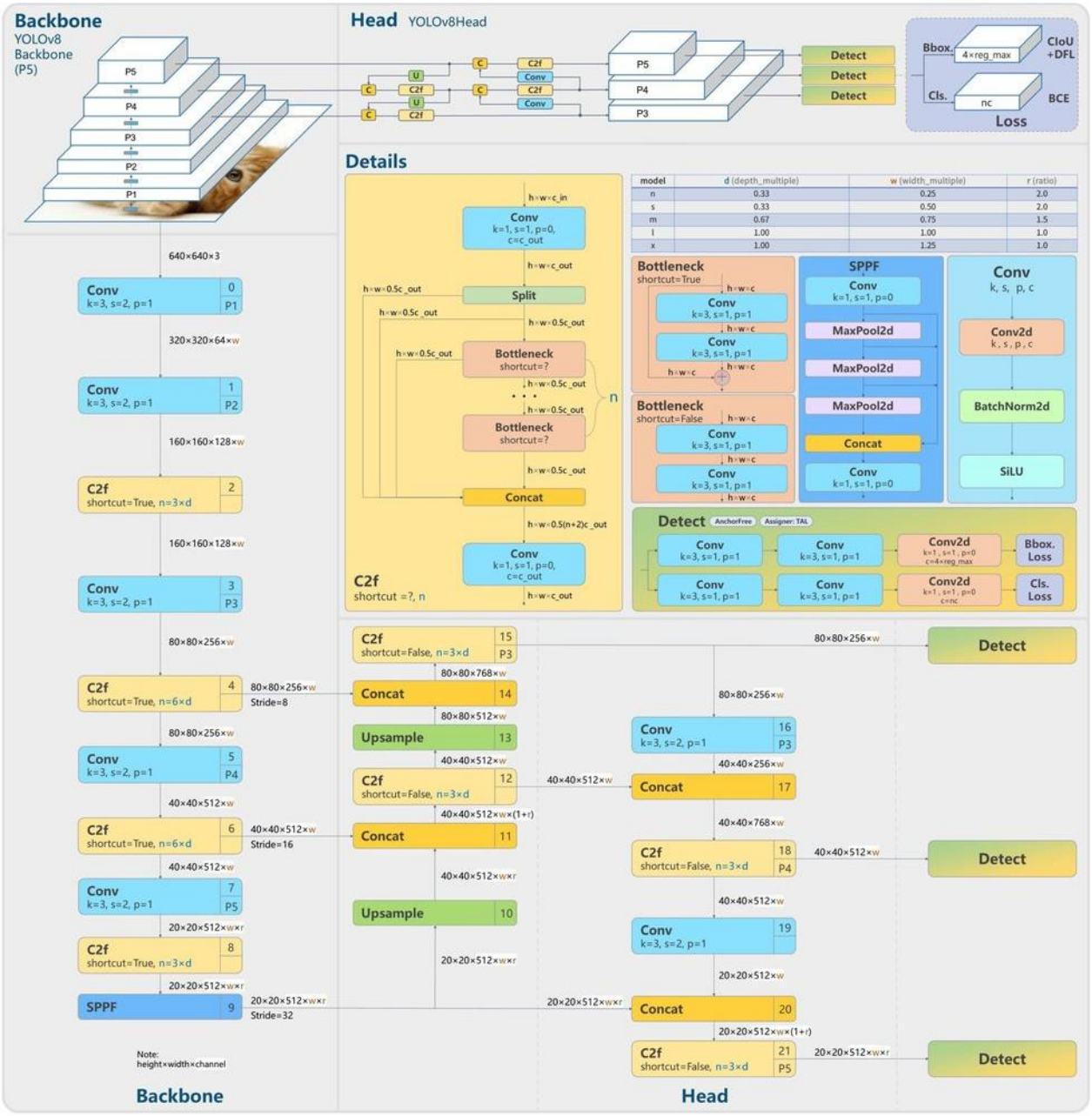


Figure 2.9: YOLOv8 Architecture

The primary reasons behind using Yolov8 are as follows [83]:

1. YOLOv8 has a high accuracy rate, according to COCO and Robotflow's 100 measurements.
2. Based on the COCO dataset, YOLOv8 shows significant accuracy. For instance, the medium YOLOv8m model has a 50.2% mAP on the COCO dataset.

3. The YOLOv8 model outperformed the YOLOv5 model in the Robotflow 100 dataset, which evaluates model efficacy across various task domains.
4. YOLOv8's developer-centric features are crucial. The Command Line Interface (CLI) simplifies model training in YOLOv8, compared to other models that require many Python files to execute tasks. Python modules also streamline and improve development compared to previous iterations.

2.7.2 YOLOv8 augments

The YOLOv8 algorithm improves the quality of images during online training. At each epoch, the model observes a slightly distinct assortment of the images provided [84]. During the training process, YOLOv8 utilizes mosaic augmentation. However, it has been observed that applying this augmentation throughout the whole training period can lead to a decrease in precision. Consequently, mosaic augmentation is deactivated for the final ten epochs. The mosaic augmentation approach combines four practice photographs, resulting in the generation of a novel composite image. This augmentation method [85] enhances the model's training and learning capabilities.

2.8 Filters

In image processing, filters are used to enhance or modify images. They can perform noise reduction, edge detection, and image sharpening tasks. Filters are typically categorized into:

1. **Linear Filters:**
    - **Gaussian Filter:** Smooths images by averaging pixel values with a Gaussian kernel.
    - **Mean Filter:** Reduces noise by averaging pixel values in a neighborhood.
    - **Sobel Filter:** Detects edges by computing the gradient of image intensity.
2. **Nonlinear Filters:**

- **Median Filter:** Reduces noise by replacing each pixel value with the median of neighboring pixel values, effective for salt-and-pepper noise.
- **Bilateral Filter:** Smooths images while preserving edges by considering spatial proximity and pixel intensity differences.
- **Adaptive Filter:** Adjusts filtering based on local image characteristics, useful for edge-preserving smoothing.

Filters are essential tools in various image-processing tasks. They enhance image quality and aid in feature extraction and analysis.

Biliteral Filter

The bilateral filter is nonlinear. Unlike linear filters, which apply a constant set of weights to all pixels in the neighborhood, the bilateral filter considers both the spatial distance and the intensity difference between the central pixel and its neighbors.

Key Characteristics of Bilateral Filter:

1. **Spatial Weighting:** The filter applies a Gaussian function based on the Euclidean distance between pixels, similar to a Gaussian blur.
2. **Intensity Weighting:** It also applies a Gaussian function based on the intensity differences between the central and neighboring pixels. This preserves edges by reducing the influence of pixels with significantly different intensities.

The bilateral filter equation combines spatial and intensity (color) components to perform edge-preserving smoothing. Here's the mathematical formulation:

$$\boldsymbol{I'_p} = \frac{\mathbf{1}}{\boldsymbol{W_p}} \sum_{\boldsymbol{q}\in \boldsymbol{N(p)}} \boldsymbol{I_q} \cdot \boldsymbol{f_s}(\|\boldsymbol{p}-\boldsymbol{q}\|) \cdot \boldsymbol{f_r}(|\boldsymbol{I_p}-\boldsymbol{I_q}|) \quad 2.6)$$

Where:

I'_p is the filtered intensity value at pixel p.

I_q is the intensity value at pixel q within the neighborhood $N(p)$ around pixel p.

$f_s(\|p-q\|)$ is the spatial Gaussian weighting function based on the Euclidean distance between pixels p and q.

$f_r(|I_p - I_q|)$ is the range Gaussian weighting function based on the intensity difference between pixels p and q.

W_p is the normalization factor to ensure the sum of the weights to one.

2.9 SPEECH SYNTHESIS

Speech synthesis is the technique of creating human speech in an artificial environment. A voice computer, also known as a speech synthesizer, is a type of computer system used to achieve this goal. It can be implemented in hardware- or software-based systems. The phenomena under study concern the automated production of oral communication that is built on written input. The analysis of text's content and subsequent translation of letters into their matching sounds make up the speech synthesis process. The technique of parsing textual material to extract factual information that computers can understand and interpret is known as text analysis. Transforming unstructured textual information into organized and structured data is the primary goal of text analysis. The procedure can be thought of as sorting many disorganized and different data [86].

2.10 LANGUAGE IDENTIFICATION

The automatic language identification process involves determining a given text's language by analyzing various clues present within the text. The methodologies employed in language identification using computers encompass a spectrum of approaches, spanning from rudimentary wordlists to cutting-edge DL algorithms. Language identification becomes increasingly complex when dealing with brief, fragmentary, and multilingual texts in languages with limited training resources. Nevertheless, the task remains feasible for lengthy books written in languages with ample resources. The issue of language identification becomes distinct when confronted with the challenge of closely related languages or dialects of a single language [87].

2.11 CUNEIFORM TEXTS IN ORACC

Open, Richly Annotated Cuneiform Corpus materials are used in the investigation. Oracc, an international collaborative initiative, offers free online digital versions of cuneiform texts from diverse initiatives. The modern electronic corpus Oracc contains large Sumerian and Akkadian texts. This thesis used XML files from Oracc, which were authorized by site administrators to represent October 2016. Most of the 13,662 writings were originally written on clay tablets. Duplicate texts with the same Oracc numbers were deleted before language identification. This removes modern duplicates from numerous Oracc projects modifying the same material. The identical ancient writings with different numerals remained. Cuneiform writing and XML files do not specify sentence endings. Thus, our data may be divided into two categories: texts with many lines and individual lines. The Oracc platform includes Sumerian, Akkadian, Hittite, Ugaritic, and Greek texts and terms. The metadata of Oracc manuscripts comprises provenance (where the text was found), genre, historical period, and other pertinent information. Transliterations of Latin alphabet cuneiform signs are the core of Oracc XML files. Even when no word information is supplied, transliterations are provided. The corpus contains many Sumerian-Akkadian bilingual manuscripts. These papers typically have linguistic similarities between the two languages, often on the same line [17].

2.12 EVALUATION PERFORMANCE

Various metrics, including recall, precision, F1-Score, and accuracy, are used to evaluate a classification model's performance. Each of these measurements has the advantage of understanding the model's performance from a certain perspective. Additionally, other equations are used to quantify the magnitude of losses.

2.12.1 Evaluation Matrix

DL evaluations test trained classifiers or learning algorithms on various datasets. Modern metrics focus on the classifier's ability to distinguish classes.

Measurement of classification performance metrics is crucial for classifier development. Although widely used methods like determining test set accuracy or error rate have substantial limitations. Despite their popularity, methods like test set accuracy and error rate evaluation have substantial drawbacks. Because multi-class problems can be solved as two-class problems, most research focuses on them. Conventionally, the dominating class is labelled positive and the minority class negative. TP and TN count correctly identified positive and negative cases. In Table 2.2 shows the confusion matrix of two classes.

Table 2.2: Confusion matrix for two classes

		Predicted Class	
		P	N
Actual Class	P	TP	FN
	N	FP	TN

There are two ways to measure a classifier's performance: numerically and graphically. Numerical evaluations give a single number to describe a classifier's performance, but graphical methods show its performance in a plot with only two or three dimensions, making it easy for people to check. Accuracy, recall, and F1-Score are all examples of numerical methods for measuring performance, while a Cost curve is an example of a graphical method [88].

$$\textbf{Accuray} = \frac{\textbf{TP} + \textbf{TN}}{\textbf{TP} + \textbf{FP}+\textbf{TN}+\textbf{FN}} \tag{2.7}$$

The concept of "accuracy" pertains to accurately classifying records' ratio to the overall number of records [88].

$$\textbf{P} = \frac{\textbf{TP}}{\textbf{TP} + \textbf{FP}} \tag{2.8}$$

Precision is a metric that quantifies the quality of positive predictions generated by a model. It is calculated by dividing the number of true positives by the total number of positive predictions [88].

$$\mathbf{R} = \frac{\mathbf{TP}}{\mathbf{TP + FN}} \tag{2.9}$$

Recall is the ratio of accurately identified anomalous records to the overall count of aberrant records [88].

$$\mathbf{F} = \frac{\mathbf{2.R.P}}{\mathbf{R + P}} \tag{2.10}$$

The F1 score is a metric that quantifies a model's effectiveness by taking the harmonic mean of its accuracy and recall [88].

2.12.2 RMSE, MSE, and MAE

The Error's measurement functions are crucial in determining the learning performance and influencing the outcomes. The optimization of error reduction is crucial to enhance the overall efficiency of the system.

The metric calculates the average squared difference between the anticipated and actual values within a given dataset. A model's fit to a dataset is better when the mean squared error (MSE) is lower [89].

$$\boldsymbol{MSE} = \sum(\boldsymbol{\hat{y}_i} - \boldsymbol{y_i})^{\mathbf{2}} / \boldsymbol{n} \tag{2.11}$$

Root Mean Square Error (RMSE) is a quantitative measure that provides insight into the discrepancy between a given dataset's anticipated and observed values. It is calculated by taking the square root of the average of the squared differences between the expected and actual values. A model fits a dataset better when the root means square error (RMSE) is lower [89]

$$\boldsymbol{RMSE} = \sqrt{\sum(\boldsymbol{\hat{y}_i} - \boldsymbol{y_i})^{\mathbf{2}} / \boldsymbol{n}} \tag{2.12}$$

The Mean Absolute Error (MAE) is a statistical measure that quantifies the average magnitude of the absolute differences between the observed and anticipated values within a given dataset. The metric calculates the mean value of the residuals within the dataset [90].

$$\boldsymbol{MAE} = \frac{\mathbf{1}}{\boldsymbol{n}} \sum_{\boldsymbol{I=1}}^{\boldsymbol{n}} |\boldsymbol{y_i} - \widehat{\boldsymbol{y_\iota}}| \quad (2.13)$$

$\widehat{\boldsymbol{y_\iota}}$ = predicted value, $\boldsymbol{y_i}$ = Actual value, and N = Number of Samples.

2.12.3 The Losses Calculation For Classification

The equations of losses that results in the output of classification by using YOLOv8:

$$\boldsymbol{L_{train}} = -\frac{\mathbf{1}}{\boldsymbol{n}} \sum_{\boldsymbol{i=1}}^{\boldsymbol{n}} \sum_{\boldsymbol{c=1}}^{\boldsymbol{C}} \boldsymbol{y_{ic}} \log(\boldsymbol{p_{ic}}) \quad (2.14)$$

y_{ic} is the ground truth binary indicator (1 if class label ccc is the correct classification for input iii, otherwise 0).

p_{ic} is the predicted probability of input i being class c.

The validation loss equation:

$$\boldsymbol{L_{val}} = -\frac{\mathbf{1}}{\boldsymbol{M}} \sum_{\boldsymbol{i=1}}^{\boldsymbol{M}} \sum_{\boldsymbol{c=1}}^{\boldsymbol{C}} \boldsymbol{y_{ic}} \log(\boldsymbol{p_{ic}}) \quad (2.15)$$

M is the number of validation samples.

p_{ic} and y_{ic} are defined similarly as above but for the validation set.

2.12.4 The Losses Calculation For Detection

YOLOv8 (You Only Look Once version 8) is an advanced object detection model that utilizes a combination of multiple loss functions to train the model. The losses for both training and validation typically include components for bounding box regression, objectness score, and class prediction. Here are the typical components of the loss function used in YOLO models, which can be extrapolated to YOLOv8 [92]:

- Training Loss Equation

The overall training loss train can be expressed as a weighted sum of three main components:

$$\boldsymbol{L_{train}} = \boldsymbol{L_{box}} + \boldsymbol{L_{box}} + \boldsymbol{L_{cls}} \quad (2.16)$$

Where:

L_{train} is the bounding box regression loss.

L_{obj} is the objectness loss.

L_{cls} is the classification loss.

Components:

1. Bounding Box Regression Loss (L_{box})

This term measures the accuracy of the predicted bounding box coordinates compared to the ground truth. It often uses a variant of the IoU (Intersection over Union) loss, such as GIoU (Generalized IoU), DIoU (Distance IoU), or CIoU (Complete IoU) [93].

$$\boldsymbol{L_{box} = \sum_{i=1}^{N}(1 - IOU(B_i, B_i^{gt}))} \quad (2.17)$$

Where B_i is the predicted bounding box and B_i^{gt} is the ground truth bounding box.

2. Objectness Loss (L_{obj}):

This term measures the model's confidence score in predicting whether a bounding box contains an object. It typically uses binary cross-entropy.

$$\boldsymbol{L_{obj} = \sum_{i=1}^{N}((-y_i \log(p_i) - (1 - y_i) \log(1 - p_i))} \quad (2.18)$$

Where y_i is the ground truth objectness score and p_i is the predicted objectness score.

3.Classification Loss

This term measures the accuracy of the predicted class labels for each bounding box. Depending on whether it's a multi-label or single-label classification, it usually employs binary cross-entropy or categorical cross-entropy.

$$\boldsymbol{L_{cls} = \sum_{i=1}^{N}\sum_{c=1}^{C}((-y_{ic} \log(p_{ic}) - (1 - y_{ic}) \log(1 - p_i))} \quad (2.19)$$

Where y_{ic} is the ground truth label for class c and p_{ic} is the predicted probability for class c.

- Validation Loss Equation

The validation loss L_{val} is computed similarly to the training loss but is evaluated on the validation dataset. It uses the same components:

$$\boldsymbol{L_{val} = L_{box}^{val} + L_{obj}^{val} + L_{cls}^{val}} \quad \mathbf{2.20})$$

Where:

$L_{box}^{val}, L_{obj}^{val}, L_{cls}^{val}$ are the bounding box, objectness, and classification losses calculated on the validation data.

Chapter 3

DESIGN AND IMPLEMENTATION OF TECHNIQUES

3.1 INTRODUCTION

This chapter explains the datasets and architecture proposal used in two parts of the thesis: the images and CSV datasets. It also presents the proposed architecture model of ML and NLP algorithms used to identify the cuneiform language, and the second part consists of three subparts: the first part uses the pretraining model for classifying cuneiform scripts, the second part is the detection of Neo-Assyrian signs, and the third part is Hebrew letters.

3.2 THE CUNEIFORM LANGUAGE IDENTIFICATION

The initial phase of the research involves employing multiple categorization techniques to analyze the cuneiform texts, specifically those of Sumerian, Neo-Assyrian, Old Babylonian, Middle Babylonian peripheral, Standard Babylonian, Neo-Babylonian, and Late Babylonian origin. Initially, a dataset comprising cuneiform writings in 7 different languages was acquired through downloading. ML methods were employed to categorize the texts, a DL model was created, and the N-Gram algorithm was utilized from the NLP algorithms. The task was split into two sections. The initial section comprises machine learning methods and the DL model, while the latter section encompasses the N-Gram model developed from the ground up. This model aims to evaluate the accuracy of different algorithms and establish a benchmark for future research and development in this field.

3.2.1 Architecture of the ML and DL proposed models

The construction of an intelligent system involves emulating a decision-making computer system capable of recognizing cuneiform signs through a multi-

class classification technique. The language identification process is carried out by an intelligent system, which examines many inputs, including signals, to determine the specific language used. Upon the completion of various essential procedures, the intelligent system generates results. The user submits a dataset in the format of a comma-separated values (.csv) file, which undergoes pre-processing techniques to address missing values and encoded data, thereby refining the dataset. The data undergoes preprocessing before being provided to multiclass classification algorithms for training and Testing. This Thesis compares the accuracy and sensitivity of the output generated by various ML methods, including ANN, K-NN, NB, LR, DT, RF, SVM, Adaboost, Gradient Boosting, XGBoost, and n-Gram NLP techniques. Figure 3.1 depicts the diagram of the general architectural framework of the intelligent system. The intelligent system is composed of various stages that are involved in the process of identifying the cuneiform language.

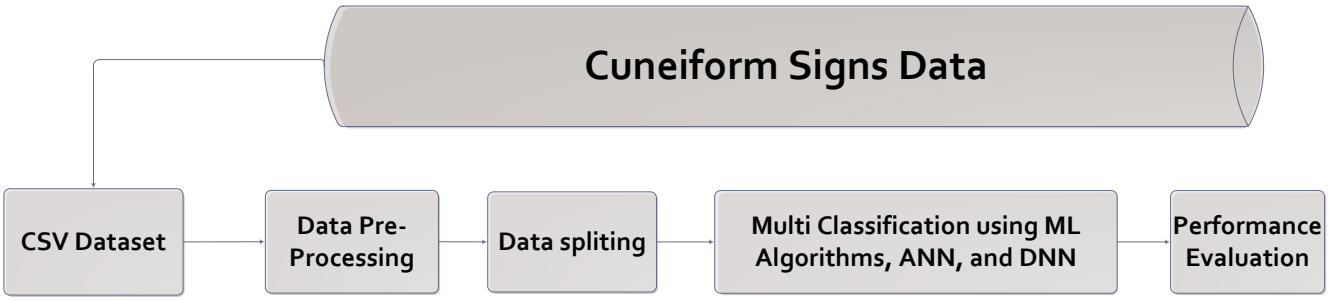


Figure 3.1: Diagram of General Architectural

Figure 3.2 displays the process for identifying intelligent cuneiform languages using the Adam optimization algorithm. The suggested model gathers data using the dataset housed in the Kaggle machine repository. The two sections of the provided model are the training and test phases, which can be utilized for validation and testing. Model design goes through several stages, from reading the data to evaluating the classification. Reading the obtained data from a CSV file is the first step.

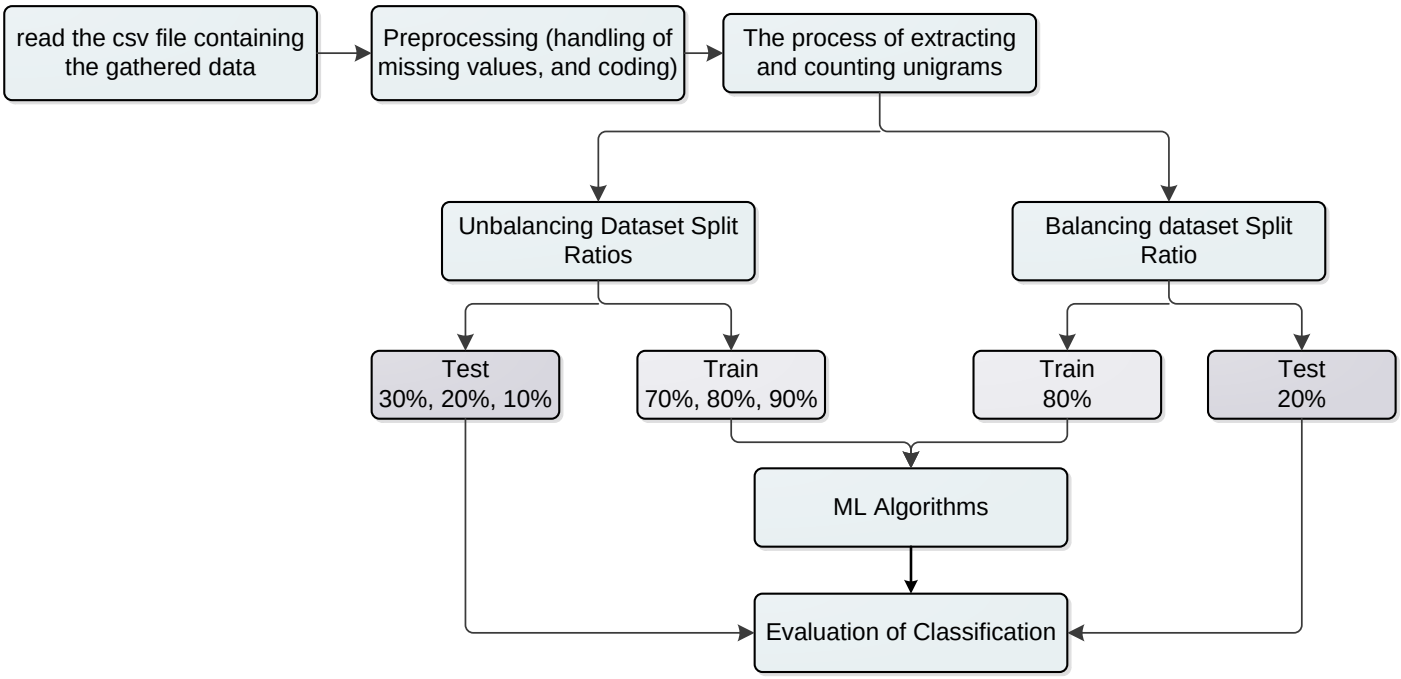


Figure 3.2: Diagram of the research approach for the suggested system

A. CSV Dataset Creation

The dataset describes the shared tasks related to the Cuneiform Language Identification (CLI) data. The material under consideration comprises seven distinct groupings, including Sumerian (SUX), Old Babylonian (OLB), Middle Babylonian peripheral (MPB), Standard Babylonian (STB), Neo-Babylonian (NEB), Late Babylonian (LTB), and Neo-Assyrian (NEA). The seven groups that make up this material was downloaded from Kaggle website https://www.kaggle.com/datasets/wilstrup/cuneiform-language-identification.

Figure 3.3 illustrates the classes within the NEA and SUX datasets encompass most of the training data. where 13.65% of the data pertains to the remaining three groups. 86.35% is distributed among the SUX, NEA, STB, and LTB categories [18].

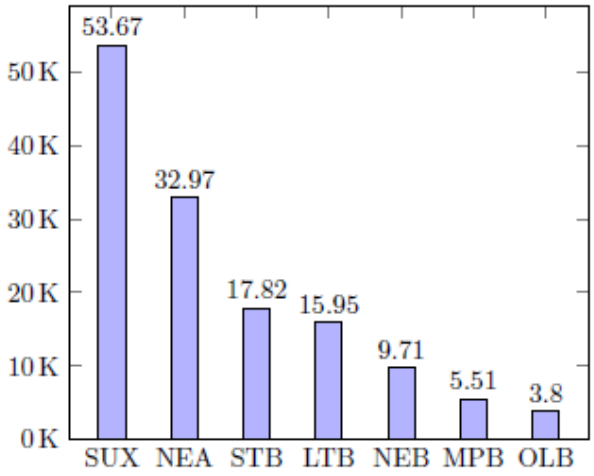


Figure 3.3 Number of samples for each label in the training set [87]

The most troublesome feature of the data is how brief most text parts are. Only seven characters, on average, are present in each section of the training set. Over 50% of the parts have five characters or fewer, while more than 10% have just one. Twelve segments in the tail of the distribution have more than 64 characters, whereas just one has more than 128. The scarcity of numerous cuneiform symbols in the provided data poses an additional challenge. There are 550 distinct characters in the training data; 128 only appear ten times or fewer, and 39 only appear once. Another crucial characteristic is the training data abundance of duplicates within and across classes. Only 86,454 of the 139,421 segments in the training set are unique. The most prevalent part in six of the seven classes can be found 3223 times. The second most frequent occurrence, amounting to 1460 instances, consistently transpires inside the confines of a particular class referred to as LTB.

B. Preprocessing Methods

The preprocessing section outlines the steps undertaken to prepare the datasets for training algorithms, including class balancing, data splitting, and punctuation removal.

- Balancing dataset: Various techniques exist for transforming an imbalanced dataset into a balanced one. In this thesis, the approach employed was resampling with varying ratios without modifying the dataset. The ratios utilized for training and testing were 30-70, 20-80, and 10-90. Subsequently, the dataset was modified by applying Over-sampling (specifically up-sampling) by replicating the samples from the minority classes for each class. By incorporating additional samples into the original dataset, the number of examples in each class has now reached 53,673, resulting in a more balanced dataset.

  Another balancing sample was done, which combines up and down-sampling, by making the number of texts for each language equal to the number of texts for the LTB language because they fall in the middle between the high-text values of the SUX, NEA, and STB languages, and the low-text values of the NEB, MPB, and OLB. Thus, the number of texts for each cuneiform language will equal 15.95. Thus, the number of repetitions of texts that occurred when using the up-sampling method is reduced, which may sometimes lead to the problem of overfitting. Table 3.1 gives more details regarding the data. Upon achieving dataset balancing, each class now comprises 53,673 samples.

Table 3.1 lists the samples in each label's training set and their proportion of 139,421, ranked from highest to lowest

Label	# of samples	% of all	# of samples after up-sampling	# of samples after up-down sampling
SUX	53,673	38.49%	53,673	15,95
NEA	32,966	23.64%	53,673	15,95
STB	17,817	12.78%	53,673	15,95
LTB	15,947	11.44%	53,673	15,95
NEB	9,707	6.96%	53,673	15,95
MPB	5,508	3.95%	53,673	15,95

OLB	3,803	2.72%	53,673	15,95

- Special Character Removal: This involves eliminating special characters such as parentheses, equals signs, and greater-than signs from the text. These characters are often extraneous and may interfere with the analysis process.
- Punctuation Removal: Punctuation marks, such as commas, periods, and exclamation marks, are removed from the text. While punctuation aids readability in natural language, it is often unnecessary for the task of spam detection.
- Encoding: Numbering each language target to a number from 0 to 6 {'LTB': 0, 'OLB': 1, 'MPB': 2, 'NEA': 3, 'NEB': 4, 'STB': 5, 'SUX': 6}. Table 3.5 shows the samples of the dataset after coding.

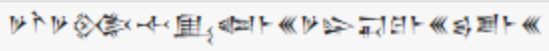
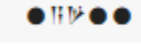
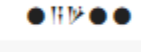
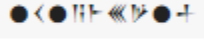
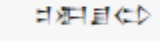
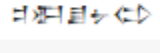
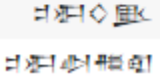
	cuneiform	lang	lang_coded
0		LTB	0
1		LTB	0
2		LTB	0
3		LTB	0
4		LTB	0
...	...	...	...
139416		SUX	6
139417		SUX	6
139418		SUX	6
139419		SUX	6
139420		SUX	6

Figure 3.4: Output after Encoding

By undertaking these preprocessing steps, the cuneiform scripts are cleansed of irrelevant information and structured in a format conducive to effective cuneiform identification. Subsequent stages of our approach, such as feature extraction and training, can utilize this refined dataset to develop robust machine-learning models for cuneiform identification.

C. Feature Extraction

For Feature extraction from text, including Unigram extraction for counting and data splitting. N-gram extraction is a method used in natural language processing to identify and extract contiguous sequences of n (words) items from a given text. the "N" represents the number of items in each group or sequence. N-grams are continuous sequences of "N" items from a given sample of text or speech. These items can be characters, words, or any other units of text. For example, if n = 1, then the extracted sequences are called “unigram”; if n = 2, then the extracted sequences are called “bigrams”; and if n = 3, then the extracted sequences are called “trigrams.” This thesis applied a unigram because the largest percentage of cuneiform signs, up to 50% or more, are one sign, approximately 40% are signs composed of two or three signs, and a very small percentage consists of 4, 5, or 6 signs. For example, if the “cuneiform” column contains the string, applying the unigram would result in the “cuneiform split” column containing the list. Once the n-grams have been extracted, they can be counted to provide helpful information about the text. The count of each n-gram can be used to calculate the frequency of occurrence of the n-gram in the text. Table 3.6 shows the unigram for each cuneiform sign.

	cuneiform	lang	lang_coded	cuneiform_split					
94424		SUX	6		0	0	0	0	0
97503		SUX	6		0	0	0	0	0
12245		LTB	0		1	0	0	0	0
13474		LTB	0		0	0	0	0	0
60545		NEB	4		0	0	0	1	1
...	...	...	...	...	...	...	...	...	...	...
134272		SUX	6		0	0	0	0	0
1581		LTB	0		0	0	0	0	1
95345		SUX	6		0	0	0	0	0

Figure 3.5 The n-gram in the text

D. Training and Classification Models

The classification phase involved the utilization of many algorithms for different ML types, such as supervised learning (LR, SVM, DT, and NB), instance-based learning (KNN), and ensemble learning (RF, Adaboost, Gboost, and XGBoost). Neural networks with varying numbers of neurons for hidden layers comprise three layers. Is used the neural networks that were created consist of three designs: [(550 - 62 - 7), (550 - 164 - 7), (550 - 264 - 7)]. The number of signs in the dataset without repetition is 550, so the input layer is 550 neurons. The output layer has the same number of target outputs (cuneiform languages).

Initially, a total of 62 concealed layers were implemented, as determined by the subsequent 3-1 and 3-2 equations [91]:

$$\boldsymbol{N^h = \sqrt{N^i\, N^o}} \tag{3.1}$$

$$\boldsymbol{\eta = {32}/{\sqrt{N^i\, N^o}}} \tag{3.2}$$

N^i: number of input neurons, N^o: number of output neurons, N^h: number of hidden neurons, and η learning rate.

The number of hidden layers was augmented to 164 and 264 to enhance performance. Once this amount was reached, the results were substantiated, prompting us to halt at this numerical threshold. The input number, 550, represents the number of tags, whereas the output, 7, shows the number of languages included in the dataset.

A DNN model was created with 550 input neurons and 3 hidden layers. Each layer contains a varying number of neurons, specifically 100 in the first layer, 200 in the second layer, and 300 in the third layer. The ReLU activation function was used in the hidden layers because it has many advantages: Computational Efficiency, Mitigation of the Vanishing Gradient Problem, and Faster Convergence. In the output, the softmax activation function was commonly used in the output layer of neural networks for multi-class classification tasks. It converts a vector of raw scores (logits) into probabilities, where each probability represents the likelihood of a particular class.

After conducting experiments using several deep DNN architectures, the most optimal outcomes were obtained using this particular model. The learning rate used in computation was (0.091168).

3.2.2 Architecture of the NLP Proposed Models

The N-gram model, shown in Table 3.7, is made from scratch because most NLP tools are made to work with Latin characters and are hard to change to work with cuneiform text. It's based on N-gram models that use Markov chains and the Markov assumption, which is how I came up with my own version. This idea says that the chance of an n-gram being in place t depends only on the k n-grams that came before it (usually k is 1). So, to find the chance of a sequence, you can multiply the conditional probabilities of its n-grams together. The code determines the chances of bigrams (n = 2)and trigram (n = 3) for a training set in each language. It figures out how likely it is that a series seen fits into each language. As a prediction, the language with the best chance of happening is given.

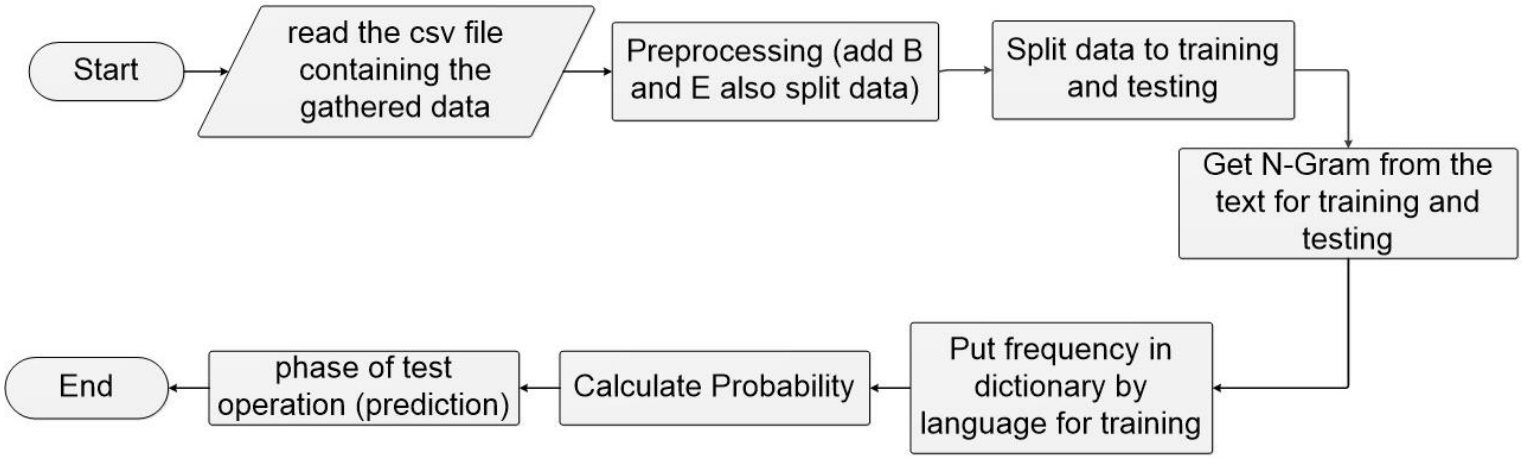


Figure 3.6 Diagram of the research approach for the N-Gram suggested system.

1. Analyze the contents of the archive file, which is composed of two columns containing cuneiform scripts and their corresponding target language.

2. Preprocess text written in cuneiform script. Executes two sequential actions: (1) Inserts tokens at the start and end of each sentence, denoted as "B" and "E" appropriately. (2) Introduces intercharacter spacing to enhance text legibility.
3. Divide the data into separate sets for training and testing purposes. Splitting an unbalanced dataset into multiple ratios is common for training and testing purposes. These ratios can be 10-90, 20-80, or 30-70. However, when balancing the dataset by techniques such as up-sampling or down-sampling, a splitting ratio of 20-80 is typically employed.
4. Count the n-grams and store them in a dictionary.
5. Organize frequencies into language-specific dictionaries for training purposes.
6. compute probabilities inside a collection of data frames stored in a dictionary.
7. Lastly, forecast the test data.

3.3 The Ancient Scripts Classification and Detection

This section will provide the overall design needed for classification and detection tasks, utilizing a pre-trained YOLOV8 model and the dataset included in both models. Additionally, it will explain the process of assembling these components.

3.3.1 The Architecture of the Proposed Models

The YOLOv8x pretraining model creates the intelligent model presented in Figure 3.7. This model is used for classification and object identification. The two models differ in terms of the dataset, the preprocessing, and the augmentation components.

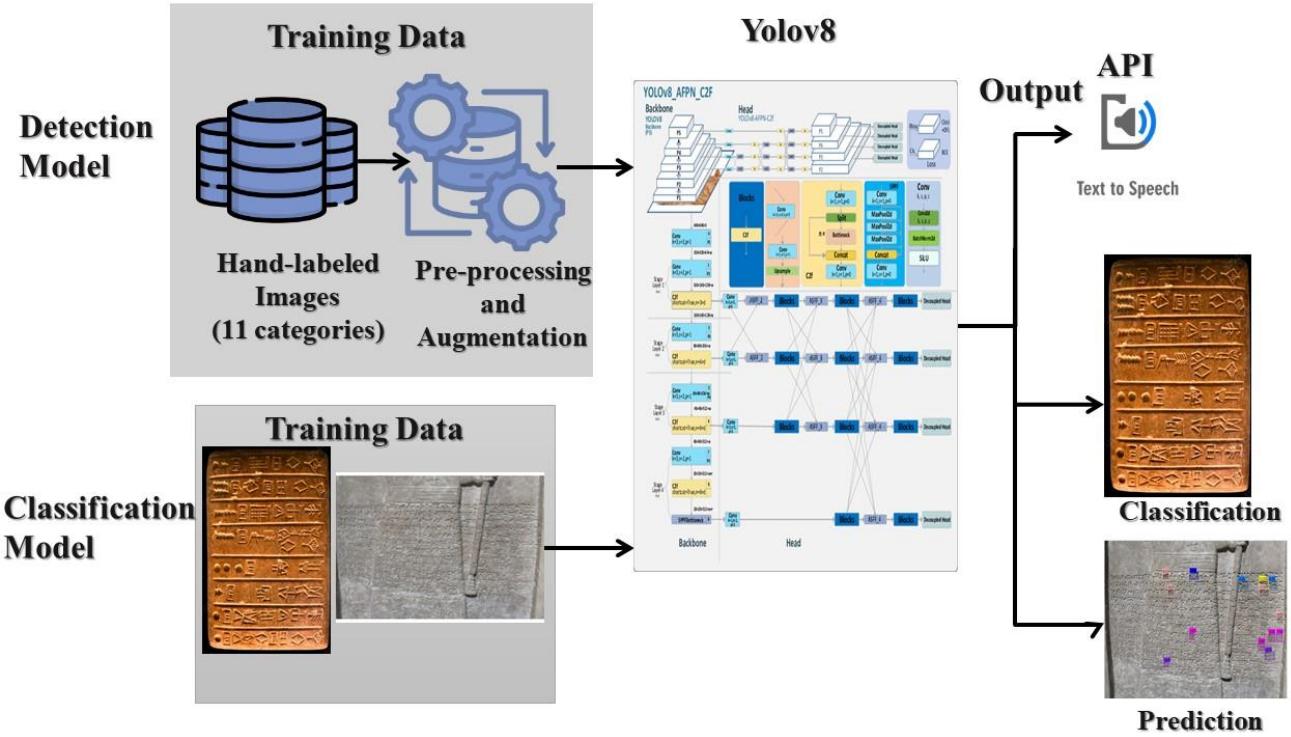


Figure 3.7: The Block Diagram Proposed Model

This collection of photographs of the Sumerian (Uruk, Ur, and Lagash), Assyrian (old, middle, and neo), and Babylonian (old, middle, and neo) tablets was accomplished by taking a collection of photographs of the tablets that are on display at the Iraq Museum and on a portion of the website of the Cuneiform digital library initiative (CDLI). Multiple photographs of the tablets were taken from various angles and perspectives. To capture a wide range of images for each tablet, the photographer consciously decided to shoot the photographs from several perspectives and adjust the lighting and the distance from which the photographs were shot. It can be seen in Figure 3.8 that photographs were obtained of paintings, statues, and floors that were written in the Assyrian language and belonged to the monarchs of the Assyrian empire. One of the most typical challenges encountered throughout the data preparation process is locating as many tablets as possible that are written on and

have not been destroyed over time. Environmental conditions have had an impact on certain panels, which has destroyed a portion of those panels.

Figure 3.8: Pictures of paintings, statues, and floors written in the Assyrian language

For the classification model, a collection of photographs of cuneiform tablets from Sumerian, Assyrian, and Babylonian periods was gathered, and the images were augmented.

For the detection model, the first two languages, the new Assyrian language written on stone tablets, and 11 Assyrian signs were chosen, were detected. The second is the Hebrew language, and all its 22 letters have been identified.

The tagged photographs were submitted for analysis, and before further processing, the images were subjected to many pre-processing steps. These steps involve resizing the images to dimensions of 640x640 pixels, applying auto-orientation, auto-adjusting contrast, and tiling. Additionally, it is necessary to offer picture augmentation techniques such as rotation, brightness adjustment, mosaic application,

and bounding box rotation. The Yolov8x object detection algorithm receives pre-processed data for training and validation purposes.

This thesis compares the accuracy, precision, and recall metrics of the output generated by multiple variants of the yolov8 algorithm, explicitly focusing on variations in split, pre-processing, and enhanced algorithms. The Yolo algorithm produces two main outputs: the bounding box coordinates and the predicted class label. An additional output has been incorporated to articulate the tablet's Classification in the cuneiform language it pertains to and the pronunciation of the signs found and recognized in the image. The projected class for each object detected in an image will be represented as a string, denoted by "NU." Utilizing the Google Text-to-Speech (gTTS) package enables the submission of the written description to the Google text-to-Speech application programming interface (API). The flowcharts for the three complete suggested models design is depicted in Figure 3.11, Figure 3.10, Figure 3.11.

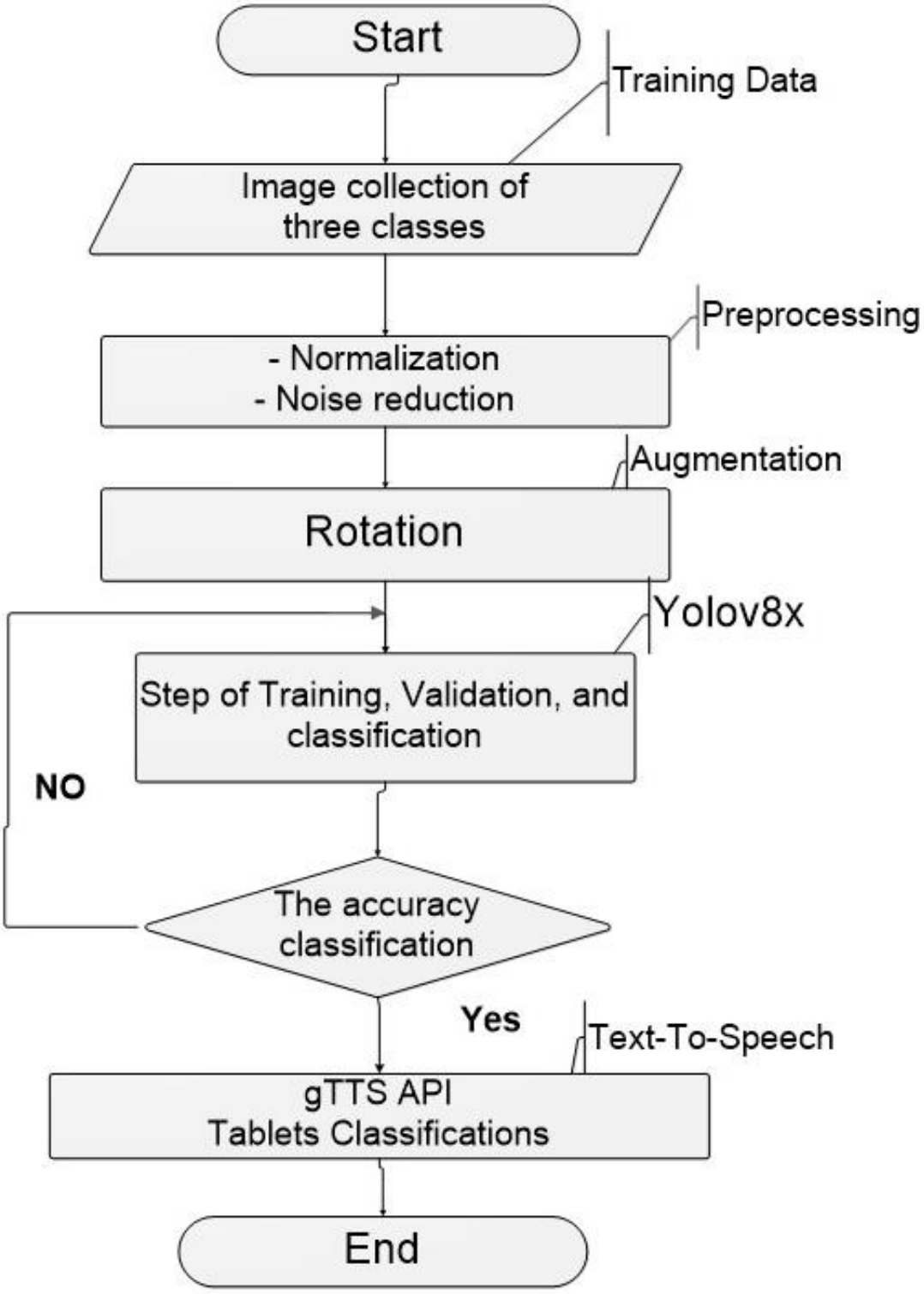


Figure 3.9: Flowchart of Classification Models

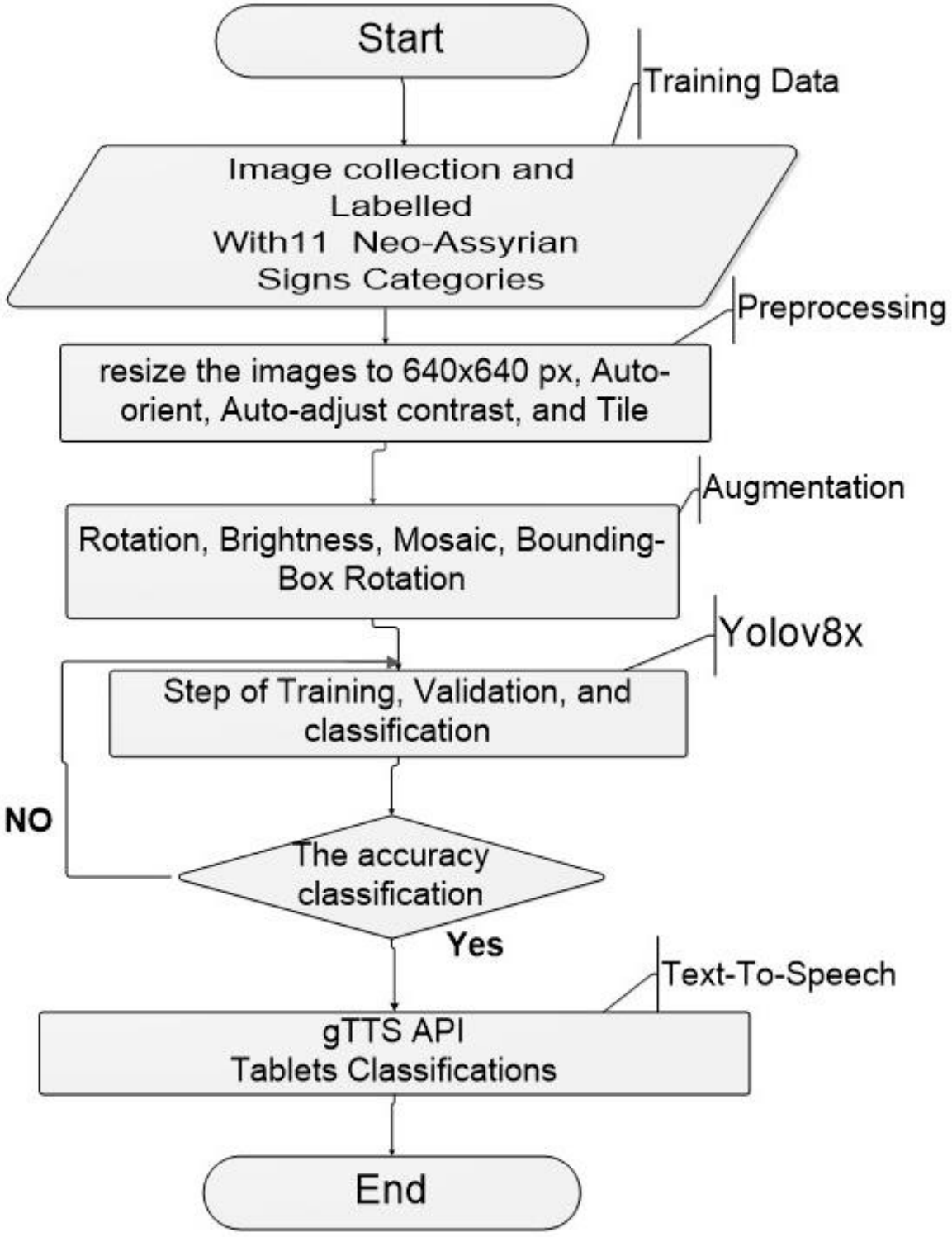


Figure 3.10: Flowchart of Neo-Assyrian Signs Detection

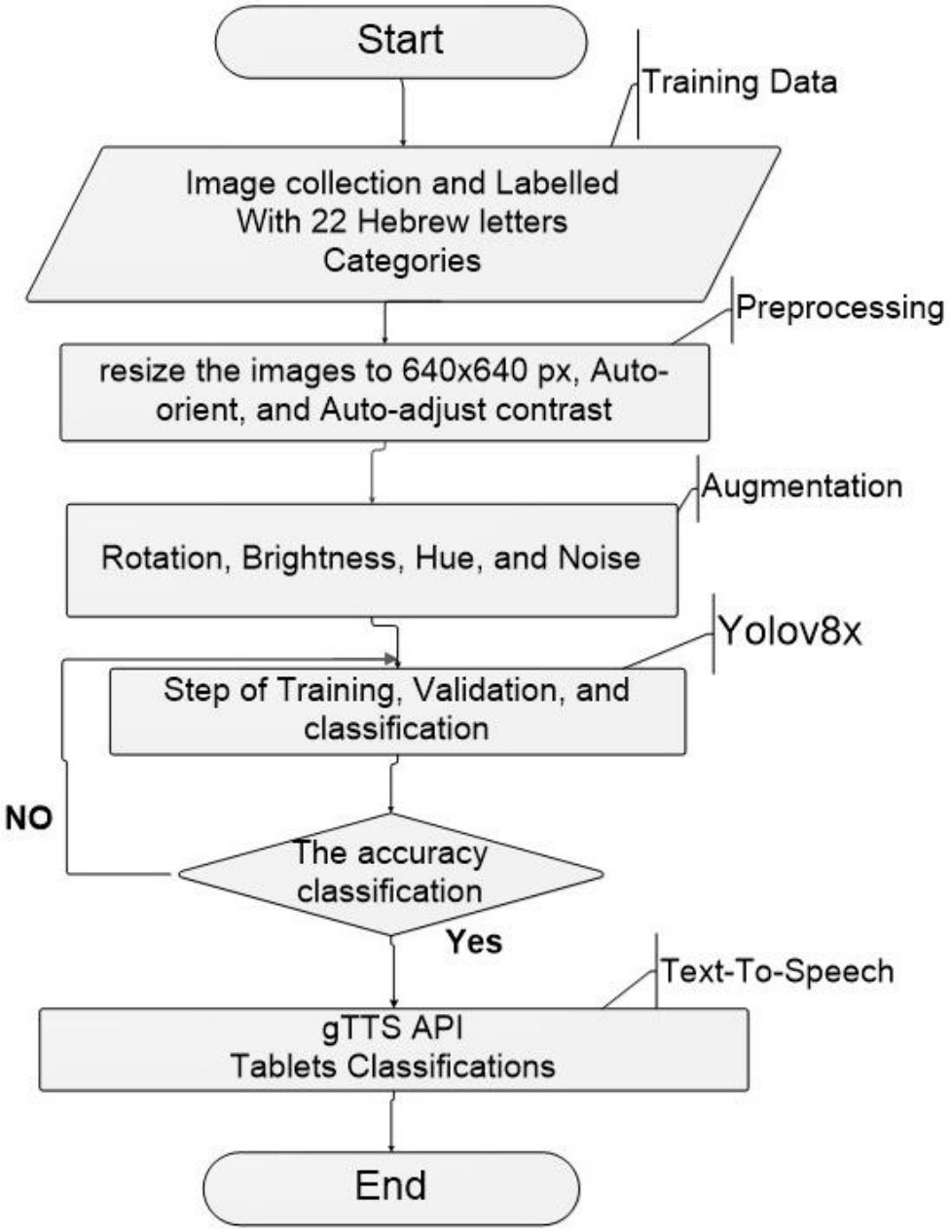


Figure 3.11: Flowchart of Hebrew Letters Detection Models

The architecture for classification and detection is the same, but the difference is that with classification, the images are entered directly without labeling using Roboflow. Only the three languages (three classes) are entered, each in a special folder, divided into training, validation, and testing.

For detection, labeling is done, followed by preprocessing and augmentation using Roboflow. Then, the data (images) are taken and entered into Yolov8. Detection is done, and the labels are pronounced.

Table 3.2 shows the Hyperparameters for preprocessing and augmentation process for detection model.

Table 3.2: Hyperparameters for preprocessing and augmentation process for detection models.

Preprocessing	Hyperparameters	Augmentation	Hyperparameters
1- Auto-orient **2- Resize** **3- Auto-adjust contrast.** **4- Tile.**	1- ±90°, ±180° 2- 640x640 px 3- Adaptive Equalization 4- 2x2 split image	**1- Rotation.** **2- Brightness.** **3- Mosaic.** **4- Bounding Box Rotation**. **5- Hue** **6- Noise**	1- ±15% 2- ±25% 3- Merage 4 different images 4- ±15° 5- ±15° 6- Up to 1.8% of pixels (salt & pepper)

3.3.1.1 The Cuneiform Tablets Classification

The classification of tablets involves a series of processes, namely data gathering, pre-processing, augmentation, and training and evaluation.

I. Dataset Collection

A collection of photos and several annotations are required to work with tablet classification models. For optimal precision, gathering thousands of images for every category is recommended. An effort was made to gather a comprehensive collection of photographs of the cuneiform tablets. Several images of cuneiform tablets from the Sumerian, Assyrian, and Babylonian (different eras) periods were gathered. Specifically, 1327 images of Assyrian,1435 images of Sumerian, and 1321 images of Babylonian writing were acquired to train a model to classify between the tablets. A collection of 880 images depicting 14 stone tablets inscribed with the New Assyrian language was gathered from an Iraqi museum. The rest of the pictures were collected

from the CDLI website. Figure 3.12 shows the Neo-Assyrian tablets collected from the museum and the Sumerian tablets.

Figure 3.12: The Tablets of the Neo-Assyrian, Sumerian, and Babylonian

II. Pre-processing and Augmentation

For preprocessing normalization and noise reduction, and augmentation the rotation and flipping were used. Rotation enhancement, which entails rotating the image at an angle between -90 and +90 degrees. To achieve a higher level of accuracy in the outcomes, this method generates photographs from a wide range of perspectives and orientations, not just the ones from which the images were taken. First, the photographs were normalized using histogram normalization. Therefore, this will guarantee that the illumination is dispersed uniformly across the entirety of the image and that the brightness is maintained throughout. Moreover, a Bilateral Filter was applied to lessen the amount of noise in every pixel. Through the use of the bilateral filter, images are rendered smoother while maintaining their respective edges. To accomplish this, it simultaneously considers the proximity of pixels and the color similarity between them. Pixels that are within close physical proximity to one

another and have similar colors are merged to create a more consistent image. Unprocessed pixels are those that are either very far apart or come in a wide range of colors. The result is a picture that is not only smoothed out but also has the edges sharpened. The bilateral filter can assist in reducing noise before edge identification. After applying these filters, the images have been prepared for computer vision model and in-depth analysis, which will result in excellent-quality photographs.

Table 3.3 shows the splitting, preprocessing, and augmentation processing that was made to the classification.

Table 3.3: The Pre-processing and Augmentation of the dataset

	Total Number of Images	**Train – Val - Test**	**Preprocessing**	**Augmentation**
Classification	Sumerian = 1435 Assyrian = 1327 Babylonian= 1321	70% - 15% - 15%	- Normalization - Noise reduction	- Rotation

III. Training and evaluation of the classification Model

The model was trained on COLAB and 200 epochs. The 200 epochs require two to five hours for the classification model. The parameters used in training are a 0.01 learning rate, 16 batch size, and 0.937 momentum.

To assess the performance of the classification model, accuracy and error are measured using the top-5 and top-1 metrics. The term "Top-1 error rate" refers to the accuracy of an algorithm on a classification job, specifically measuring the rate at which the system correctly predicts the top-ranked class. A response is deemed in the Top-5 if it is among the classifier's five most accurate predictions. The top-1 error rate measures the frequency at which the network accurately predicts the label with the highest likelihood. The "top-5 number" is the frequency of the correct label appearing among the network's five most accurate predicted classes.

3.3.1.2 The Neo-Assyrian Signs Detection

The detection of Neo-Assyrian signs involves a series of processes: data gathering, labeling, pre-processing, augmentation, and finally, training and evaluation.

I. Neo-Assyrian Dataset Creation

A substantial number of manually annotated input photographs is necessary to train models of superior quality. A compilation of visual representations depicting the Assyrian tablets housed within the Iraqi Museum has been assembled. Fourteen Assyrian tablets, the images of which were captured and inscribed onto stone tablets. Figure 3.13 depicts a series of visual representations of Assyrian tablets.

Figure 3.13: Some pictures of Neo-Assyrian Tablets

II. Dataset Labelling

The most prevalent indications in the sentences were selected after looking over the gathered images. The form (Neo-Assyrian sign) and pronunciation of these 11 signs are displayed in Table 3.4 since we discovered that they were more common than the other signs. The word (sign name) is written as a label when describing signs. The word signs are printed in well-known English letters or any other language to make reading the signs easier. For instance, the sign ( ) is pronounced similarly to A, and the signs ( ) are pronounced similarly to MAN.

Table 3.4 shows the scores chosen, which explains the form of the sign and its pronunciation.

Table 3.4: The Neo-Assyrian signs with Signs Name

S. No	Neo-Assyrian Sign	Sign Name
1		AN
2		ŜA
3		MAN
4		A
5		Di
6		E
7		EN
8		NA
9		NU
10		BI
11		RI

III. Dataset Preprocessing and Augmentation

The YOLOv8 complex solution, developed by Ultralytics, is accessible on the GitHub platform. The user has provided a numerical reference without any accompanying text. The system consists of a neural network that has been effectively trained and associated scripts for training and detection. Including square input

images, accompanied by labels in .txt format, is a crucial prerequisite. The dataset underwent annotation, pre-processing, and augmentation procedures utilizing the Roboflow platform. The dataset has a total of 882 tagged images, accompanied by 5114 annotations. In our study, we employed several preprocessing techniques, namely resizing the image to dimensions of 640x640 pixels, applying auto-orientation, auto-adjustment, and tiling the image into a 2x2 grid. The augmentation techniques employed include rotation within the range of -15° to +15°, brightness adjustment within the range of -25% to +25%, Mosaic effect, and bounding box rotation within the range of -15° to +15°.

For pre-processing, auto-orient works to change the directions of the pixels in the image, which leads to changing the objects' directions and helps detect the target if the image is rotated. Implementing automatic constraints would enhance our neural networks' ability to comprehend the objects' nature. Contrast preprocessing enhances the clarity of edges by amplifying the contrasts between neighboring pixels. Tile(2x2) was used to split images into tiles to improve accuracy on small objects, and we used the 2 rows and two columns split (2x2). Resize (640x640px) because the size of the accepted input image for yolov8 is 640x640px.

Data augmentation is effective because it enhances the semantic breadth of a dataset. A widely used data augmentation technique involves applying random rotations to the data. The source image undergoes a random rotation in either a clockwise or anticlockwise direction, altering the object's position within the frame. Significantly, in object detection problems, updating the bounding box to include the generated object is imperative. The brightness Introduces fluctuations in image luminosity to enhance the adaptability of the model to variations in lighting conditions and camera configurations. A mosaic is a decorative art form that involves creating images or patterns by arranging small pieces of colored glass, stone, or other materials. The process involves amalgamating four source photos into a single composition. This accomplishes several tasks: The simulation generates four random

crops, preserving the relative scale of the objects about the image. This can enhance model performance in situations involving occlusion and translation. It merges classes that may require simultaneous consideration in the training dataset. Depending on the images, the number of items in your photographs can vary, resulting in a mosaic output that may comprise between zero and four bounding boxes. Bounding box rotation augmentation involves modifying the content of a source image's bounding boxes to create fresh training data. By doing this, you will have enhanced authority in generating training data that is better aligned with the specific conditions of the problem.

IV. Dataset splitting

The dataset was split into different ratios (three versions (V) of splitting) of training, validation, and testing, as shown in Table 3.5.

Table 3.5: Dataset Split for Neo-Assyrian Signs Detection

Version	Train – Val – Test	# of all images	Preprocessing	augmentation
1	2388 - 224 – 116	2728	1- Auto-orient. 2- Resize (640x640 px) 3- Auto-adjust contrast. 4- Tile.	1- Rotation. 2- Brightness. 3- Mosaic. 4- Bounding Box Rotation.
2	2247 - 87 – 46	2380	1- Auto-orient. 2- Auto-adjust contrast.	
3	1854 – 176 - 88	2118		

Different ratios were chosen for training and testing. Firstly, this is because the data set is unbalanced. The number of labels for each class is not equal, and there are differences in large numbers. This is only to improve training and add more images to the few classes. We have a small number of images about the number of classes because for the training to be highly accurate, each class must have at least 1500, and this is not available. The number of annotations for each class, A=1,289, SHA=843, NI=575, E=502, AN=496, MAN 438, NU=285, RI=267, EN=224, BI=118, and DI=77.

V. Dataset Training

Training scripts incorporating variations in the dataset are exclusively employed in the YOLOv8 framework. The script's editing process is essential to export the anticipated images and their labels as separate files, exhibit them visually, and use them beyond the confines of prediction notebooks. The YOLOv8 architecture was trained on Google Collaboratory using the CUDA programming language for 100 and 50 epochs, correspondingly. The training was conducted using a GPU Tesla T4, equipped with 40 multiprocessors and a total RAM capacity of 15109 MB. The total time taken for training for a 100 epoch ranges from 2 to 3.5 hours, while for a 50 epoch, the training time reaches 1.5 hours.

3.3.1.3 The Hebrew Signs Detection

I. Images Dataset Creation

A substantial number of manually annotated input photographs is necessary to train models of superior quality. An assemblage of Hebrew inscriptions has been gathered. The collected data set comprises images sourced from the Old Testament book depicting the genesis of creation, as well as images of a recently discovered Hebrew manuscript located in Iraq. Furthermore, a collection of deceased photographs was posted on the internet. The data collection process faced challenges due to the presence of Hebrew texts with overlapping and diminutive letters, making their identification somewhat arduous. Additionally, identifying Hebrew letters was limited to 22, necessitating a diverse range of visual data and a substantial quantity to enhance the training. To acquire a greater quantity of labelled data, there are two options: either increase the size of the dataset or employ data augmentation techniques to amplify the dataset's size. Expanding the dataset can improve the model's capacity to precisely detect things. Figure 3.14 displays several images of Hebrew texts.

Figure 3.14: Some pictures of Hebrew texts

II. Dataset Labelling

All 22 Hebrew letters were identified, as shown in Table 3.6, using the roboflow annotation tool, which is one of the tools used in labelling.

Verbal signs are printed in familiar English letters or any other language to make the signs easier to read. For example, the tag (א) is pronounced as Alef, and the remaining tags (ב) are pronounced as Bet.

Table 3.6: The Letters and its pronunciations

L. No	Hebrew Letters	Letters Name
1	א	Alef
2	ב	Bet
3	ג	Gimel
4	ד	Dalet
5	ה	He
6	ו	Vav
7	ז	Zayin
8	ח	Chet
9	ט	Tet
10	י	Yod
11	כ, ך	Kaf

12	ל	Lamed
13	מ, ם	Mem
14	נ , ן	Nun
15	ס	Samech
16	ע	Ayin
17	פ , ף	Feh
18	צ, ץ	Tsadeh
19	ק	Qof
20	ר	Resh
21	ש	Shin
22	ת	Tav

III. Hebrew Dataset Preprocessing and Augmentation

The dataset underwent annotation, pre-processing, and augmentation procedures utilizing the Roboflow platform. The dataset has 471 tagged images, accompanied by 21,699 annotations. In this thesis, was employed several preprocessing techniques, namely resizing the image to dimensions of 640x640 pixels, applying auto-orientation, and auto-adjustment. The augmentation techniques employed include rotation within the range of -15° to +15°, brightness (-15% - +15 %), Hue (-15% - +15 %), saturation (-31% - +31 %), and noise up to 1.8% of pixels augmentation. The preprocessing is the same as previous work for detecting the Neo-Assyrian signs, but the difference used augmentation hue and noise instead of mosaic. Hue augmentation is a process that randomly modifies the color channels of an input image, which pushes a model to explore different color schemes for objects and scenes in the input images. Saturation augmentation is akin to hue adjustment, but it explicitly modifies the vibrancy of the image. Noise is a type of flaw that machines find particularly vexing compared to human comprehension. While humans can disregard noise or incorporate it into the appropriate context effortlessly, algorithms encounter difficulty. The phenomenon referred to as adversarial attacks originates from manipulating pixels in a manner that is invisible to humans yet significantly impacts a neural network's capacity to anticipate outcomes accurately.

IV. Hebrew Dataset splitting

The dataset was split into different ratios (two versions (V) of splitting) of training, validation, and testing, as shown in Table 3.7.

Table 3.7: Hebrew Dataset Splitting

Version No.	Train – Val - Test	# of all images	Pre-processing	Augmentation	Background images
1	993 - 94 – 46	1133	1- Auto-orient. 2- Resize (640x640 px) 3- Auto-adjust contrast.	1- Rotation. 2- Brightness. 3- Hue. 4- Noise.	Without Background images
2					With Background images

Different numbers of images were chosen for training and testing. The two versions involved the utilization of three pre-processing techniques (auto-orient, resize, auto-adjust) and four augmentation techniques (rotation, brightness, hue, noise), resulting in the generation of 1133 images (train=993 - 88%, val=94 - 8%, test=46 - 4%) using the roboflow tool. In the second version, the background images were added within dataset images.

V. Hebrew Dataset Training

As a Neo-Assyrian dataset, the data was also trained on COLAB and 100 epochs. The total time for training for a 100 epoch ranges from 2 to 3.5 hours.

3.3.2 Datasets Summary

Figure 3.8 shows the summary of all datasets used in all models.

Table 3.8: Summary Of All Datasets

Model	# of all images	Pre-processing	Balancing
CSV Cunieform Langueges	139,421	1- Balancing dataset 2-Special character removal 3-Punctuation Removal 4-Encoding	1-Up Sampling 2-Up-Down Sampling

Model	**# of all images**	**Pre-processing**	**Augmentation**
Cuneiform Tablets	Sumerian = 1435 Assyrian = 1327 Babylonian= 1321	1-Normalization 2-Noise reduction	Rotation
New-Assyrian	882	1- Auto-orient. 2- Resize (640x640 px) 3- Auto-adjust contrast. 4- Tile.	1- Rotation. 2- Brightness. 3- Mosaic. 4- Bounding Box Rotation.
Hebrew	471	1- Auto-orient. 2- Resize (640x640 px) 3- Auto-adjust contrast.	1- Rotation. 2- Brightness. 3- Hue. 4- Noise.

Chapter 4

EVALUATION AND PERFORMANCE RESULTS

4.1 INTRODUCTION

This chapter presents all the results for four parts: cuneiform language identification, classification tablets, Neo-Assyrian signs detection, and Hebrew letters detection using the Yolov8 object detection algorithm.

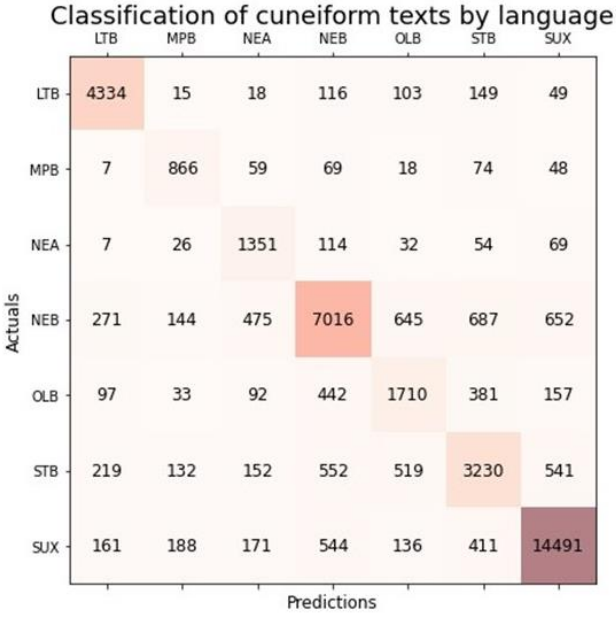
4.2 THE PERFORMANCE OF CUNEIFORM LANGUAGE IDENTIFICATION MODELS

4.2.1 Confusion Matrix Evaluation Results

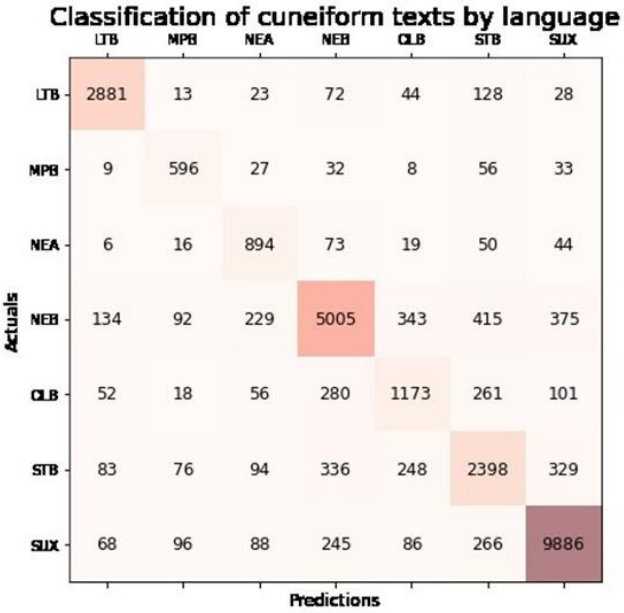
These metrics encompass quantifying true and false positive elements and generating a comparative graph to analyze their respective values. As previously stated, the imbalanced data set was enhanced by employing the ratio sampling technique, up-sampling, and up-down-sampling.

Figure 4.1 presents the confusion matrix for ANN and several training-test ratios and training epochs for ANN.

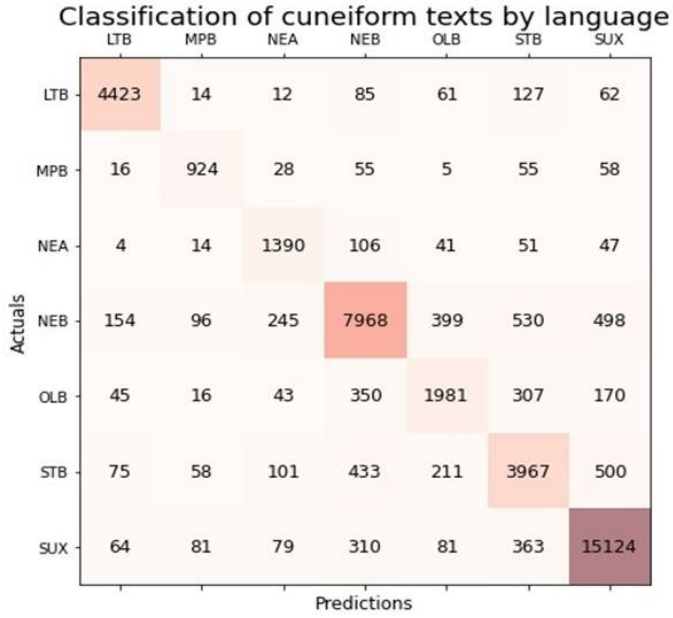
Ratio Splitting Sampling



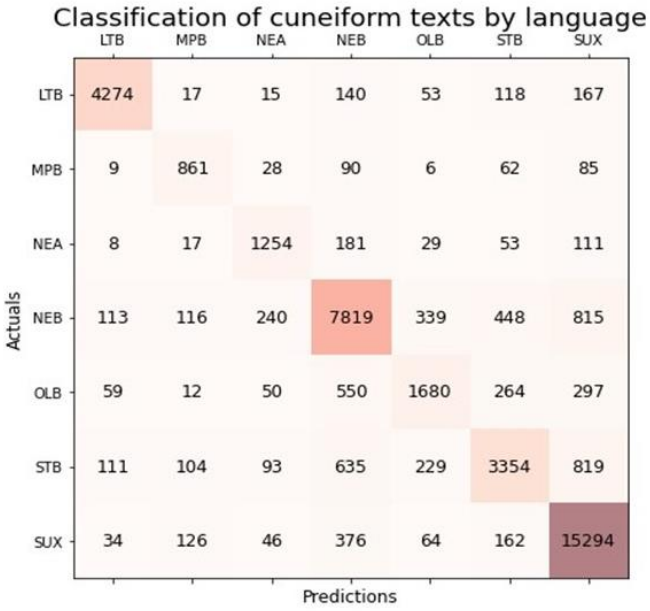
(a) (550-62-7), (70-30), 50 epochs



(b) (550-164-7), (80-20), 50 epochs

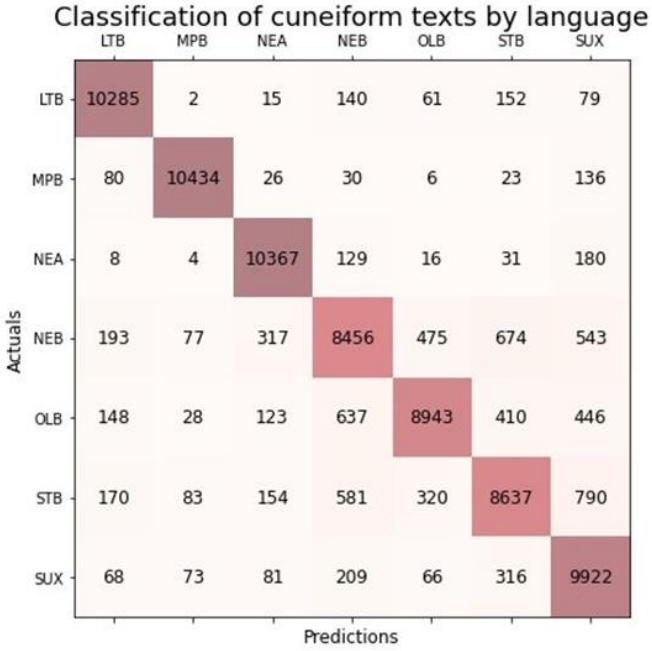


(c) (550-264-7), (70-30), 50 epochs

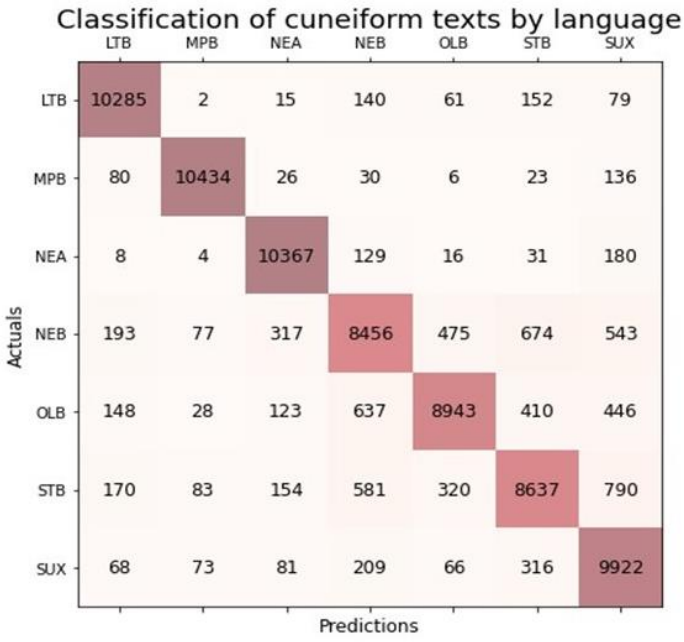


(d) (550-100-200-300-7), (70-30), 100 epochs

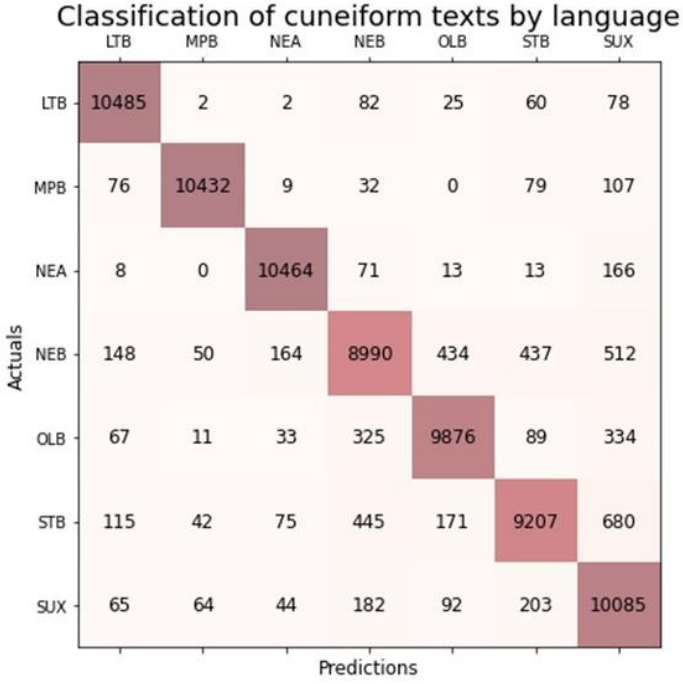
UP-Sampling Method



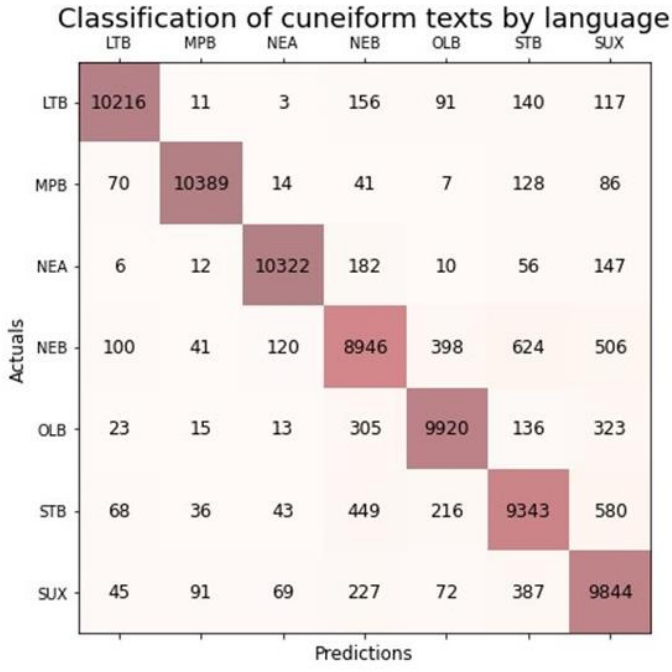
(e) (550-62-7), 50 epochs



(f) (550-62-7), 50 epochs

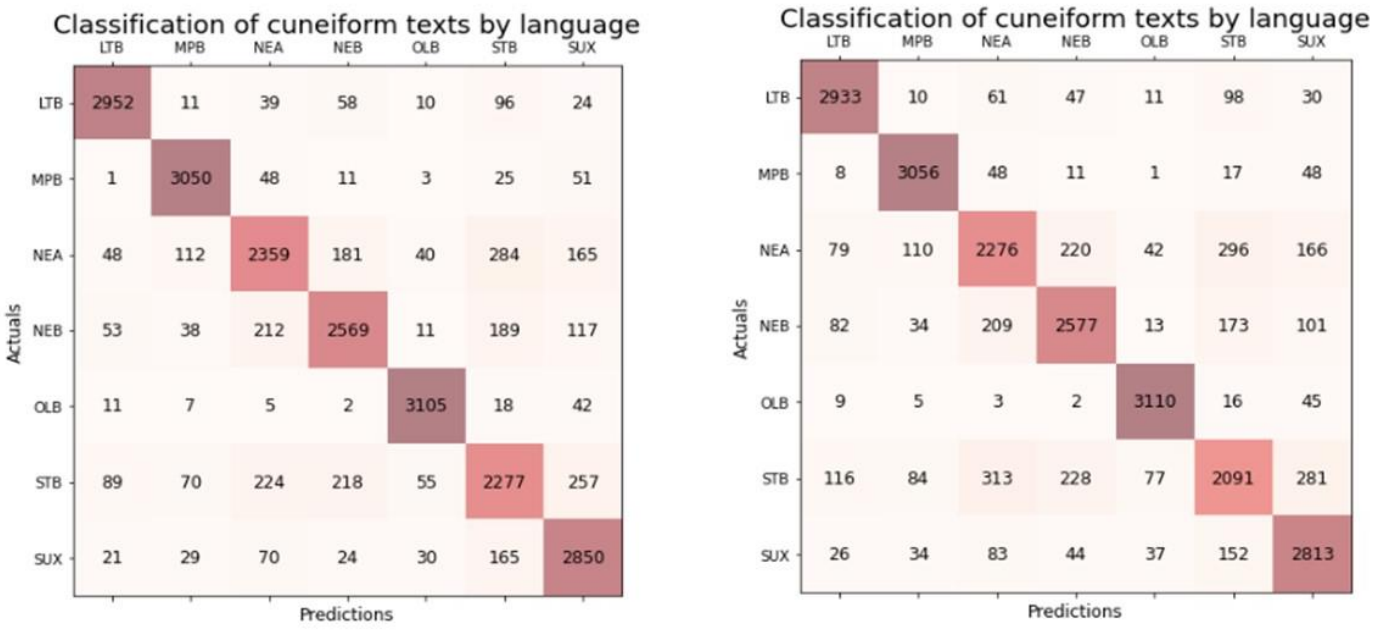


(g) (550-264-7), 100 epochs



(h) (550-100-200-300-7), 50 epochs

Up-Down Sampling



(i) (550-264-7), 150 epochs **(j) (550-100-200-300-7), 150 epochs**

Figure 4.1: ANN & DNN confusion matrix. (a),(b),(c), and (d) Ratio Splitting, (e), (f),(g), and (h) Up-Sampling, and (i) and (j) UP-Down Sampling

Evaluating Intelligent Cuneiform Language Identification Performance Table 4.1 presents the optimal outcome achieved by ANN while employing a 93% 80-20 train-test splitting methodology with 50 and 100 epochs, respectively.

Table 4.1: Optimal outcome is achieved with the various train-test ratios

ANN Module	Training-Testing	Epoch	Accuracy (%)	Sensitivity (%)	precision (%)
Ratio Splitting Sampling					
(550-62-7)	70-30	50	0.79	0.71	0.75
		100	0.79	0.75	0.70
		150	0.79	0.76	0.72
	80-20	50	0.79	0.75	0.70
		100	0.71	0.75	0.70

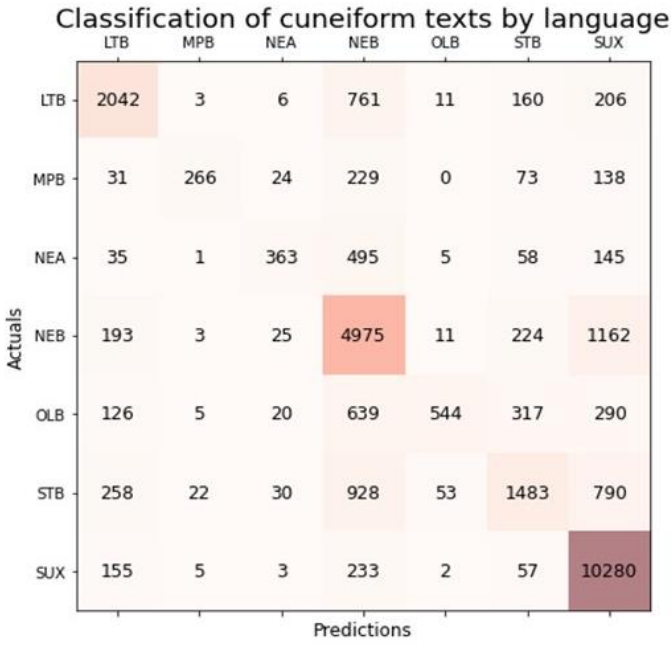
		150	0.79	0.71	0.71
	90-10	50	0.78	0.75	0.68
		100	0.78	0.75	0.69
		150	0.79	0.76	0.71
(550-164-7)	70-30	50	0.80	0.7457	0.72
		100	0.81	0.7557	0.74
		150	0.81	0.75	0.74
	80-20	50	**0.82**	**0.75**	**0.83**
		100	0.81	0.75	0.73
		150	0.81	0.75	0.74
	90-10	50	0.80	0.74	0.72
		100	**0.82**	**0.75**	**0.74**
		150	0.80	0.74	0.72
(550-264-7)	70-30	50	0.82	0.78	0.75
		100	0.82	0.78	0.75
		150	0.83	0.78	0.76
	80-20	50	0.82	0.78	0.75
		100	0.82	0.78	0.75
		150	0.82	0.77	0.75
	90-10	50	**0.86**	**0.82**	**0.81**
		100	0.82	0.78	0.75
		150	0.82	0.77	0.75
(550-100-200-300-7)	70-30	50	0.82	0.76	0.77
		100	**0.83**	**0.76**	**0.78**
		150	0.78	0.73	0.70
	80-20	50	0.82	0.77	0.75

		100	0.82	0.77	0.75
		150	0.79	0.75	0.72
	90-10	50	0.79	0.75	0.72
		100	0.80	0.73	0.73
		150	0.79	0.74	0.73
UP- Sampling Method					
(550-62-7)	80-20	50	**0.92**	**0.92**	**0.92**
		100	0.82	0.82	0.82
		150	0.82	0.82	0.82
(550-164-7)		50	**0.92**	**0.92**	**0.92**
		100	0.90	0.90	0.90
		150	0.90	0.90	0.90
(550-264-7)		50	0.92	0.92	0.92
		100	**0.93**	**0.93**	**0.93**
		150	0.92	0.92	0.92
(550-100-200-300-7)		50	**0.93**	**0.93**	**0.93**
		100	0.92	0.92	0.92
		150	0.92	0.92	0.92
UP- Down Sampling Method					
(550-62-7)	80-20	50	0.79	0.79	0.79
		100	0.80	0.79	0.80
		150	0.80	0.79	0.80
(550-164-7)		50	0.84	0.84	0.84
		100	0.84	0.84	0.84
		150	0.85	0.85	0.85
(550-264-7)		50	**0.86**	**0.86**	**0.86**

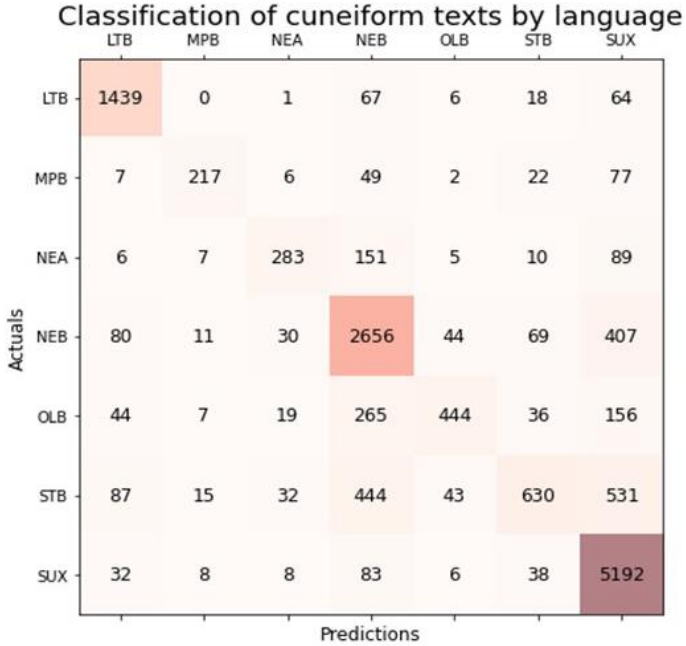
		100	**0.86**	**0.86**	**0.86**
		150	**0.86**	**0.86**	**0.86**
(550-100-200-300-7)		50	0.84	0.84	0.84
		100	0.84	0.84	0.84
		150	0.84	0.84	0.84

Figure 4.2 contains the confusion matrix for the rest of the ML algorithms and several training-test ratios, each trained using the Siket-learn library.

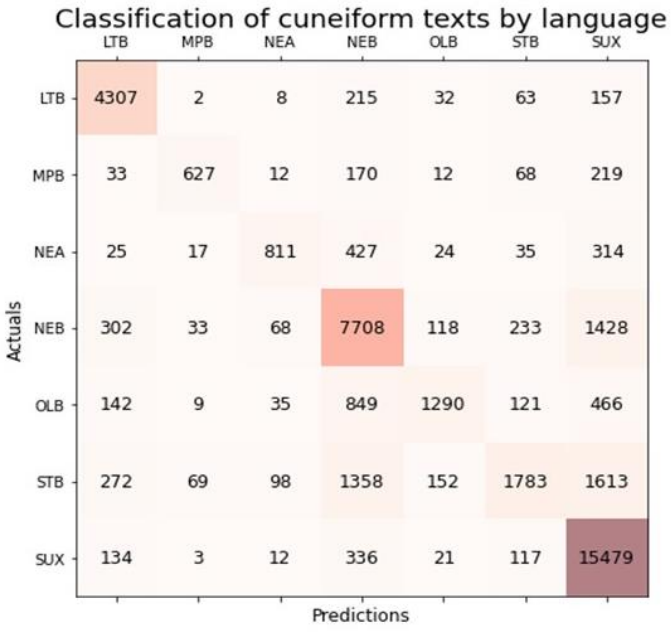
Ratio Splitting Sampling



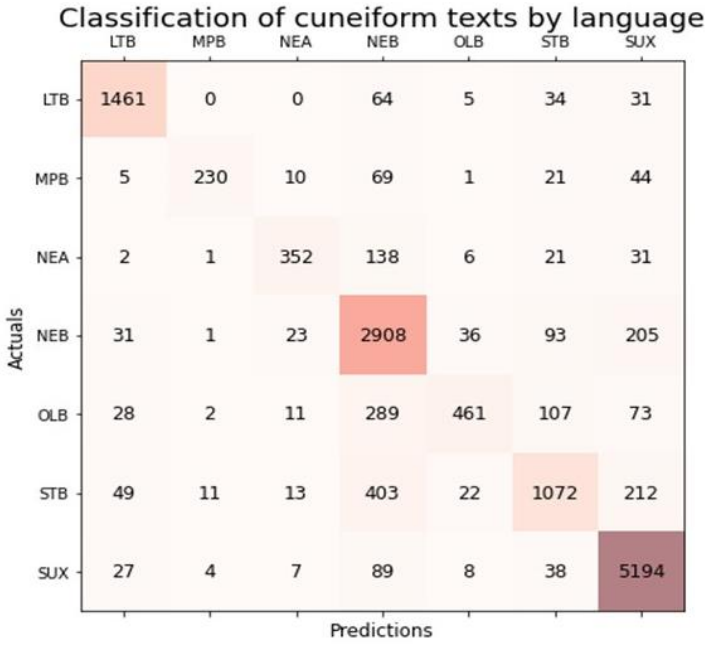
(a) SVM - (90-10)



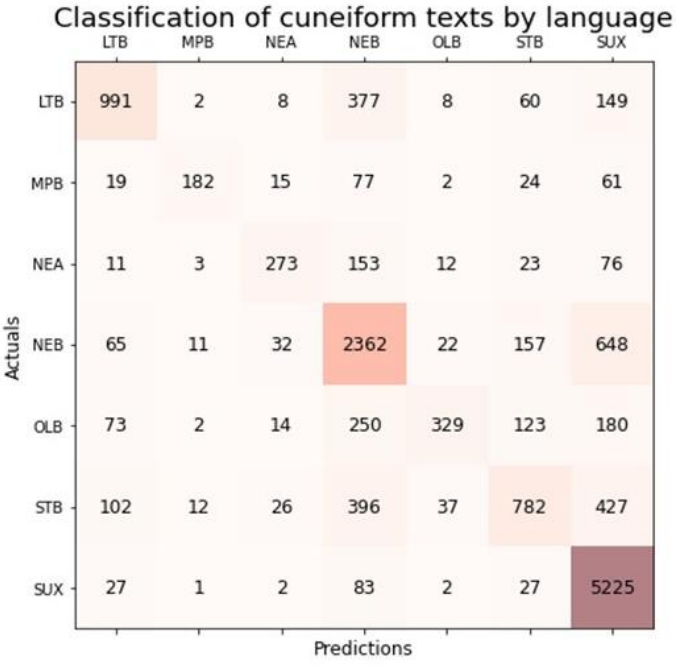
(b) KNN - (90-10)



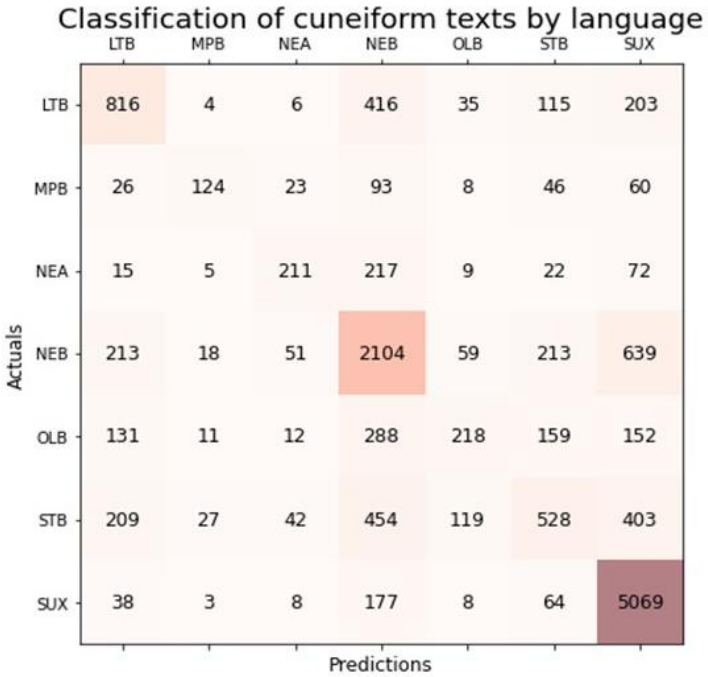
(c) NB - (80-20)



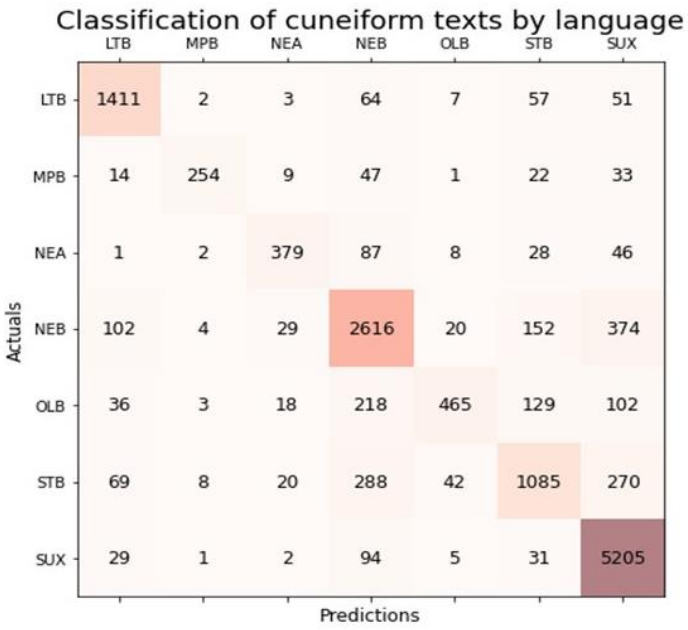
(d) RF - (90-10)



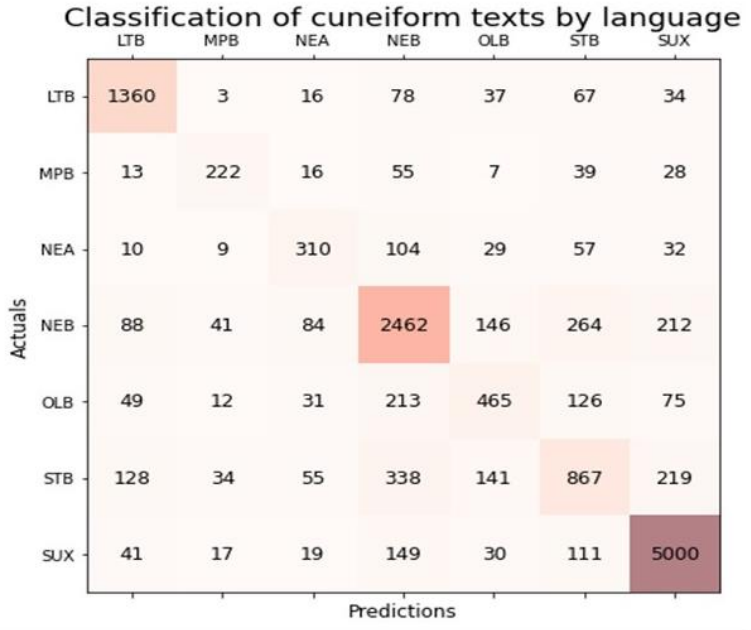
(e) Gradient Boost - (90 – 10)



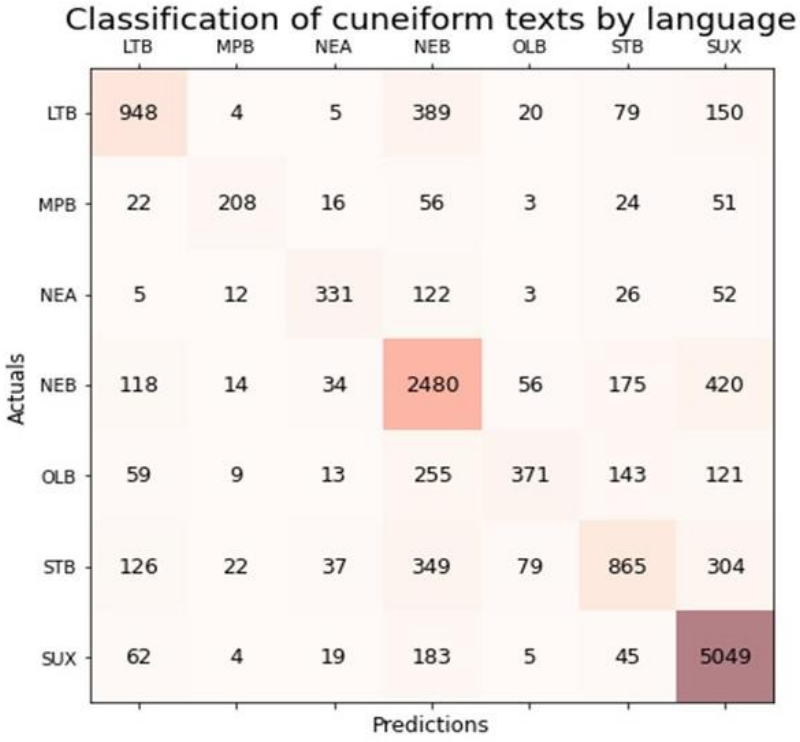
(f) Adaboost – (90 – 10)



(g) XGBoost (90 - 10)



(h) DT – (90-10)



(i) LR (90 – 10)

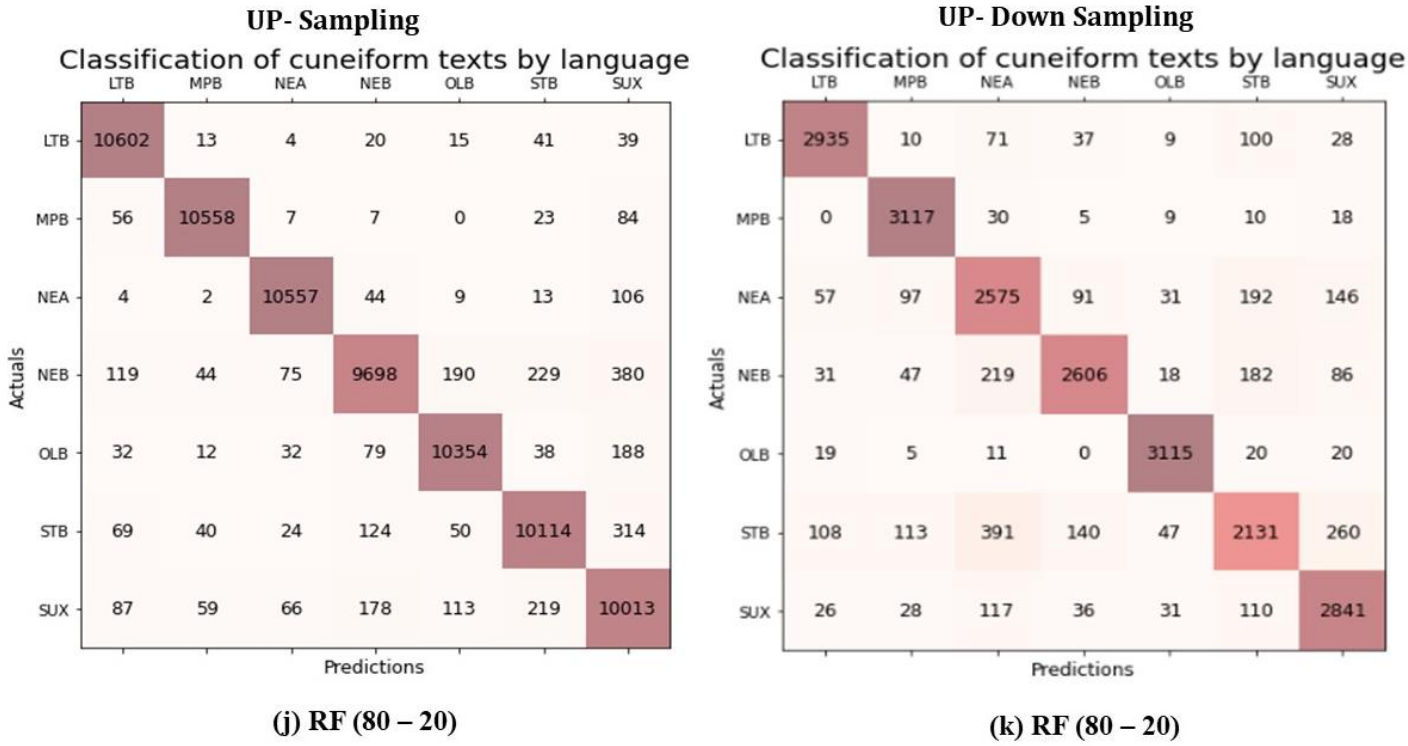


Figure 4.2: Confusion matrix for the best results of the rest of ML algorithms with training – test ratios

The Adam Optimization technique is employed to compute the ANN, K-NN, NB, LR, DT, RF, SVM, Adaboost, Gradient Boosting, XGBoost, and n-Gram NLP algorithms, as presented in Table 4.2, to achieve intelligent coniform language identification with enhanced accuracy, sensitivity, and specificity. The Random Forest algorithm yields the optimal outcomes.

Table 4.2: The Evaluation Of The ML

Train-Test	Algorithm	Accuracy	Sensitivity	precision
Ratio Splitting Sampling				
70-30	Support Vector Machine	0.71	0.58	0.75
	K-Nearest Neighbor	0.77	0.68	0.77
	Naïve Bayes	0.61	0.52	0.58
	Random Forest	**0.83**	0.76	0.83
	Gradient Boosting	0.73	0.58	0.76
	AdaBoost	0.65	0.48	0.58

	XGBoost	0.81	0.81	0.71
	Decision Tree	0.76	0.6614	0.66
	Logistic Regression	0.73	0.61	0.71
80-20	Support Vector Machine	0.72	0.5857	0.75
	K-Nearest Neighbor	0.77	0.7385	0.71
	Naïve Bayes	0.72	0.5857	0.75
	Random Forest	**0.83**	**0.76**	**0.85**
	Gradient Boosting	0.72	0.75	0.58
	AdaBoost	0.65	0.48	0.58
	XGBoost	0.81	0.82	0.71
	Decision Tree	0.76	0.66	0.67
	Logistic Regression	0.73	0.65	0.71
90-10	Support Vector Machine	0.74	0.68	0.78
	K-Nearest Neighbor	0.78	0.69	0.7871
	Naïve Bayes	0.62	0.55	0.59
	Random Forest	**0.83**	**0.77**	**0.8485**
	Gradient Boosting	0.73	0.75	0.58
	AdaBoost	0.65	0.48	0.58
	XGBoost	0.82	0.73	0.83
	Decision Tree	0.77	0.67	0.68
	Logistic Regression	0.73	0.65	0.71
UP- Sampling Method				
80-20	Support Vector Machine	0.88	0.88	0.88
	K-Nearest Neighbor	0.88	0.88	0.88
	Naïve Bayes	0.55	0.55	0.62
	Random Forest	**0.96**	**0.96**	**0.96**
	Gradient Boosting	0.79	0.79	0.79
	AdaBoost	0.62	0.62	0.62
	XGBoost	0.79	0.80	0.79
	Decision Tree	0.94	0.94	0.94
	Logistic Regression	0.69	0.69	0.69
UP- Down Sampling Method				

80-20	Support Vector Machine	0.72	0.54	0.75
	K-Nearest Neighbor	0.77	0.65	0.79
	Naïve Bayes	0.79	0.79	0.79
	Random Forest	**0.87**	**0.86**	**0.87**
	Gradient Boosting	0.78	0.78	0.78
	AdaBoost	0.62	0.62	0.62
	XGBoost	0.78	0.78	0.78
	Decision Tree	0.79	0.79	0.79
	Logistic Regression	0.74	0.62	0.71

After balancing the dataset using the up-sampling method, some results increased, and others decreased. Upon analyzing the outcomes, it becomes evident that the random forest algorithm yields the most favourable results. This algorithm falls under the category of ensemble learning, which aims to enhance performance by partitioning the dataset into several samples and aggregating predictions from diverse models. The performance of the methods ANN, KNN, DT, SVM, and GBoost showed improvement when the dataset size was increased. Regarding the remaining algorithms, namely AdaBoost, XGBoost, and LR, it is observed that their outcomes have exhibited a slight decline.

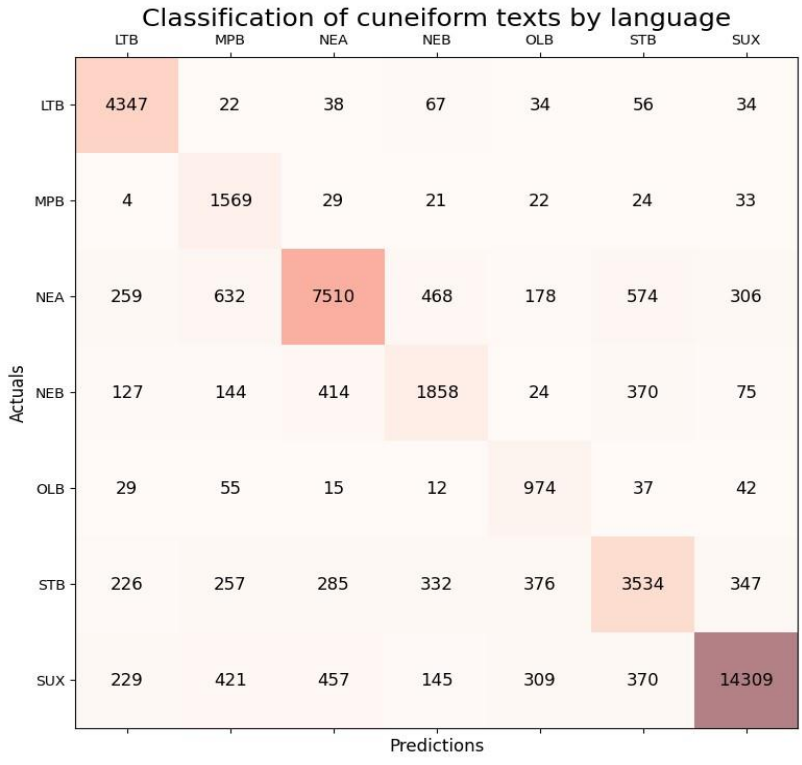
When a model is excessively intricate and closely matches the training data, it results in inadequate generalisations to fresh data. Logistic regression models may encounter difficulties when there are many features or when the relationship between the features and the target is not linear, leading to excessive complexity. Overfitting can result in suboptimal performance on the test set and diminished prediction capability on unseen data, and this happens when increasing the dataset by repetition in some algorithms.

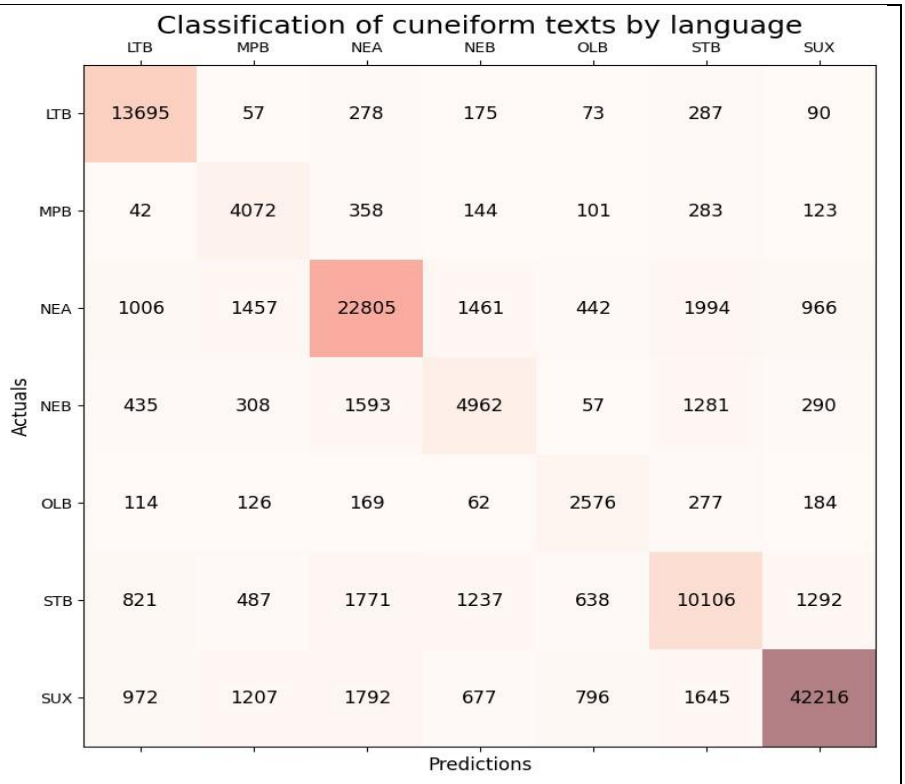
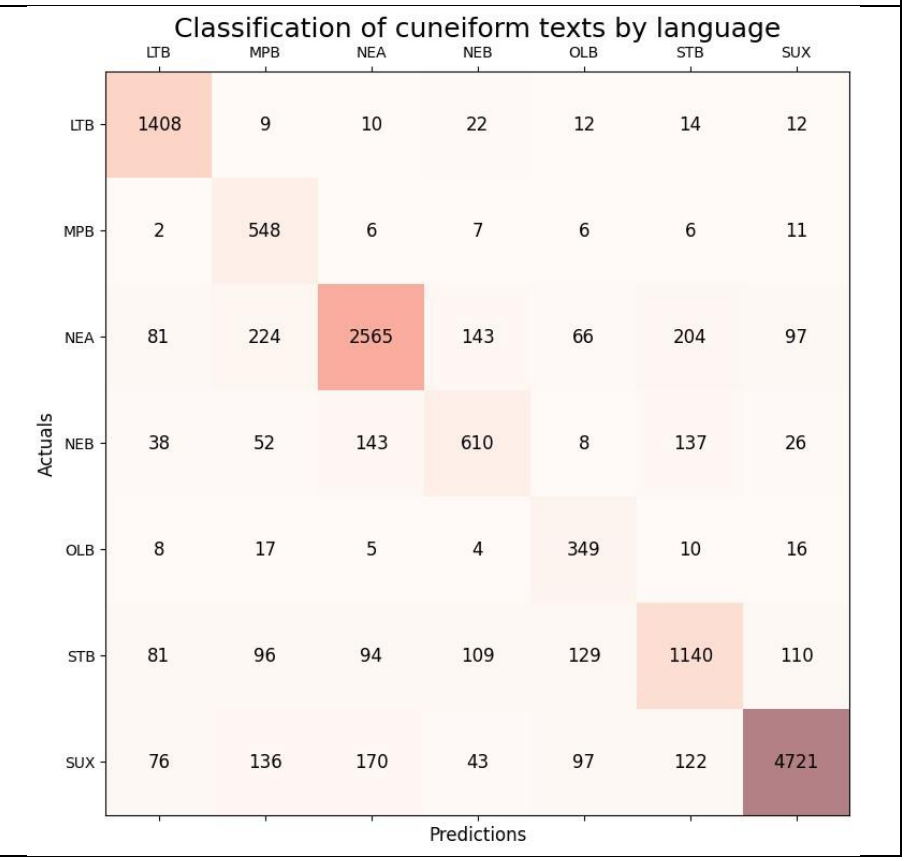
This principle also applies to XGBoost and Adaboost. In certain instances, augmenting data might result in a reduction in precision. In this particular scenario, the duplication of data has hindered the algorithm from adequately learning from a varied and distinct dataset, resulting in a decline in accuracy.

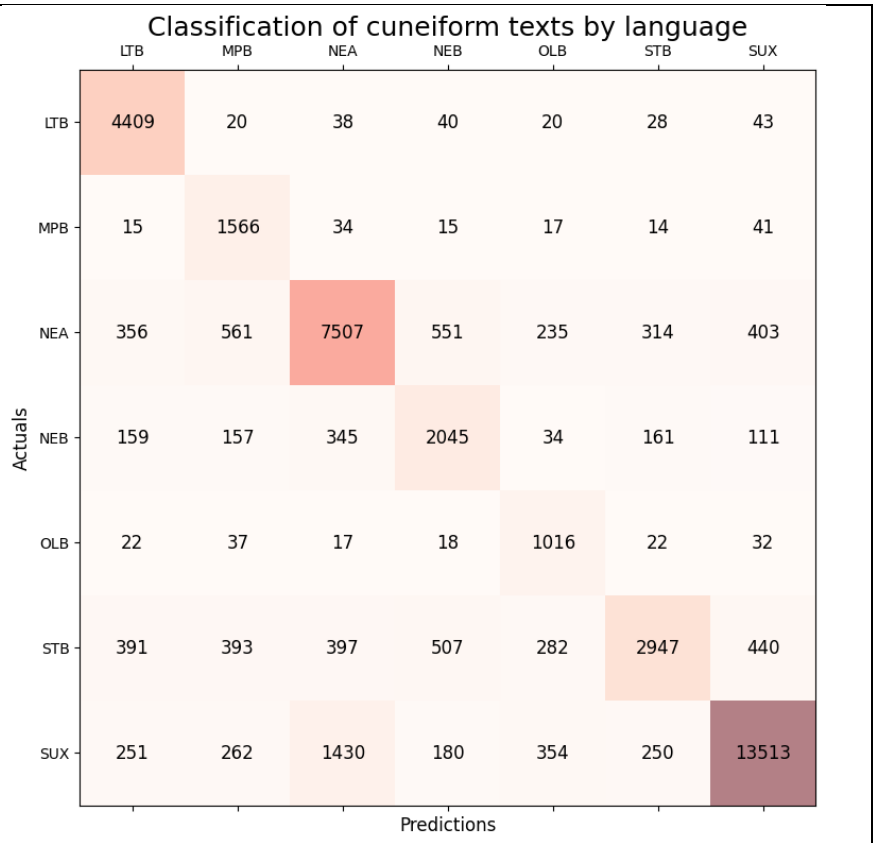
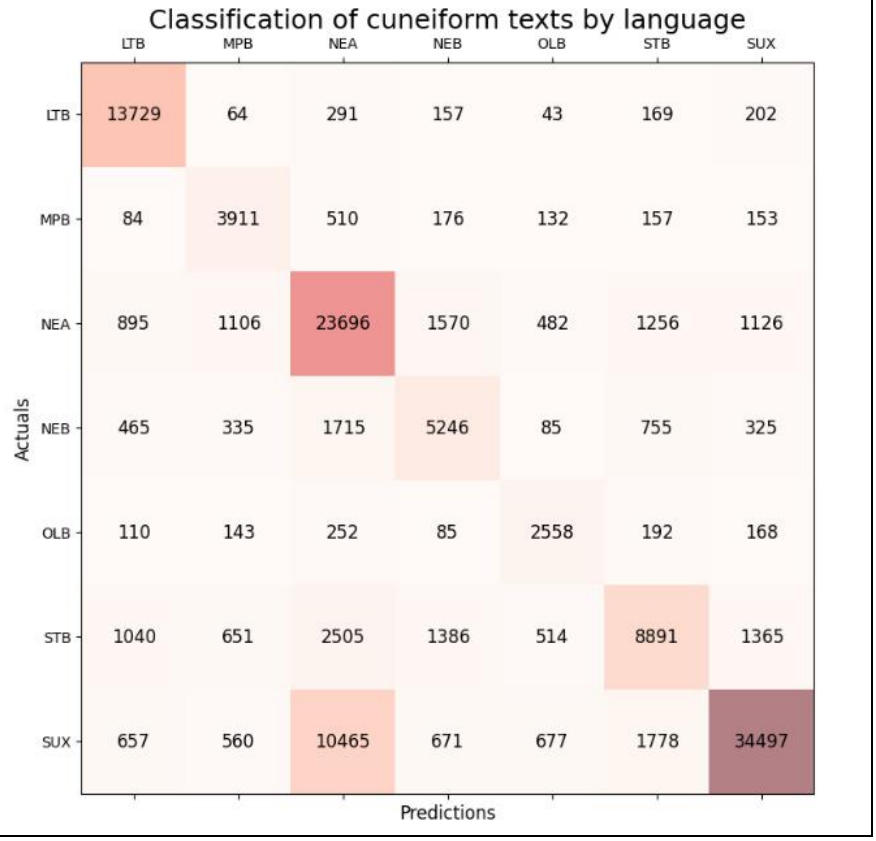
To decrease the overfitting that may happen when increasing the dataset by repetition, the number of texts for the cuneiform writings was reduced and increased to make them equal to the number of texts for the LTB language because it is a language with several texts in the middle between the languages.

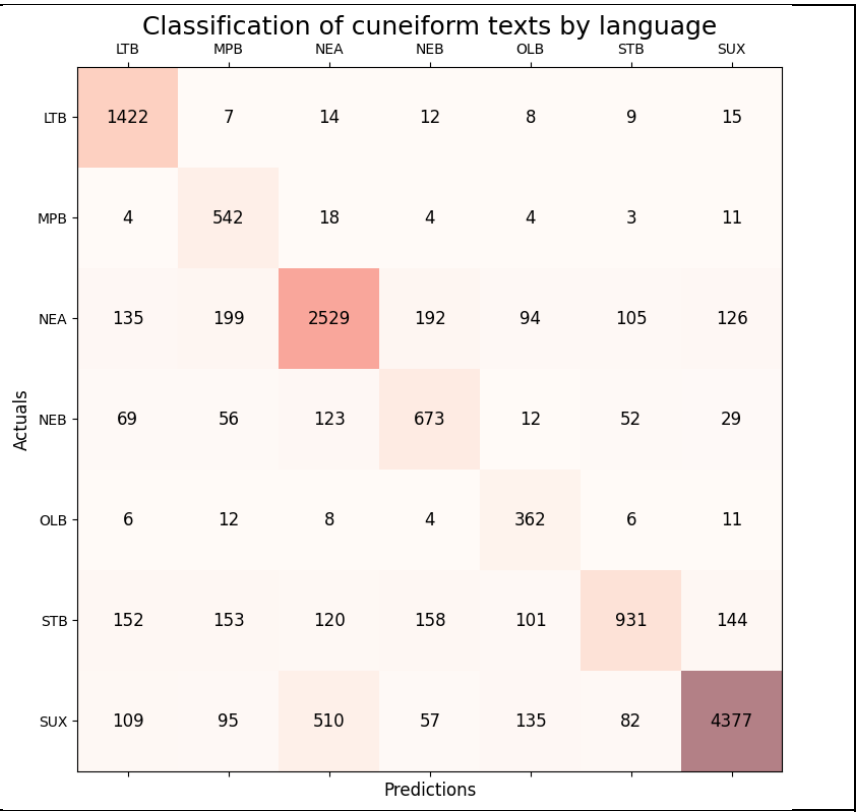
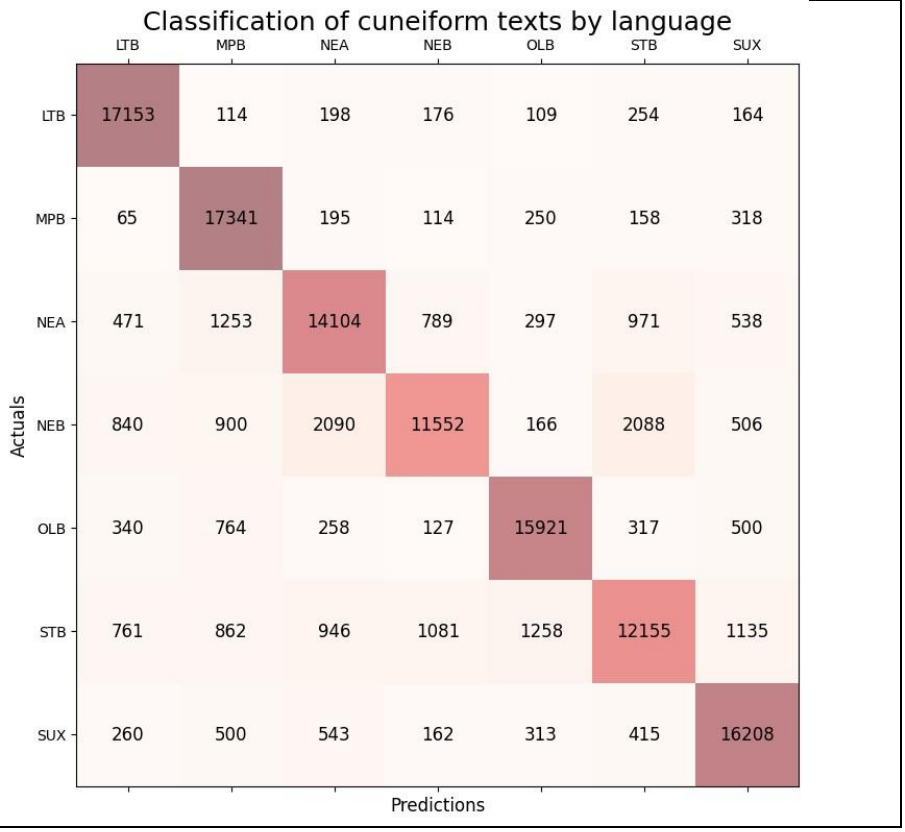
Table 4.3 shows the result of the bi-gram and tri-gram accuracy and confusion matrix for different ratios of split datasets. The best result in bi-gram is 82% for ratio splitting.

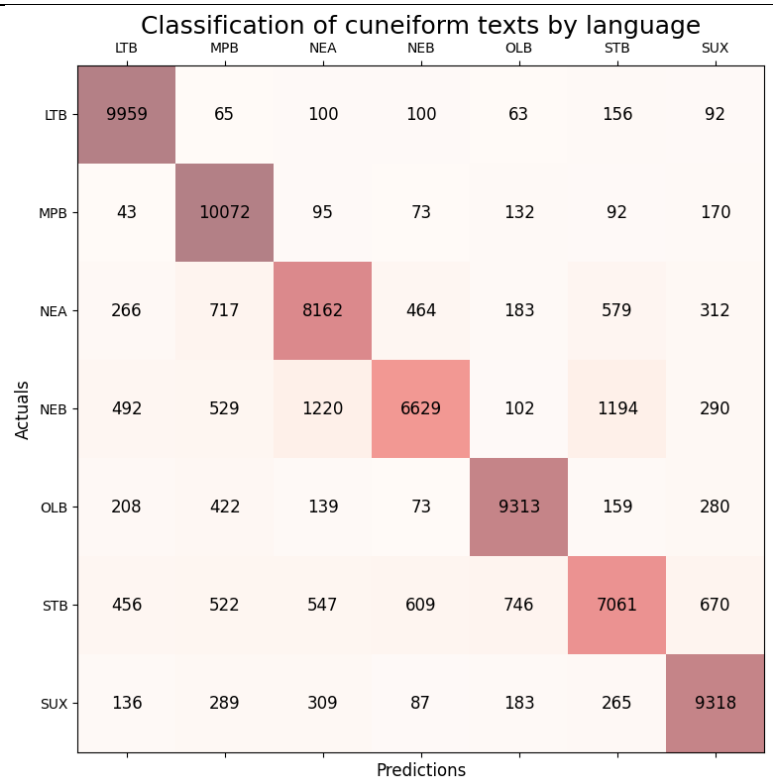
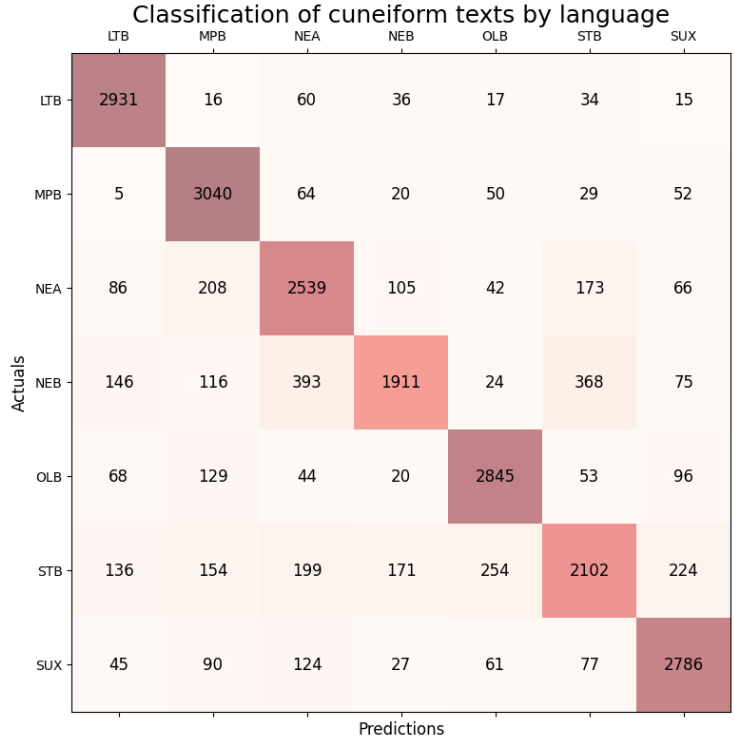
Table 4.3: The result of the Bi-Gram and Tri-Gram accuracy and confusion matrix for ratio splitting

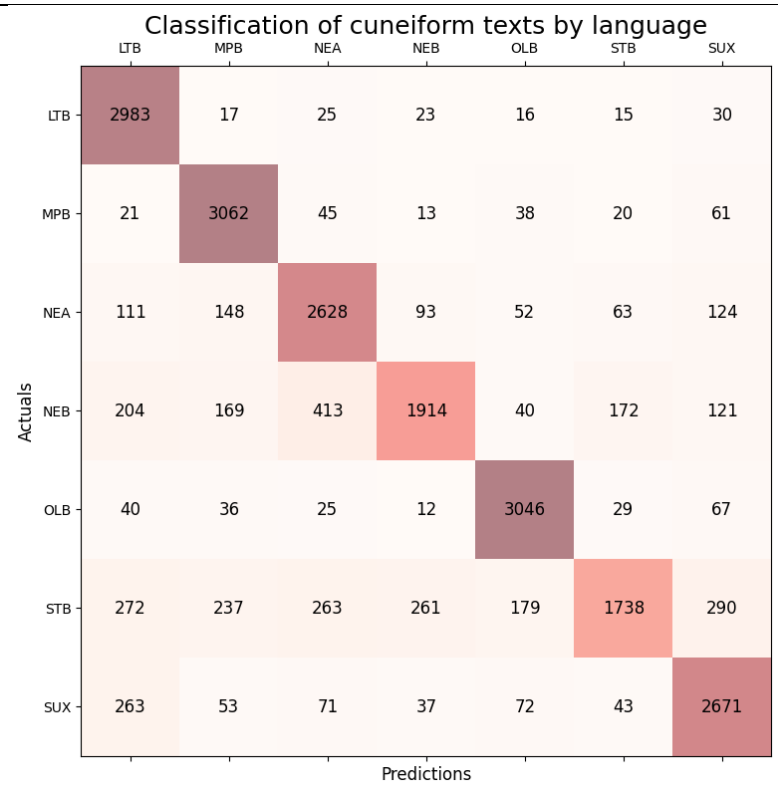
n-gram	**ratio**	**Accuracy**	**Confusion Matrix**
Bi-gram	70-30	81%	

	80-20	78%	
	90-10	81%	

Tri-gram	70-30	79%	
	80-20	72%	

	90-10	77%	
Up Sampling			
Bi-gram	80-20	82%	

Tri-gram		79%	
Up-Down Sambling			
Bi-gram	80-20	81%	

Tri-gram		81%	

The accuracy may remain the same when the training-test ratio is increased from 70-30 to 90-10 because the model might already be well-trained, and additional training data doesn't provide new information. Other reasons include data redundancy, reaching a performance saturation point, effective regularisation, and preventing overfitting. Furthermore, accuracy alone might not capture subtle changes in performance.

4.2.2 MAE, MSE, and RMSE Results

Table 4.4 Table 4.4 presents the mean square error, root square error, and mean absolute error for the testing phases and the corresponding number of epochs for each phase. It should be noted that the MAE yields lower values than the MSE and RMSE across all algorithms.

Table 4.4: The MSE, RMSE, and MAE values for the testing stages

Train-Test	Algorithm	Epochs	MSE	RMSE	MAE
Ratio Splitting Sampling					
	ANN				

	(models design)				
70-30	(550-62-7)	50	1.4528	1.2053	0.4735
		100	1.5176	1.2319	0.4855
		150	1.4285	1.1952	0.4638
80-20		50	1.4294	1.1955	0.4692
		100	1.4829	1.2177	0.4830
		150	1.4185	1.1910	0.4624
90-10		50	1.4849	1.2185	0.4877
		100	1.4757	1.2147	0.4809
		150	1.4261	1.1942	0.4662
70-30	(550-164-7)	50	1.2631	1.1238	0.4258
		100	1.2636	1.1241	0.4143
		150	1.2502	1.1181	0.4142
80-20		50	1.1991	1.0950	0.3991
		100	1.2313	1.1096	0.4093
		150	1.1891	1.0770	0.8812
90-10		50	1.2191	1.1041	0.4044
		100	1.2751	1.1292	0.4187
		150	1.2838	1.1330	0.4292
70-30	(550-264-7)	50	1.1837	1.0880	0.3961
		100	1.1628	1.0783	0.3842
		150	1.2061	1.0982	0.3962
80-20		50	1.0380	1.0188	0.3441
		100	1.1923	1.0919	0.3892
		150	1.2190	1.1041	0.4011
90-10		50	0.9078	0.9528	0.3104
		100	1.1944	1.0929	0.3772
		150	1.2797	1.1312	0.4164
	DNN (Model design)				
70-30	(550-100-200-300-7)	50	1.1647	1.0792	0.3972
		100	1.1841	1.0882	0.3903
		150	1.4077	1.1864	0.4712
80-20		50	1.2738	1.1286	0.4250
		100	1.1789	1.0858	0.3967
		150	1.3401	1.1576	0.4563
90-10		50	1.3349	1.1553	0.4467

		100	1.3073	1.1434	0.4367
		150	1.3002	1.1403	0.4453
70-30	Support Vector Machine		2.3824	1.5435	0.7224
	K-Nearest Neighbor		1.6622	1.2893	0.5417
	Naïve Bayes		9.2143	3.0355	2.4589
	Random Forest		1.0454	1.0224	0.3653
	Gradient Boosting		2.1408	1.4631	0.6691
	AdaBoost		2.9627	1.7212	0.8901
	XGBoost		1.3558	1.1644	0.4368
	Decision Tree		1.6634	1.2897	0.5486
	Logistic Regression		2.3221	1.5238	0.6857
	Di-Gram		0.4539	0.4048	0.1639
	Tri-Gram		0.8839	0.4048	0.1649
80-20	Support Vector Machine		2.3244	1.5246	0.7126
	K-Nearest Neighbor		1.5886	1.2604	0.5212
	Naïve Bayes		3.0380	9.2295	2.4622
	Random Forest		1.0126	1.00632	0.3550
	Gradient Boosting		2.1946	1.4814	0.6849
	AdaBoost		3.0458	1.7452	0.9086
	XGBoost		1.3660	1.1687	0.4363
	Decision Tree		1.5785	1.2564	0.5262
	Logistic Regression		2.2998	1.5165	0.6823
	Bi-Gram		0.2639	0.3841	0.1639
	Tri-Gram		0.3475	0.4048	0.1475
90-10	Support Vector Machine		1.3244	1.2246	0.5126
	K-Nearest Neighbor		1.5716	1.2536	0.5209
	Naïve Bayes		9.2078	3.0344	2.4514
	Random Forest		0.9977	0.9988	0.3528
	Gradient Boosting		2.1998	1.4831	0.6799
	AdaBoost		3.0513	1.7468	0.9050
	XGBoost		1.3164	1.1473	0.4253
	Decision Tree		1.5896	1.2607	0.5328
	Logistic Regression		2.3067	1.5187	0.6832
	Bi-Gram		0.9475	0.3841	0.1475

	Tri-Gram		0.7311	0.3621	0.1311
UP- Sampling Method					
	ANN (Model design)				
80-20	(550-62-7)	50	0.6809	0.8251	0.2306
		100	0.8027	0.9190	0.2658
		150	1.1542	1.0743	0.3827
	(550-164-7)	50	0.6809	0.8251	0.2306
		100	0.6037	0.8090	0.1786
		150	0.6589	0.8117	0.2212
	(550-264-7)	50	0.5180	0.7197	0.1730
		100	0.5027	0.7090	0.1658
		150	0.5085	0.7131	0.1675
	DNN (Model design)				
	(550-100-200-300-7)	50	0.5603	0.7485	0.1836
		100	0.5743	0.7578	0.1872
		150	0.5954	0.7716	0.1858
	Support Vector Machine	1.3244		1.3246	0.4226
	K-Nearest Neighbor	1.6886		1.2704	0.5212
	Naïve Bayes	3.5576		1.8861	1.0842
	Random Forest	0.3485		0.5903	0.1062
	Gradient Boosting	1.5363		1.2394	0.4836
	AdaBoost	3.1334		1.7701	0.9366
	XGBoost	1.5347		1.2388	0.4805
	Decision Tree	0.4287		0.6548	0.1347
	Logistic Regression	2.4400		1.5620	0.7350
	Bi-Gram	0.8470		0.2412	0.1475
	Tri-Gram	0.6211		0.2621	0.1111
UP-Down Sampling Method					
80-20	ANN (Model design)				
	(550-62-7)	50	1.5449	1.2429	0.4964
		100	1.4968	1.2234	0.4823
		150			

	(550-164-7)	50	1.2204	1.1047	0.3912
		100	1.1990	1.0949	0.3791
		150	1.1738	1.0834	0.3706
	(550-264-7)	50	1.0940	1.0459	0.3422
		100	1.0963	1.0471	0.3425
		150	1.0975	1.0476	0.3449
	DNN (Model design)				
	(550-100-200-300-7)	50	1.2434	1.1150	0.3863
		100	1.2571	1.1212	0.3907
		150	1.2243	1.1065	0.3822
	Support Vector Machine	3.1521		1.7754	0.9114
	K-Nearest Neighbor	2.0102		1.4178	0.6191
	Naïve Bayes	3.2591		1.8053	0.9914
	Random Forest	1.1324		1.0641	0.3450
	Gradient Boosting	1.7125		1.3086	0.5358
	AdaBoost	3.1815		1.7836	0.9681
	XGBoost	1.8371		1.3553	0.5674
	Decision Tree	1.7417		1.3197	0.5297
	Logistic Regression	1.5502		0.5969	0.4755
	Bi-Gram	0.9310		0.2511	0.1211
	Tri-Gram	0.846		0.232	0.1347

To summarize, MSE and RMSE emphasise larger errors more strongly, whereas MAE treats all errors equally. RMSE's sensitivity to outliers is attributed to the squaring process, while MAE demonstrates greater resilience when outliers are present.

4.3 THE EVALUATION OF THE CUNEIFORM TABLETS CLASSIFICATION MODEL

The top-1 and top-5 error rates of an algorithm on a classification task measure the performance accuracy of the classification model. The results of curve (C) top1-acc and curve (D) top2-acc are 97% and 100%, respectively, after 200 epochs, with a validation loss curve (B)and a train loss curve (A), as shown in Figure 4.3Figure 4.3.

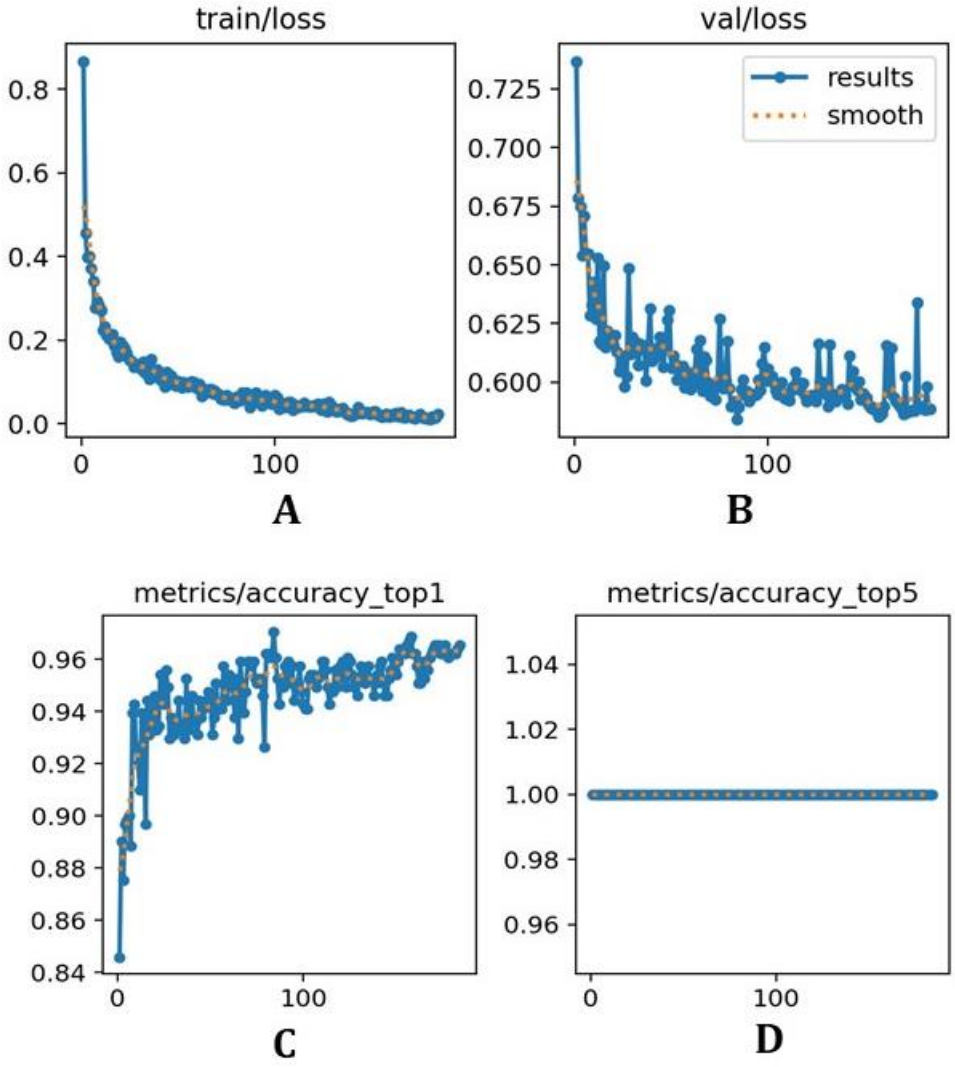


Figure 4.3: The Classification Mode Accuracy. (A) is the train loss , (B) is the val loss results, (C) top1 accurecy abd (D) top5 accurecy

The confusion matrix for the three classes results in 0.95 for Assyrian, 0.99 for Babylonian, and 0.97 for Sumerian ratio TP as shown in Figure 4.4.

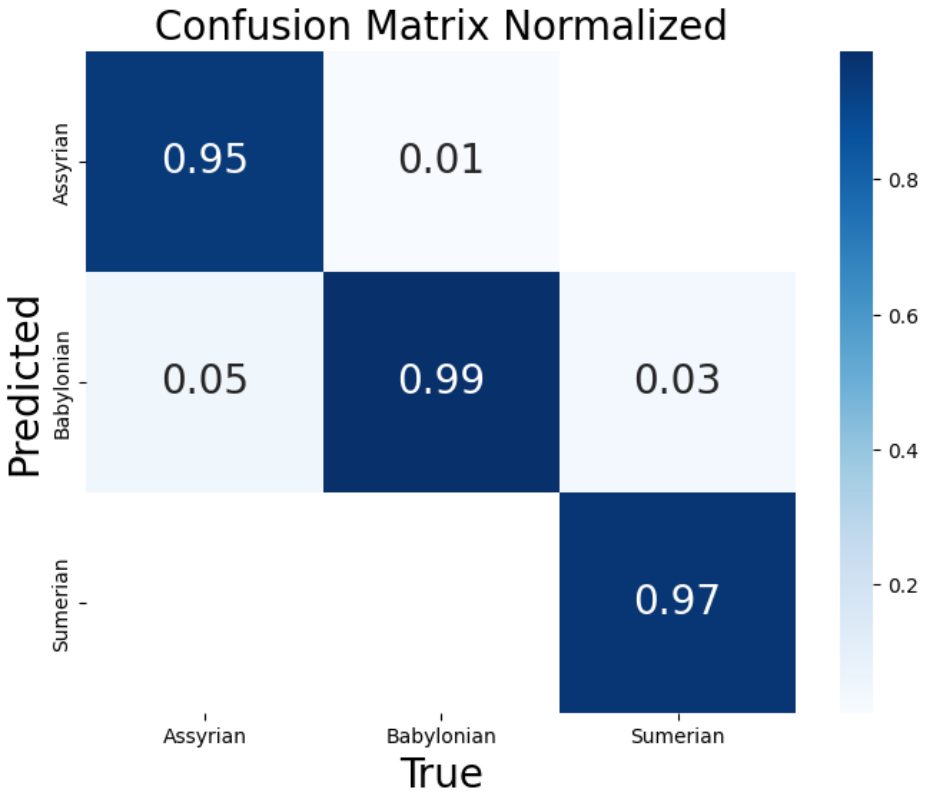


Figure 4.4: Confusion Matrix of Classification Model

The generated results are regarded as being of a very high quality for models. The training and testing results for classification reached the best results; however, we also need to add more photos to the dataset to expand the diversity of images and strengthen the training. The training and testing results for classification obtained better results.

4.3.1 Classification Results

The results depicted in Figure 4.5 pertain to the categorization model. The outcome in (A) indicates with 100% certainty that the image is associated with the Sumerian script and 0% certainty that it is associated with the Sumerian and Babylonian script. In (B), there is an 85% likelihood that the image is of Assyrian origin, 15% of Babylonian, and a 0% likelihood that it is a Sumerian script. The results in (C) are 100% for Assyrian and 0% for Sumerian and Babylonian. The last result in (D) was 100% Babylonian and 0% for the Sumerian and Assyrian script.

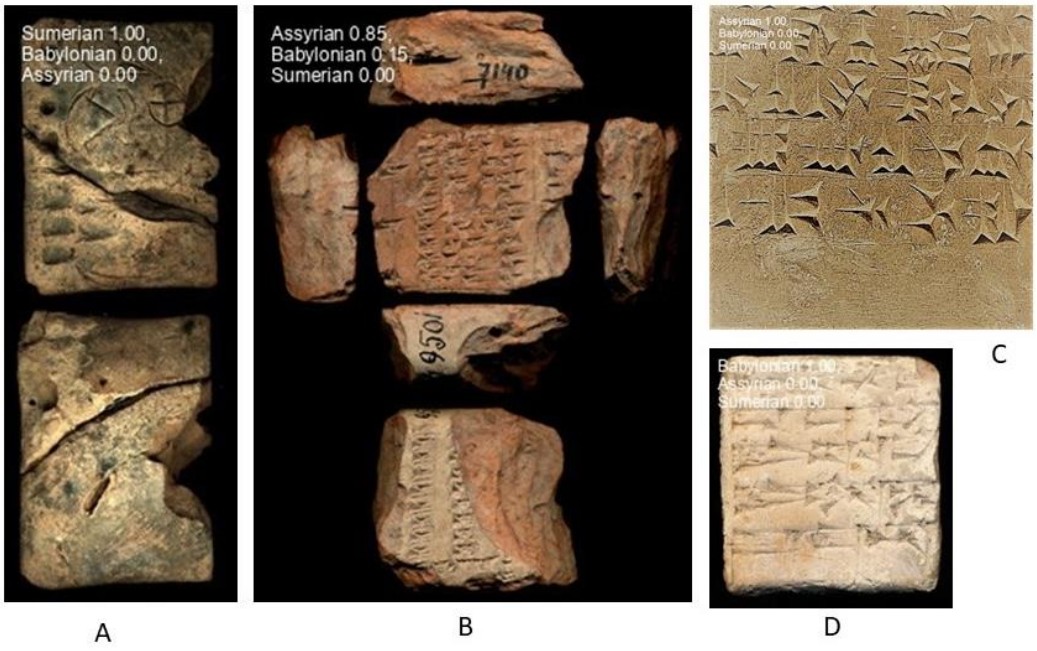


Figure 4.5: The Classification Model Results

4.4 THE EVALUATION OF THE NEO-ASSYRIAN SIGNS DETECTION MODEL

This is present the losses curves and annotation results of neo-assyrian signs detection results.

4.4.1 Dataset Training and Evaluation Results

Figure 4.6 displays a collection of images with corresponding labels used for training purposes and presents the validation outcomes.

(A) TRAIN (B) VALIDATION

Figure 4.6: The pictures with the labels of batch. (A) trained and (B) validation

The detection dataset reaches 5114 annotations by using the Roboflow platform. The number of annotations for each class is shown in Figure 4.7. The highest number of annotations for sign (A) is 1,289 and for the sign (DI) is 77 which is the minimum one.

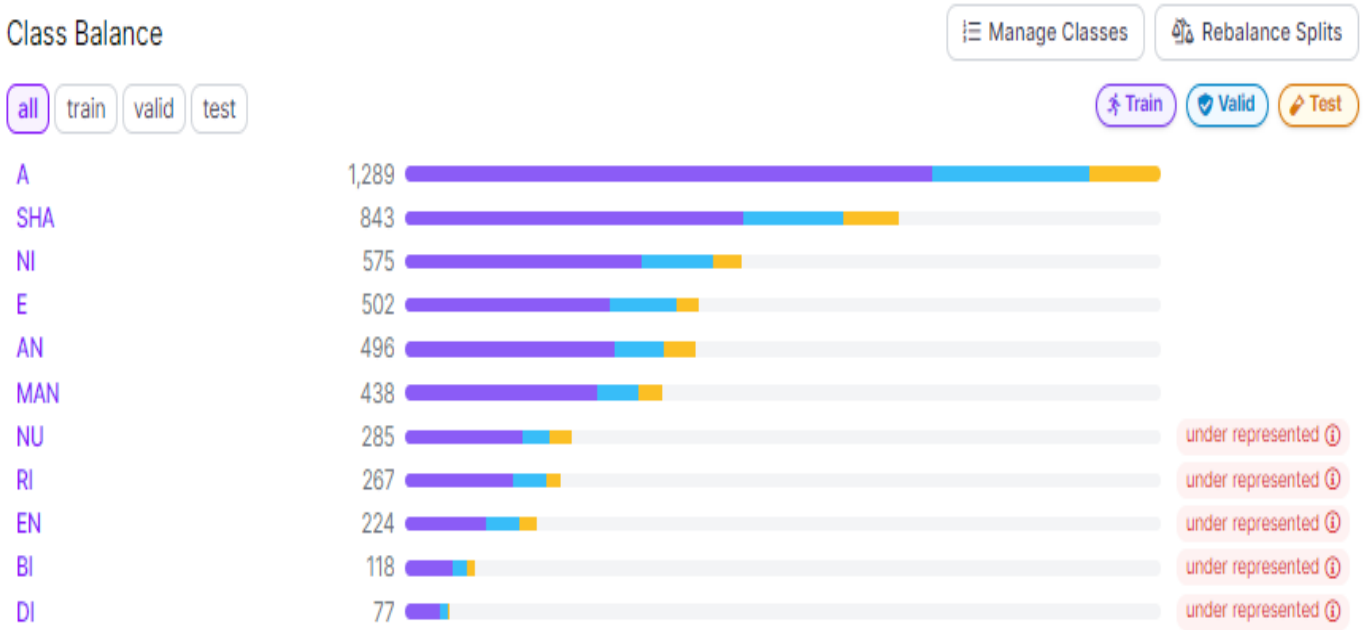


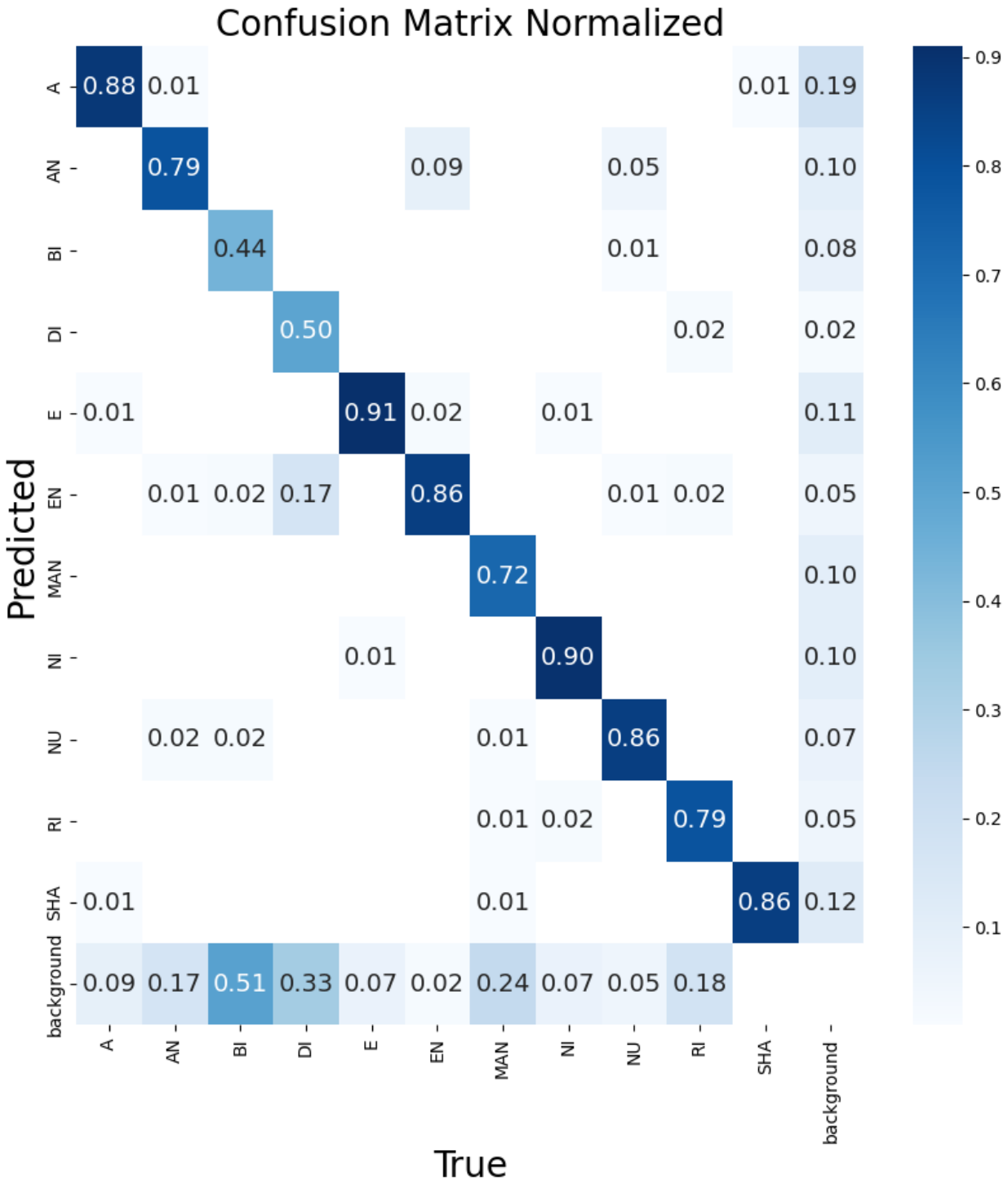
Figure 4.7: The Class Balance of Annotation Neo-Assyrian signs

The most notable outcome achieved in the second edition of the experiment was the mean average precision at 50% intersection over union (mAP50) of 82.7%, the mAP50-90 of 61.3%, a precision rate of 71.319%, and a recall rate of 85.647%, all of which were obtained after 50 epochs. The most favorable evaluation outcomes following the completion of training are presented in Table 4.5.

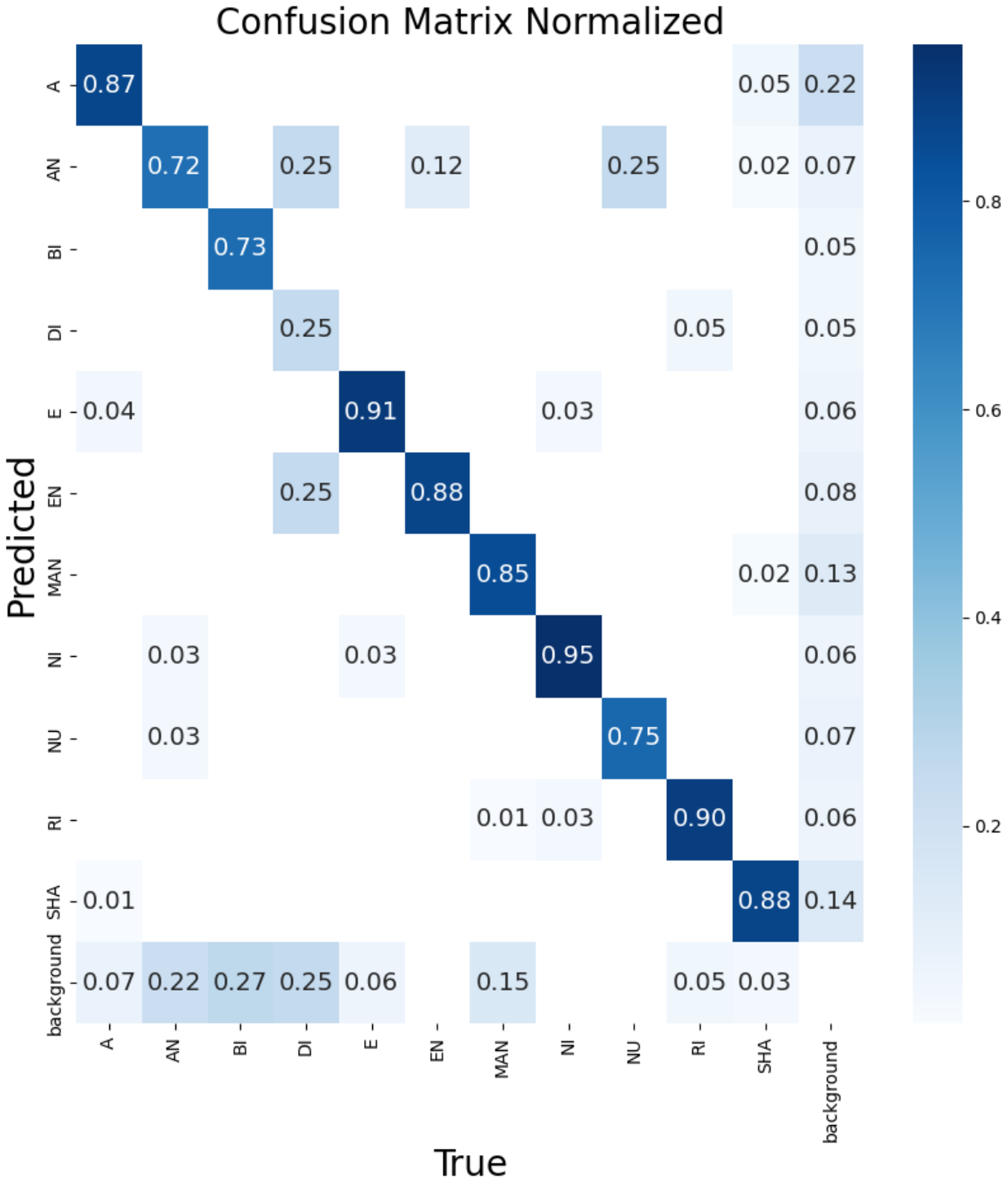
Table 4.5: The best evaluation results after training

Version	**Epochs**	**mAP50**	**Map50-90**	Precision	Recall
1	100	63.6%	45.115%	75.62%	58.66%
2	50	82.7%	61.2%	71.319%	85.647%
3	100	76.5%	53.7%	71.74%	79.412%

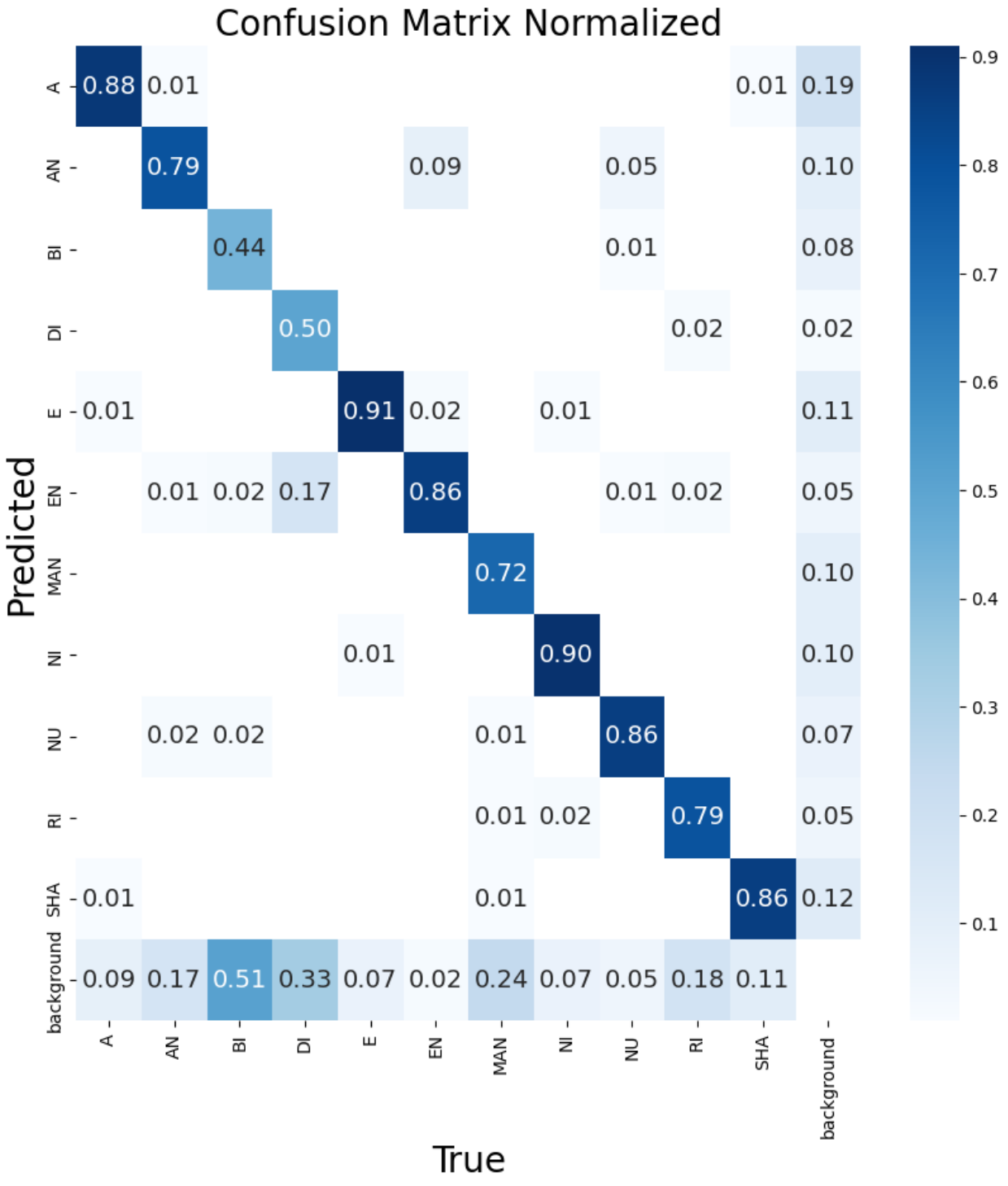
The confusion matrix for three iterations of the split dataset is depicted in Figure 4.8.


Confusion Matrix Normalized
Predicted
True
A
AN
BI
DI
E
EN
MAN
NI
NU
RI
SHA
background
0.88 0.01 0.01 0.19
0.79 0.09 0.05 0.10
0.44 0.01 0.08
0.50 0.02 0.02
0.01 0.91 0.02 0.01 0.11
0.01 0.02 0.17 0.86 0.01 0.02 0.05
0.72 0.10
0.01 0.90 0.10
0.02 0.02 0.01 0.86 0.07
0.01 0.02 0.79 0.05
0.01 0.01 0.86 0.12
0.09 0.17 0.51 0.33 0.07 0.02 0.24 0.07 0.05 0.18
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1

(A) Version 1


Confusion Matrix Normalized
Predicted
True
A
AN
BI
DI
E
EN
MAN
NI
NU
RI
SHA
background
0.87 0.05 0.22
0.72 0.25 0.12 0.25 0.02 0.07
0.73 0.05
0.25 0.05 0.05
0.04 0.91 0.03 0.06
0.25 0.88 0.08
0.85 0.02 0.13
0.03 0.03 0.95 0.06
0.03 0.75 0.07
0.01 0.03 0.90 0.06
0.01 0.88 0.14
0.07 0.22 0.27 0.25 0.06 0.15 0.05 0.03
0.8
0.6
0.4
0.2

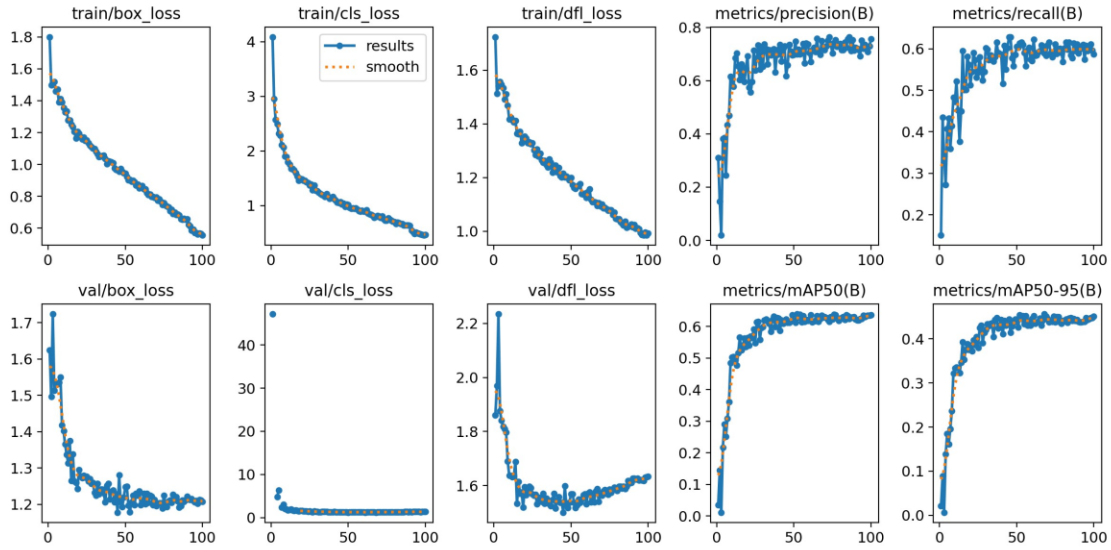
(B) Version 2



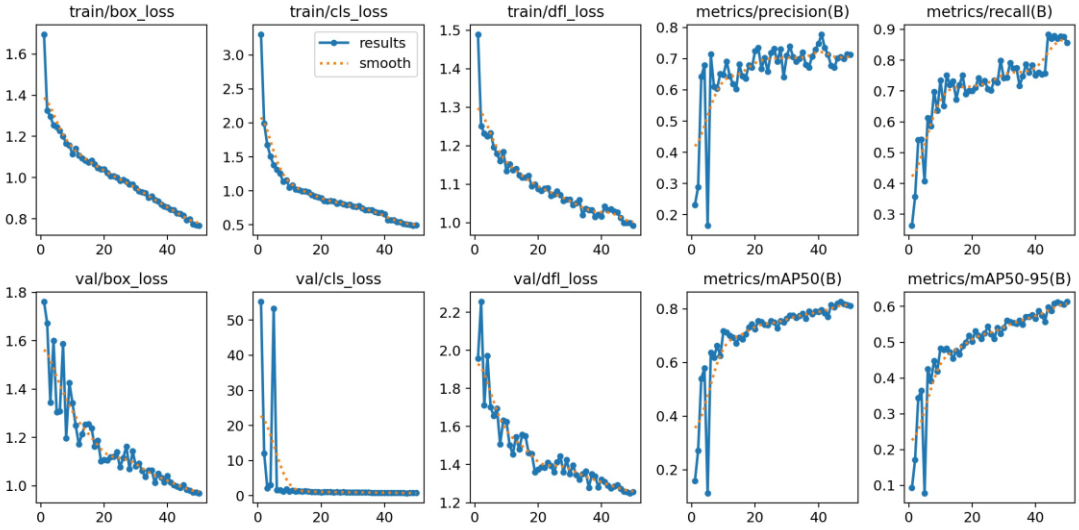
(C) Version 3

Figure 4.8: Confusion Matrix for three versions (V) of the split dataset. (A)Version 1, (B) Version 2, and (C) Version 3

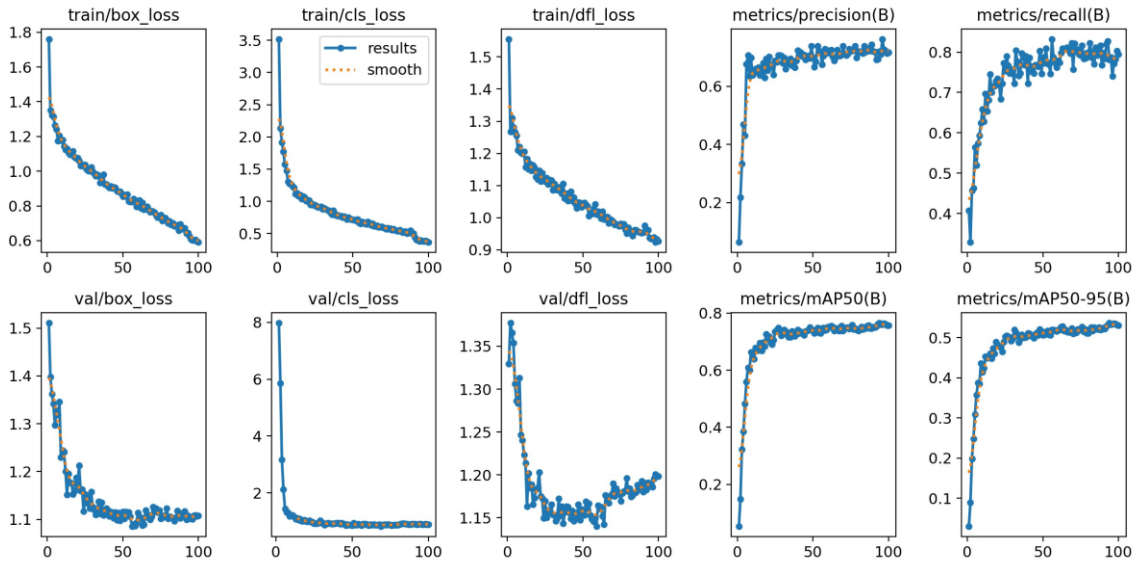
Yolo object detection employs three distinct loss functions: box-loss, which pertains to the regression of the bounding box and is measured using MSE; obj-loss, which focuses on the confidence of object losses; and cls-loss, which involves the classification loss and is evaluated using cross-entropy. The validation box-loss, cls-loss, and df1-loss values for three versions, namely V1, V2, and V3, are as follows: V1; 1.2078 – 1.3489 – 1.633, V2; 0.97018 – 0.79408 – 1.2563, and V3; 1.1075, 0.89497, 1.1982. Figure 4.9 displays the loss, mean average precision (mAP), precision, and recall outcomes of the three variants in the form of plotted graphs.



(A) Version 1



(B) Version 2

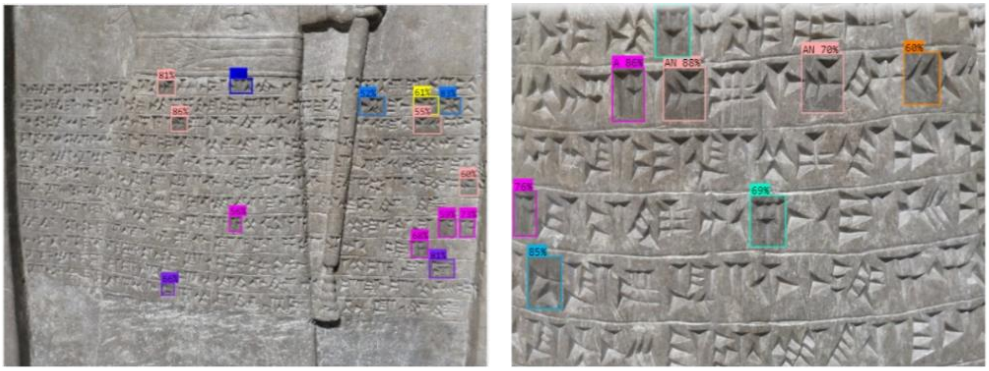


(C) Version 3

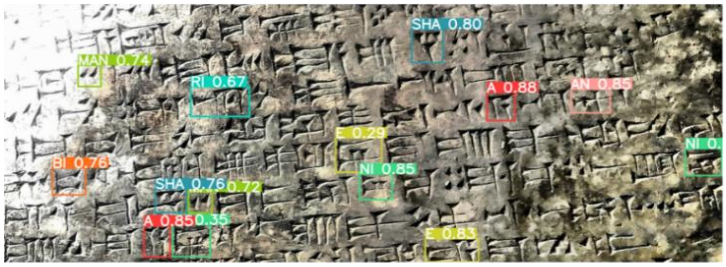
Figure 4.9: The Loss, mAP, Precision, and Recall results for three versions. (A)Version 1, (B) Version 2, and (C) Version 3

4.4.2 Prediction Results

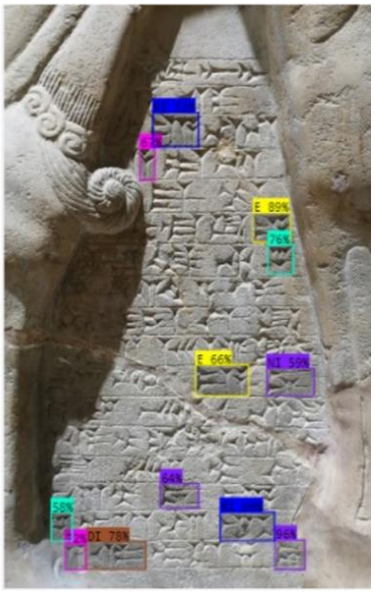
It is evident that the accuracy of the results is consistently observed in instances where the image possesses a high level of resolution and the indicators being analyzed are unambiguous. As the sharpness of the image diminishes, and the size of the indicators lowers, the accuracy also experiences a loss. The visual representation depicted in Figure 4.10 exhibits a considerable distance, with the indicators being less significant. Upon conducting tests on the V1, it becomes evident that not all signals are detected, as specific signals remain undetected. The outcomes demonstrate exceptional performance in identifying and categorizing the signals. The optimal outcome achieved in the second iteration is a mean average precision at 50% intersection over union of 82.7%.



a) Version 1 Prediction



b) Version 2 Prediction



c) Version 3 Prediction

Figure 4.10: The results prediction for three versions of Neo-Assyrian: a) Version one (V1), b) Version two (V2), and c) Version three (V3)

4.5 THE EVALUATION OF Hebrew Letters DETECTION MODEL

This is present the losses curves and annotation results of Hebrew letters detection results.

4.5.1 Dataset Training and Evaluation Results

Figure 4.11 displays a collection of images with corresponding labels used for training purposes. Additionally, the figure also presents the validation outcomes.

(A) **TRAIN** (B) **VALIDATION**

Figure 4.11: The pictures with the labels of the batch. (A) trained and (B) validation

The largest number for annotation is 2381 for the letter Vav, and the smallest number is 123 for the letter Tet. **Figure 4.12** shows the number of annotations reached after labeling by the Roboflow platform.

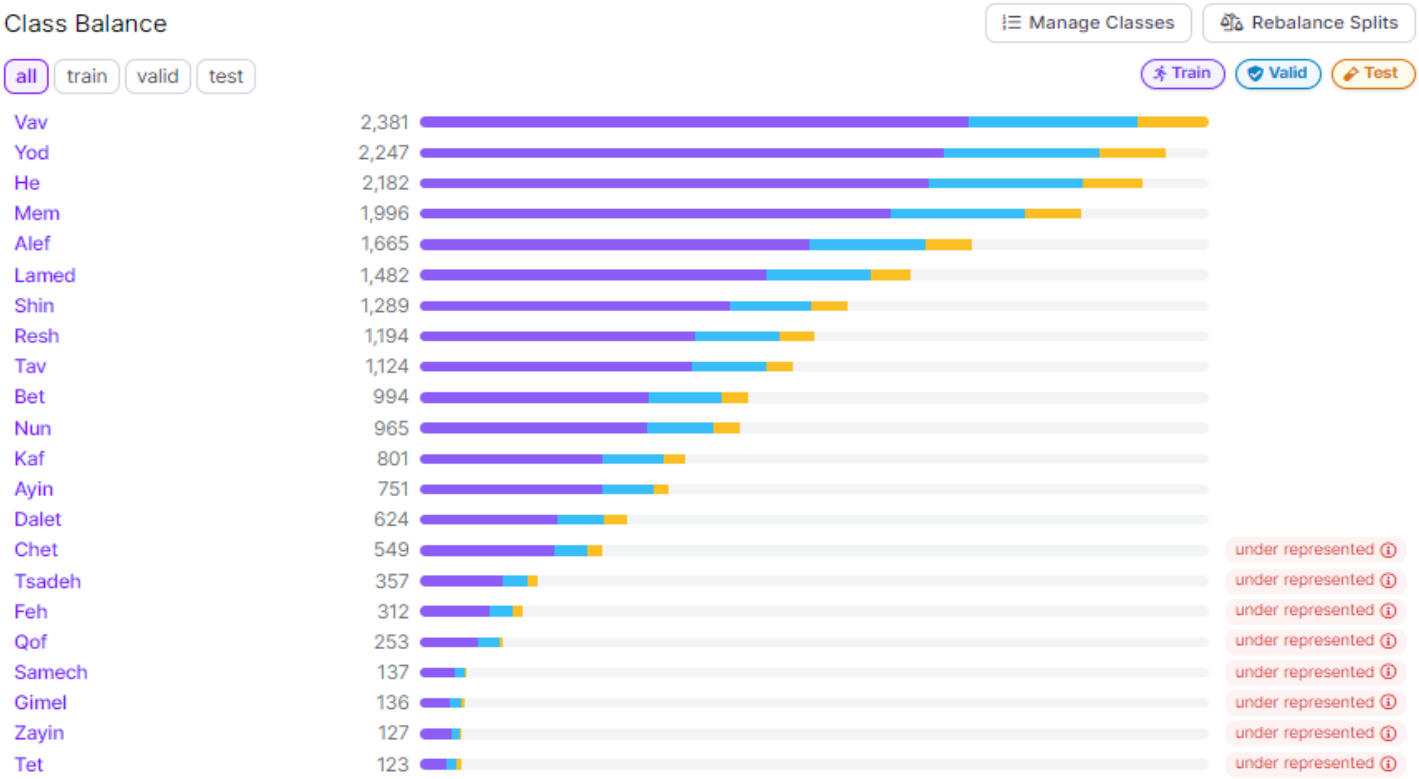


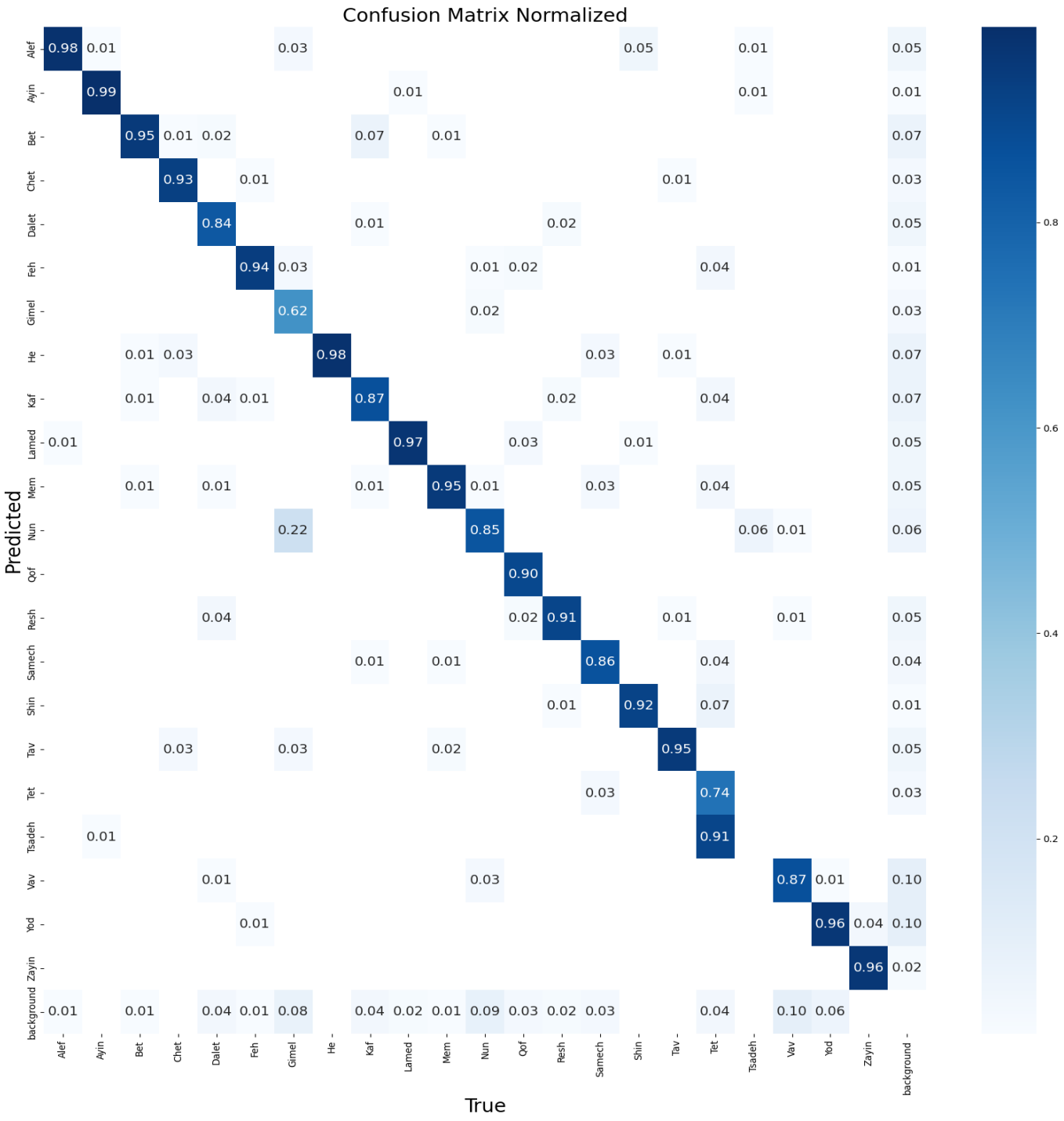
Figure 4.12: Number of annotations to each letter

The most notable outcome achieved in the second edition of the experiment was the mean average precision at 50% intersection over union (mAP50) of 92%, the mAP50-90 of 73.5%, a precision rate of 93.2%, and a recall rate of 89.8%, all of which were obtained after 100 epochs. The most favourable evaluation outcomes following the completion of training are presented in Table 4.6.

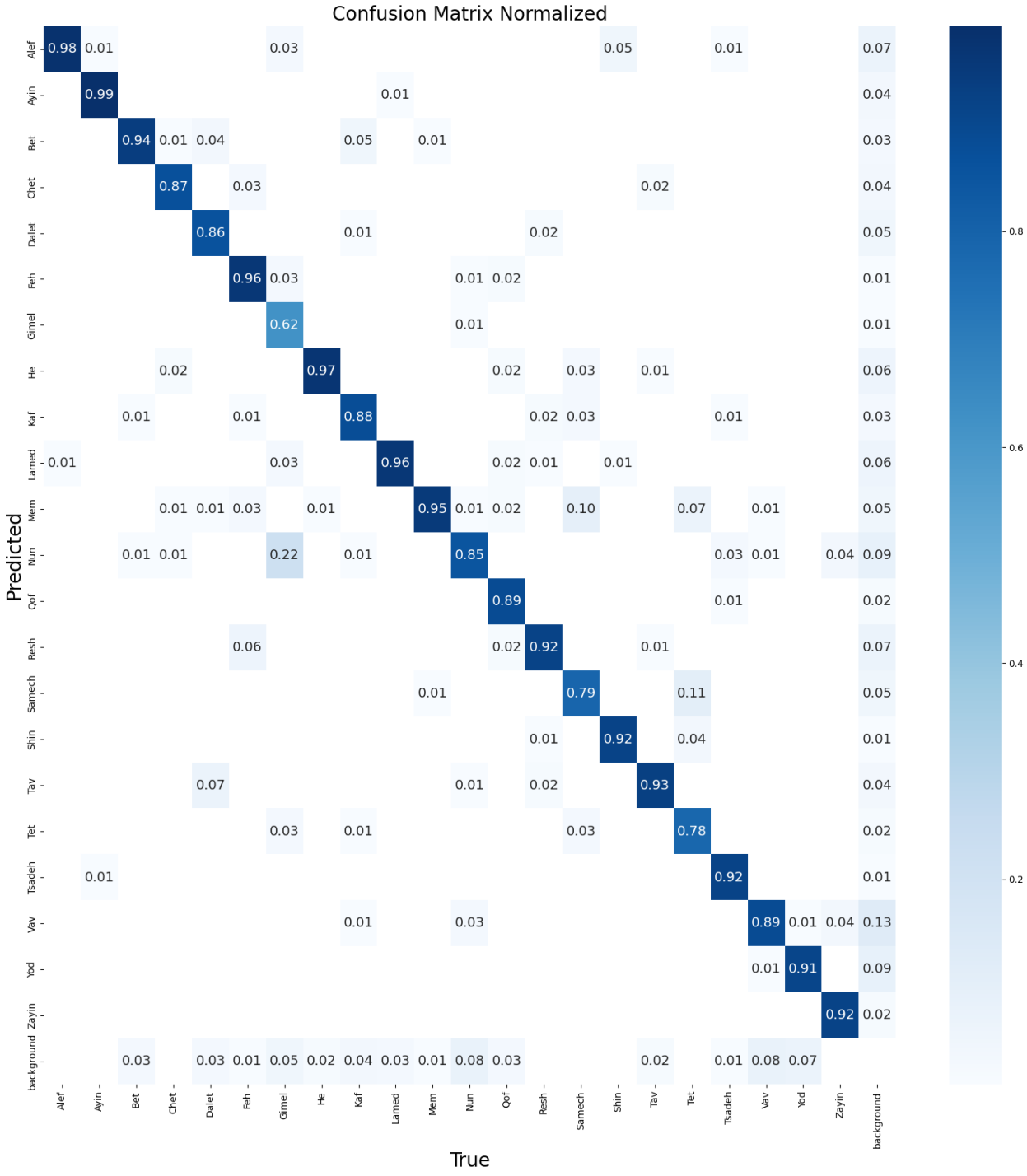
Table 4.6: The best evaluation results after training

Version	**Epochs**	**mAP50**	**Map50-90**	Precision	Recall
1	100	92%	73.5%	93.2%	89.8%
2	100	91.3%	72.5%	92.6%	88.9%

The confusion matrix for three iterations of the split dataset is depicted in Figure 4.13.


Confusion Matrix Normalized
Predicted
True
Alef
Ayin
Bet
Chet
Dalet
Feh
Gimel
He
Kaf
Lamed
Mem
Nun
Qof
Resh
Samech
Shin
Tav
Tet
Tsadeh
Vav
Yod
Zayin
background
0.8
0.6
0.4
0.2

(A) Version 1

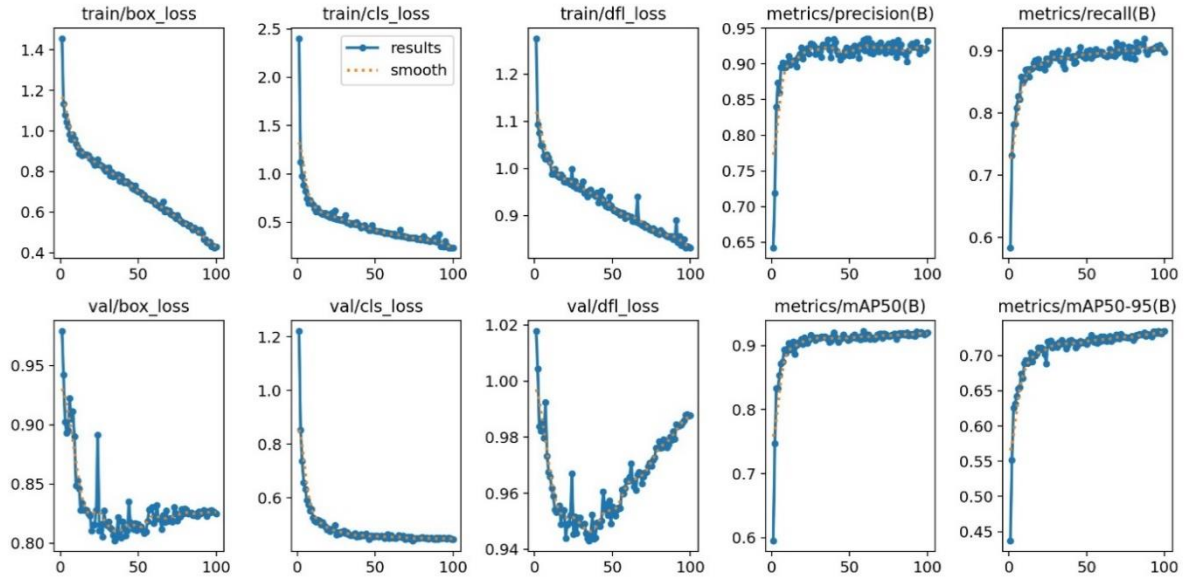


(B)Version 2

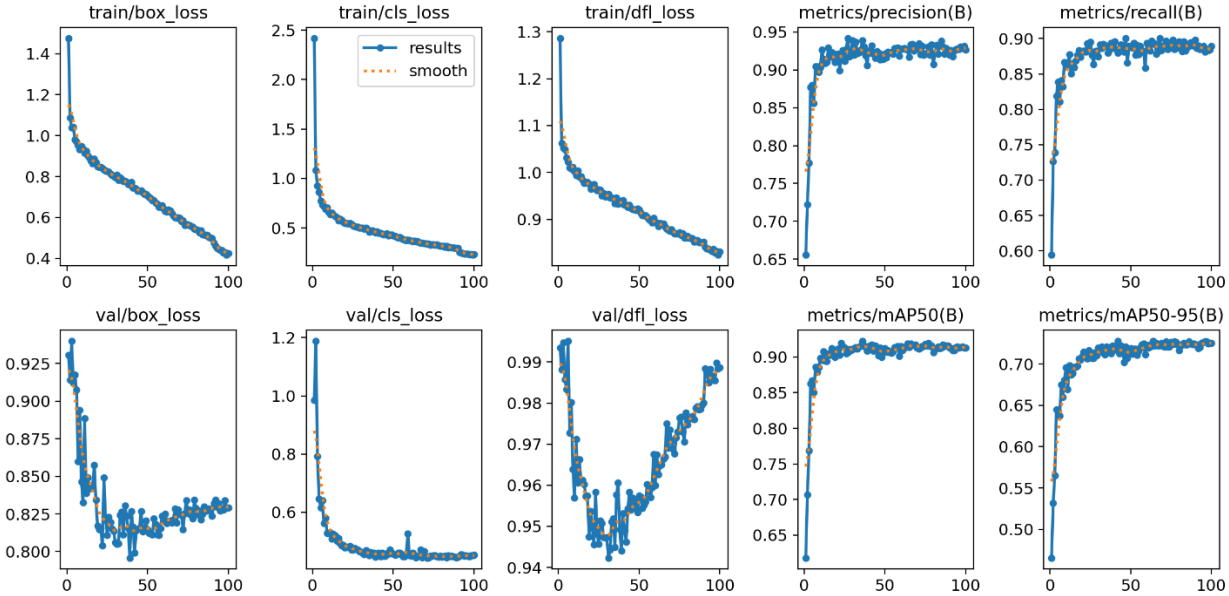
Figure 4.13: Confusion Matrix for three versions (V) of the split dataset

The validation box-loss, cls-loss, and df1-loss values for three versions, namely V1, and V2, are as follows: V1; 0.82475 – 0.44619 – 0.98768, and V2; 0.82918 – 0.45316 – 0.98868.

Figure 4.14 displays the loss, mean average precision (mAP), precision, and recall outcomes of the three variants in the form of plotted graphs.



(A)Version 1

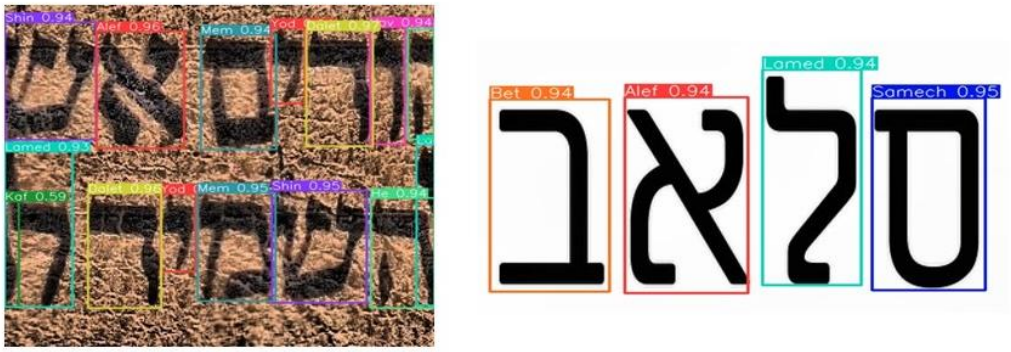


(B) Version 2

Figure 4.14: The Loss, mAP, Precision, and Recall results for two versions. (A) Version 1 and (B) Version 2

4.5.2 Prediction Results

The results are accurate whenever the image is high resolution and the signs are unmistakable. The accuracy begins to decline as the clarity of the image decreases and the size of the signs becomes smaller. The images in Figure 4.15 are the results after version one and version two training. All results are excellent in the detection and labelling of the signals. The best result in version one is mAP50=92%.



a) Version 1 Prediction

b) Version 2 Prediction

Figure 4.15: The results prediction for two versions of Hebrew: a) Version one (V1), and b) Version two (V2)

4.6 DISCUSSION

The first section successfully demonstrates the ability to distinguish languages and dialects within cuneiform inscriptions transcribed using Unicode characters. A dataset consisting of multiple algorithms was utilized in the fields of ML and NLP to assess the effectiveness of the Var-Dial evaluation campaign. This thesis has provided

substantial evidence of its effectiveness in detecting cuneiform languages. Satisfactory outcomes were attained regarding the metrics of accuracy, sensitivity, and precision, which were measured at 0.96%, 0.96%, and 0.96%, respectively. This endeavor has the potential to contribute to the advancement of scholarly pursuits and scientific investigations.

The classification model was classified between the three cuneiform languages: Assyrian, Sumerian, and Babylonian. A good result was obtained after training, with high accuracy, including top1 97% and top5 100%. To increase the accuracy of the classification model and increase the accuracy of training, it is trained on more diverse data. Thus, the classification tolerates different data and reduces error with data that is not trained on.

For Neo-Assyrian and Hebrew signs detection, the limitation imposed by the minimal number of collected photographs constrains the outcomes. To optimize the efficacy of the discovery process, it is recommended that each category be comprised of a minimum of 1,500 images. If the photograph has distinct markers, then the definition warrants commendation, and the same approach applies when the photograph is captured nearby. When photographs exhibit fuzzy signals or are captured at a considerable distance, the resolution level diminishes progressively, directly correlating to the increase in distance or the deterioration of image quality. The highest achieved performance was 82.7% after 50 epochs for the detection of the Neo-Assyrian signs and 92% for Hebrew letters detection, regarded as satisfactory. However, further enhancements are required by augmenting the supporting evidence.

There are a limited number of studies on the Hebrew language. A study closely related to our research involved using four types of CNNs to recognize Hebrew letters. The study also included assigning distinct pronunciations to each letter, resulting in an impressive accuracy rate of 99.96%.

Table 4.7 shows the Comparative Results

Table 4.7: Comparative Results

NO.	Literature Review models	The Models Have been Made with Accuracy
1	[24] 2023 The accuracy results: SVM (88%), KNN (88%), DT (94%), RF (95%), DNN (93%).	The accuracy result: SVM (88%), KNN (88%), NB (55%), RF (96%), GB (79%), AdaBoost (62%), XGBoost (79%), DT (94%), and LR (69%)
2	[18] 2020 SVM (0.7309), NB (0.7301), RF (0.5615), LR (0.6982), CNN (0.6249), RNN (0.6364)	
3	[20] 2020 CNN for Hebrew letter identification (The LetNet-5 has the highest accuracy for validation at 97,48%, and for Testing is 97.48%)	Detection of Hebrew letters using Yolov8 (92%).
4	[19] 2022 Yolov5 (42,81 %) Detecto (81,73 %)	Neo-Assyrian signs detection using Yolov8 (82.7%).
5	[12] 2020 CNNs (65.6 %)	

Chapter 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

Objective:

- Discern and identify cuneiform languages or dialects within writings obtained from the Kaggle website.
- Data compiled and arranged in a CSV file from the Orrac website, including cuneiform signs and their Unicode representations.
- Great dataset for classification of cuneiform tablets and using yolov8 to make classification.
- Great dataset for detecting Neo-Assyrian signs and using yolov8 to make detection.
- Great dataset for detecting Hebrew letters and using yolov8 to make detection.

Approach:

- Employed several machine learning algorithms and achieved satisfactory outcomes.
- Incorporated a broader range of machine learning algorithms and an N-Gram natural language processing algorithm.
- These approaches yielded favourable findings for researchers.

YOLOv8 Implementation:

- Used YOLOv8 to identify Neo-Assyrian signs and Hebrew letters.

- YOLOv8, developed in 2023, was utilized for this purpose and regarded as exemplary.
- Achieved satisfactory outcomes in gathering photographs from the Iraqi Museum of Neo-Assyrian tablets.

Data Processing:

- Procedure involved labeling, pre-processing, and augmenting the dataset.
- Accumulated almost 900 photos of Neo-Assyrian tablets.
- Thesis achieved the potential for identifying 11 Assyrian signals.
- For Hebrew writing, satisfactory results were reached by collecting formal data (471 samples).
- After pre-processing and augmentation, 21,699 annotations were reached.

Further Applications:

- The YOLOv8 model was also used to categorize cuneiform tablets.
- Generated a dataset comprising cuneiform tablets from Assyrian, Sumerian, and Babylonian languages.
- Yielded a substantial number of images for each language, surpassing 1000 images.
- Achieved commendable outcomes.

5.2 FUTURE WORK

The initial segment of the thesis involved training several artificial intelligence systems on distinct cuneiform writings. This enhances and aids in distinguishing among old manuscripts. These training exercises can be integrated into websites or applications to enter letters or symbols and classify them into whatever language or dialect they belong to.

To enhance the accuracy of cuneiform language recognition in future research, developing a more refined dataset and ensuring its balance is recommended, reducing the error rate associated with the extraction process. Additionally, it is advisable to employ alternative algorithms that are more resilient to enhance the precision of the outcomes. There is room for improvement regarding the dataset utilized by augmenting it with additional textual content to get a more balanced representation.

It is imperative to augment the dataset size and employ diverse neural network architectures to enhance the accuracy of Neo-Assyrian sign and Hebrew letter recognition and cuneiform tablet classification. One such architecture is the Region-Based Convolutional Neural Networks (RCNN) with selective search, which has demonstrated superior detection rates in numerous cases. In addition, it is possible to initiate the optimization of neural network architectures, such as CNNs, to achieve optimal accuracy and facilitate more effective evaluation in light of newly annotated data.

Several panels are damaged. Efforts will be made to address the areas that have been damaged and finalize the text.

In Hebrew, vowels and consonants are distinguished by a distinct diacritic appended to each. Also make translation Hebrew to Arabic.

Given that the classification and detection systems were developed independently, efforts will be made to merge them into a single program.

One effective approach to enhance classification accuracy and expedite the procedure is to utilize distinct signs that differentiate each cuneiform language. This strategy will be implemented in future classification efforts.

REFERENCES

[1] A. Sahala, M. Silfverberg, A. Arppe, and K. Lindén, "BabyFST-towards a finite-state based computational model of ancient babylonian," in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2020, pp. 3886–3894.

[2] Ali .A, Abdul Monem .S, and Abdul Mohssen .j, "Cuneiform symbols recognition by support vector machine (SVM)," *Journal of Al-Qadisiyah for computer science and mathematics*, vol. 11, no. 1, pp. 1–11, 2019, doi: 10.29304/jqcm.2019.11.1.449.

[3] Y. Assael, T. Sommerschield, and J. Prag, "Restoring ancient text using deep learning: a case study on Greek epigraphy," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 6368–6375.

[4] R. Abitbol, I. Shimshoni, and J. Ben-Dov, "Machine Learning Based Assembly of Fragments of Ancient Papyrus," *Journal on Computing and Cultural Heritage*, vol. 14, no. 3, Jul. 2021, doi: 10.1145/3460961.

[5] J. Luo, Y. Cao, and R. Barzilay, "Neural decipherment via minimum-cost flow: From ugaritic to linear B," in *ACL 2019 - 57th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference*, Association for Computational Linguistics (ACL), 2020, pp. 3146–3155. doi: 10.18653/v1/p19-1303.

[6] A. Sahala, M. Silfverberg, A. Arppe, and K. Linden, "Automated Phonological Transcription of Akkadian Cuneiform Text," in *Language Resources and Evaluation Conference*, European Language Resources Association (ELRA), 2020, pp. 3528–3534.

[7] S. Gordin, G. Gutherz, A. Elazary, A. Romach, E. Jiménez, and J. Berant, “Reading Akkadian cuneiform using natural language processing,” *PLoS One*, vol. 15, no. 10, p. e0240511, 2020.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Commun ACM*, vol. 60, no. 6, pp. 84–90, Jun. 2017, doi: 10.1145/3065386.

[9] S. K. Zhou, H. Greenspan, C. Davatzikos, J. S. Duncan, B. Van Ginneken, and A. Madabhushi, “A Review of Deep Learning in Medical Imaging: Imaging Traits, Technology Trends, Case Studies with Progress Highlights, and Future Promises,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 820–838, May 2021, doi: 10.1109/JPROC.2021.3054390.

[10] T. Hodel, D. Schoch, C. Schneider, and J. Purcell, “General Models for Handwritten Text Recognition: Feasibility and State-of-the Art. German Kurrent as an Example,” *Journal of Open Humanities Data*, vol. 7, Jul. 2021, doi: 10.5334/johd.46.

[11] L. Buitinck, G. Louppe, and M. Blondel, “API design for machine learning software: experiences from the scikit-learn project,” in *ECML/PKDD 2013 Workshop: Languages for Data Mining and Machine Learning*, 2013.

[12] T. Dencker, P. Klinkisch, S. M. Maul, and B. Ommer, “Deep learning of cuneiform sign detection with weak supervision using transliteration alignment,” *PLoS One*, vol. 15, no. 12 December, Dec. 2020, doi: 10.1371/journal.pone.0243039.

[13] R. Punia, N. Schenk, C. Chiarcos, and É. Pagé-Perron, “Towards the first machine translation system for Sumerian transliterations,” in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 3454–3460.

[14] A. M. S. Rahma, A. A. Saeid, and M. J. A. Hussien, “Recognize assyrian cuneiform characters by virtual dataset,” in *2017 6th International Conference on Information and Communication Technology and Accessibility (ICTA)*, IEEE, 2017, pp. 1–7.

[15] H. Lahoud, D. L. Share, and A. Shechter, “A developmental study of eye movements in Hebrew word reading: the effects of word familiarity, word length, and reading proficiency,” *Front Psychol*, vol. 14, 2023, doi: 10.3389/fpsyg.2023.1052755.

[16] D. Fisseler, F. Weichert, and M. Cammarosano, “Towards an interactive and automated script feature analysis of 3D scanned cuneiform tablets,” *4th Conference Scientific Computing and Cultural Heritage*, p. 16, 2013.

[17] T. Jauhiainen, H. Jauhiainen, T. Alstola, and K. Lindén, “Language and Dialect Identification of Cuneiform Texts,” in *Proceedings of the Sixth Workshop on NLP for Similar Languages, Varieties and Dialects*, 2019, pp. 89–98.

[18] E. Doostmohammadi and M. Nassajian, “Investigating Machine Learning Methods for Language and Dialect Identification of Cuneiform Texts,” Sep. 2020, doi: 10.18653/v1/W19-1420.

[19] A. Hamplová, D. Franc, J. Pavlíček, A. Romach, and S. Gordin, “Cuneiform Reading Using Computer Vision Algorithms,” in *Proceedings of the 2022 5th International Conference on Signal Processing and Machine Learning*, 2022, pp. 242–245.

[20] T. L. Tobing, S. Y. Yayilgan, S. George, and T. Elgvin, “Isolated Handwritten Character Recognition of Ancient Hebrew Manuscripts,” in *Archiving 2022: Expanding Connections Across Digital Cultural Heritage - Final Program and Proceedings*, Society for Imaging Science and Technology, 2022, pp. 35–39. doi: 10.2352/issn.2168-3204.2022.19.1.08.

[21] E. Stötzner, T. Homburg, J. P. Bullenkamp, and H. Mara, “R-CNN based Polygonal Wedge Detection Learned from Annotated 3D Renderings and Mapped Photographs of Open Data Cuneiform Tablets,” 2023, doi: 10.2312/gch.20231157.

[22] E. Stötzner, T. Homburg, and H. Mara, “CNN based Cuneiform Sign Detection Learned from Annotated 3D Renderings and Mapped Photographs with Illumination Augmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 1680–1688.

[23] G. Gutherz, S. Gordin, L. Sáenz, O. Levy, and J. Berant, “Translating Akkadian to English with neural machine translation,” *PNAS Nexus*, vol. 2, no. 5, May 2023, doi: 10.1093/pnasnexus/pgad096.

[24] M. Mahmood, F. M. Jasem, A. A. Mukhlif, and B. Al-Khateeb, “Classifying cuneiform symbols using machine learning algorithms with unigram features on a balanced dataset,” *Journal of Intelligent Systems*, vol. 32, no. 1, Jan. 2023, doi: 10.1515/jisys-2023-0087.

[25] Ralf Herbrich and Thore Graepel, *Machine Learning An Algorithmic Perspective*, Second. Chapman and Hall/CRC, 2015.

[26] M. Mohammed, M. B. Khan, and E. B. M. Bashie, *Machine learning: Algorithms and applications*. CRC Press, 2016. doi: 10.1201/9781315371658.

[27] L. P. Coelho and W. Richert, *Building machine learning systems with Python : get more from your data through creating practical machine learning systems with Python*, Second. Packt, 2015.

[28] Batta Mahesh, “Machine Learning Algorithms-A Review,” *International Journal of Science and Research (IJSR)*, pp. 1–6, 2019, doi: 10.21275/ART20203995.

[29] I. H. Sarker, A. S. M. Kayes, S. Badsha, H. Alqahtani, P. Watters, and A. Ng, "Cybersecurity data science: an overview from machine learning perspective," *J Big Data*, vol. 7, no. 1, Dec. 2020, doi: 10.1186/s40537-020-00318-5.

[30] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[31] M. Maalouf, "Logistic regression in data analysis: an overview," 2011.

[32] I. H. Sarker, "A machine learning based robust prediction model for real-life mobile phone data," *Internet of Things*, vol. 5, pp. 180–193, 2019.

[33] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy, "Improvements to Platt's SMO algorithm for SVM classifier design," *Neural Comput*, vol. 13, no. 3, pp. 637–649, 2001.

[34] J. Han, M. Kamber, and J. Pei, "Data Mining. Concepts and Techniques, 3rd Edition (The Morgan Kaufmann Series in Data Management Systems)," 2011.

[35] V. Jakkula, "Tutorial on Support Vector Machine (SVM)," pp. 1–13, 2006.

[36] Y. Goldberg, "A Primer on Neural Network Models for Natural Language Processing," 2016.

[37] U. Anders and O. Korn, "Model selection in neural networks," *the official journal of the International Neural Network Society*, vol. 12, no. 2, pp. 309–323, 1999.

[38] D. W. Aha, D. Kibler, M. K. Albert, and J. R. Quinian, "Instance-Based Learning Algorithms," *Mach Learn*, vol. 6, pp. 37–66, 1991.

[39] S. Sun and R. Huang, "An adaptive k-nearest neighbor algorithm," in *2010 seventh international conference on fuzzy systems and knowledge discovery*, IEEE, 2010, pp. 91–94.

[40] D. T. Larose and C. D. Larose, *k-nearest neighbor algorithm*, Second. Wiley Data and Cybersecurity, 2014. doi: 10.1002/9781118874059.ch7.

[41] F. Huang, G. Xie, and R. Xiao, "Research on ensemble learning," in *2009 International Conference on Artificial Intelligence and Computational Intelligence, AICI 2009*, 2009, pp. 249–252. doi: 10.1109/AICI.2009.235.

[42] L. Breiman, "Random Forests," *Springer*, vol. 45, pp. 5–32, 2001.

[43] I. H. Sarker, A. S. M. Kayes, and P. Watters, "Effectiveness analysis of machine learning classification models for predicting personalized context-aware smartphone usage," *J Big Data*, vol. 6, no. 1, Dec. 2019, doi: 10.1186/s40537-019-0219-y.

[44] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," *Artif Intell Rev*, vol. 54, pp. 1937–1967, 2021.

[45] K. Chowdhary and K. R. Chowdhary, "Natural language processing," *Fundamentals of artificial intelligence*, pp. 603–649, 2020.

[46] P. M. Nadkarni, L. Ohno-Machado, and W. W. Chapman, "Natural language processing: An introduction," *Journal of the American Medical Informatics Association*, vol. 18, no. 5. pp. 544–551, Sep. 2011. doi: 10.1136/amiajnl-2011-000464.

[47] P. Majumder, M. Mitra, and B. B. Chaudhuri, "N-gram: a language independent approach to IR and NLP," in *In International conference on universal knowledge and language*, 2002.

[48] H. J. Escalante, T. Solorio, and M. Montes-Y-Gómez, "Local Histograms of Character N-grams for Authorship Attribution," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, Jun. 2011, pp. 288–298.

[49] M. Jankowska, E. Milios, and V. Kešelj, “Author Verification Using Common N-Gram Profiles of Text Documents,” in *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, 2014, pp. 387–397.

[50] M. Kuta and J. Kitowski, “Optimisation of character n-gram profiles method for intrinsic plagiarism detection,” in *Artificial Intelligence and Soft Computing: 13th International Conference, ICAISC 2014, Zakopane, Poland, June 1-5, 2014, Proceedings, Part II 13*, Springer, 2014, pp. 500–511.

[51] I. Kanaris, K. Kanaris, I. Houvardas, and E. Stamatatos, “WORDS VS. CHARACTER N-GRAMS FOR ANTI-SPAM FILTERING,” *International Journal on Artificial Intelligence Tools*, vol. 16, no. 6, pp. 1–20, 2006.

[52] S. Malmasi and M. Dras, “Language identification using classifier ensembles,” in *Proceedings of the joint workshop on language technology for closely related languages, varieties and dialects*, 2015, pp. 35–43.

[53] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching Word Vectors with Subword Information,” *Trans Assoc Comput Linguist*, vol. 5, pp. 135–146, 2017, doi: 10.1162/tacl_a_00051/1567442/tacl_a_00051.pdf.

[54] G. Giannakopoulos and V. Karkaletsis, “N-gram Graphs: Representing Documents and Document Sets in Summary System Evaluation.,” in *TAC*, Citeseer, 2009, pp. 1–10.

[55] Y. Lecun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553. Nature Publishing Group, pp. 436–444, May 27, 2015. doi: 10.1038/nature14539.

[56] Z. Zhang, P. Cui, and W. Zhu, “Deep learning on graphs: A survey,” *IEEE Trans Knowl Data Eng*, vol. 34, no. 1, pp. 249–270, 2020.

[57] A. Shrestha and A. Mahmood, “Review of deep learning algorithms and architectures,” *IEEE Access*, vol. 7. Institute of Electrical and Electronics Engineers Inc., pp. 53040–53065, 2019. doi: 10.1109/ACCESS.2019.2912200.

[58] M. M. Najafabadi, F. Villanustre, T. M. Khoshgoftaar, N. Seliya, R. Wald, and E. Muharemagic, “Deep learning applications and challenges in big data analytics,” *J Big Data*, vol. 2, no. 1, Dec. 2015, doi: 10.1186/s40537-014-0007-7.

[59] K. G. Kim, “Book Review: Deep Learning,” *Healthc Inform Res*, vol. 22, no. 4, p. 351, 2016, doi: 10.4258/hir.2016.22.4.351.

[60] J. Li and Z. Pan, “Network Traffic Classification Based on Deep Learning,” *KSII Transactions on Internet and Information Systems*, vol. 14, no. 11, pp. 4246–4267, Nov. 2020, doi: 10.3837/tiis.2020.11.001.

[61] A. Graves, A.-R. Mohamed, and G. Hinton, “SPEECH RECOGNITION WITH DEEP RECURRENT NEURAL NETWORKS,” in *2013 IEEE international conference on acoustics, speech and signal processing*, 2013, pp. 6645–6649.

[62] G. E. Hinton, “Learning multiple layers of representation,” *Trends Cogn Sci*, vol. 11, no. 10, pp. 1–21, Oct. 2007.

[63] S. Rezaei and X. Liu, “Deep Learning for Encrypted Traffic Classification: An Overview,” *IEEE Communications Magazine*, vol. 57, no. 5. Institute of Electrical and Electronics Engineers Inc., pp. 76–81, May 01, 2019. doi: 10.1109/MCOM.2019.1800819.

[64] Jing Ran, Yexin Chen, and Shulan Li, “THREE-DIMENSIONAL CONVOLUTIONAL NEURAL NETWORK BASED TRAFFIC CLASSIFICATION FOR WIRELESS COMMUNICATIONS,” in *In 2018 IEEE Global Conference on Signal and Information Processing (GlobalSIP)*, 2018, pp. 624–627.

[65] IEEE Computer Society., *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) : date, 7-12 June 2015.*

[66] A. Voulodimos, N. Doulamis, A. Doulamis, and E. Protopapadakis, "Deep Learning for Computer Vision: A Brief Review," *Computational Intelligence and Neuroscience*, vol. 2018. Hindawi Limited, 2018. doi: 10.1155/2018/7068349.

[67] L. H. Thai, T. S. Hai, and N. T. Thuy, "Image Classification using Support Vector Machine and Artificial Neural Network," *International Journal of Information Technology and Computer Science*, vol. 4, no. 5, pp. 32–38, May 2012, doi: 10.5815/ijitcs.2012.05.05.

[68] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," *Multimed Tools Appl*, vol. 82, no. 6, pp. 9243–9275, Mar. 2023, doi: 10.1007/s11042-022-13644-y.

[69] A. B. Amjoud and M. Amrouch, "Object detection using deep learning, CNNs and vision transformers: a review," *IEEE Access*, vol. 11, pp. 35479–35516, 2023, doi: 10.1109/ACCESS.2023.3266093.

[70] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, 2020, pp. 390–391.

[71] M. A. Zaytar and C. El Amrani, "Sequence to Sequence Weather Forecasting with Long Short-Term Memory Recurrent Neural Networks," 2016.

[72] L. Jiao *et al.*, "A survey of deep learning-based object detection," *IEEE Access*, vol. 7, pp. 128837–128868, 2019, doi: 10.1109/ACCESS.2019.2939201.

[73] A. C. Paiva, Universidade Federal Fluminense. Institute of Computing, Institute of Electrical and Electronics Engineers. Rio de Janeiro Section., and Institute of Electrical and Electronics Engineers., *Proceedings of the 2020 International Conference on Systems, Signals and Image Processing (IWSSIP) : July 1-3, 2020, Niterói, Brazil*.

[74] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas," *Mach Learn Knowl Extr*, vol. 5, no. 4, pp. 1680–1716, 2023.

[75] IEEE Robotics and Automation Society. and Institute of Electrical and Electronics Engineers, *2018 IEEE International Conference on Mechatronics and Automation : IEEE ICMA 2018 : August 5-8, 2018, Changchun, China.*

[76] M. Hussain, "YOLOv1 to v8: Unveiling Each Variant–A Comprehensive Review of YOLO," *IEEE Access*, vol. 12, pp. 42816–42833, 2024.

[77] Z. Li, C. Peng, G. Yu, X. Zhang, Y. Deng, and J. Sun, "Detnet: Design backbone for object detection," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 334–350.

[78] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2117–2125.

[79] "GitHub - ultralytics/ultralytics: NEW - YOLOv8 🚀 in PyTorch > ONNX > OpenVINO > CoreML > TFLite." Accessed: Oct. 05, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[80] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proceedings of the AAAI conference on artificial intelligence*, 2020, pp. 12993–13000.

[81] X. Li *et al.*, “Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection.”

[82] L. Shen, H. Tao, Y. Ni, Y. Wang, and V. Stojanovic, “Improved YOLOv3 model with feature map cropping for multi-scale road object detection,” *Meas Sci Technol*, vol. 34, no. 4, p. 045406, 2023.

[83] “YOLOv8 Object Detection Model.” Accessed: Oct. 06, 2023. [Online]. Available: https://roboflow.com/model/yolov8

[84] S. Tamang, B. Sen, A. Pradhan, K. Sharma, and V. K. Singh, “Enhancing covid-19 safety: Exploring yolov8 object detection for accurate face mask classification,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 11, no. 2, pp. 892–897, 2023.

[85] W. Hao and S. Zhili, “Improved mosaic: Algorithms for more complex images,” in *Journal of Physics: Conference Series*, IOP Publishing Ltd, Nov. 2020. doi: 10.1088/1742-6596/1684/1/012094.

[86] A. Annapoorani, N. S. Kumar, and V. Vidhya, “Blind-Sight: Object Detection with Voice Feedback,” *International Journal of Scientific Research & Engineering Trends*, vol. 7, no. 2, pp. 2395–566, 2021.

[87] T. Jauhiainen, M. Lui, M. Zampieri, T. Baldwin, and K. Lindén, “Automatic language identification in texts: A survey,” *Journal of Artificial Intelligence Research*, vol. 65. AI Access Foundation, pp. 675–782, 2019. doi: 10.1613/JAIR.1.11675.

[88] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, “Network Traffic Classifier with Convolutional and Recurrent Neural Networks for Internet of Things,” *IEEE Access*, vol. 5, pp. 18042–18050, Sep. 2017, doi: 10.1109/ACCESS.2017.2747560.

[89] C. J. Willmott and K. Matsuura, “Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance,” *Clim Res*, vol. 30, no. 1, pp. 79–82, 2005.

[90] T. Chai and R. R. Draxler, “Root mean square error (RMSE) or mean absolute error (MAE)?,” *Geosci. Model Dev. Discuss*, vol. 7, pp. 1525–1534, 2014, doi: 10.5194/gmdd-7-1525-2014.

[91] K. Shibata and Y. Ikeda, “Effect of number of hidden neurons on learning in large-scale layered neural networks,” in *2009 ICCAS-SICE*, IEEE, 2009, pp. 5008–5013.

[92] Z. Zhu, L. Xie, and A. Yuille, “Object recognition with and without objects,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, 2017, pp. 3609–3615.

[93] T. Ahmad, Y. Ma, M. Yahya, B. Ahmad, S. Nazir, and A. U. Haq, “Object Detection through Modified YOLO Neural Network,” *Sci Program*, vol. 2020, pp. 1–10, 2020, doi: 10.1155/2020/8403262.

المستخلص

تقدم الكتابة المسمارية رؤى قيمة حول العصور البعيدة من تاريخ البشرية. ومع ذلك، فإن تفسير الألواح الطينية القديمة يستغرق وقتًا طويلاً ويتطلب معرفة واسعة يتم اكتسابها على مدار عدة سنوات. يمثل تحديد اللغات التي تمثلها الرموز المسمارية تحديًا كبيرًا بسبب محدودية الموارد ومسألة الترميز.

استخدمت هذه الأطروحة عشرة تقنيات للتعلم الآلي، بما في ذلك التعلم الخاضع للإشراف والتجميع والتعلم القائم على المثيل، بالإضافة إلى خوارزمية واحدة لمعالجة اللغة الطبيعية (NLP) تسمىn-gram ، لتحديد اللهجة المسمارية. لذلك، للكشف عن العلامات الموجودة في الألواح المسمارية الآشورية، تم استخدام نموذج التدريب المسبق للكشف عن الكائنات "أنت تنظر مرة واحدة فقط" الإصدار 8.(Yolov8)

تنتج المجموعات أفضل النتائج لمصنف Random Forest، حيث تعمل على الميزات على مستوى الشخصية، مع متوسط ماكرو F1-Score يبلغ 96%، وأفضل نتيجة لخوارزمية n-grams هي 82% من bi-gram. أفضل mAP50 وصل إلى 82.7%، ودقة 71.319%، و % 85.647 recall الكشف عن الالواح الآشورية بمساعدة الكمبيوتر في صور الألواح المسمارية التي تم جمعها من المتحف العراقي. وهكذا، ايضا تم التعرف على 22 حرفًا عبري ووصلت إلى أفضل متوسط لنسبة مئوية قدرها 50 (وصل mAP50 إلى 92%، ودقة 93.2%، و 89.8% recall). بالنسبة للتصنيف، تم جمع الألواح المسمارية من موقع المتحف العراقي ومبادرة المكتبة الرقمية المسمارية (CDLI) وتصل إلى ما يقارب أكثر من 1000 صورة في كل لغة، top1 97% وtop5 100%.

تصنيف علامات اللغات القديمة باستخدام التعلم الآلي

رسالة

مقدمة الى كلية هندسة المعلومات في جامعة النهرين كجزء من متطلبات نيل

درجة

ماجستير

في

هندسة المنظومات

من قبل

ايلاف أحمد سعيد عبد الجبار

(بكالوريوس في هندسة المنظومات 2018)

ذو الحجة 1445

حزيران 2024